Java™ Data Objects

JSR 000012

Version 1.0

Proposed Final Draft

Java Data Objects Expert Group

Specification Lead: **Craig Russell,**
Sun Microsystems Inc.

Technical comments:
jdo-comments@eng.sun.com
Process comments:
community-process@sun.com

Forte Tools
Sun Microsystems, Inc.
901 San Antonio Road
Palo Alto, CA 94043 U.S.A.
650 960-1300 fax: 650 969-9131

Acknowledgments

I have come to know Rick Cattell during many shared experiences in the Java database standards arena. Rick is a Distinguished Engineer at Sun Microsystems and has been the database guru and Enterprise Cardinal in the Java “Church” for many years. I am deeply in his debt for his many contributions to JDO, both technical and organizational.

I want to thank the experts on the JDO expert group who contributed ideas, APIs, feedback, and other valuable input to the standard, especially Heiko Bobzin, Constantine Plotnikov, Luca Garulli, Philip Conroy, Steve Johnson, Michael Birk, Michael Rowley, Gordan Vosicki, and Martin McClure.

I want to recognize Michael Bouschen, David Jordan, and Jeff Norton for their careful review of JDO for consistency, readability, and usability. Without their contributions, JDO would not have been possible.

Table of Contents

Table of Contents

Table of Contents

Table of Contents

Table of Contents

Table of Contents

Table of Contents

Table of Contents

List of Figures

1 Introduction

Java is a language that defines a runtime environment in which user-defined classes execute. The instances of these user-defined classes might represent real world data. The data might be stored in databases, file systems, or mainframe transaction processing systems. These data sources are collectively referred to as Enterprise Information Systems (EIS). Additionally, small footprint environments often require a way to manage persistent data in local storage.

The data access techniques are different for each type of data source, and accessing the data presents a challenge to application developers, who currently need to use a different Application Programming Interface (API) for each type of data source.

This means that application developers need to learn at least two different languages in order to develop business logic for these data sources: the Java programming language; and the specialized data access language required by the data source.

Currently, there are two Java standards for storing Java data persistently: serialization and JDBC. Serialization preserves relationships among a graph of Java objects, but does not support sharing among multiple users. JDBC requires the user to explicitly manage the values of fields and map them into relational database tables.

Developers can be more productive if they focus on creating Java classes that implement business logic, and use native Java classes to represent data from the data sources. Mapping between the Java classes and the data source, if necessary, can be done by an EIS domain expert.

JDO defines interfaces and classes to be used by application programmers when using classes whose instances are to be stored in persistent storage (persistence-capable classes), and specifies the contracts between suppliers of persistence-capable classes and the runtime environment (which is part of the JDO Implementation).

The supplier of the JDO Implementation is hereinafter called the JDO vendor.

1.1 Overview

There are two major objectives of the JDO architecture: first, to provide application programmers a transparent Java-centric view of persistent information, including enterprise data and locally stored data; and second, to enable pluggable implementations of data stores into application servers.

The Java Data Objects architecture defines a standard API to data contained in local storage systems and heterogeneous enterprise information systems, such as ERP, mainframe transaction processing and database systems. The architecture also refers to the Connector architecture[see Appendix A reference 4] which defines a set of portable, scalable, secure, and transactional mechanisms for the integration of EIS with an application server.

This architecture enables a local storage expert, an enterprise information system (EIS) vendor, or an EIS domain expert to provide a standard data view (JDO Implementation) for the local data or EIS.

1.2 Scope

The JDO architecture defines a standard set of contracts between an application programmer and an JDO vendor. These contracts focus on the view of the Java instances of persistence capable classes.

JDO uses the Connector Architecture [see Appendix A reference 4] to specify the contract between the JDO vendor and an application server. These contracts focus on the important aspects of integration with heterogeneous enterprise information systems: instance management, connection management, and transaction management.

In order to provide transparent storage of local data, the JDO architecture does not require the Connector Architecture in non-managed (non-application server) environments.

1.3 Target Audience

The target audience for this specification includes:

- application developers
- JDO vendors
- enterprise information system (EIS) vendors and EIS Connector providers
- container providers
- enterprise system integrators
- enterprise tool vendors

JDO defines two types of interfaces: one of primary interest to application developers (the JDO instance life cycle) and the other of primary interest to container providers and JDO vendors.

1.4 Organization

This document describes the rationale and goals for a standard architecture for specifying the interface between an application developer and a local file system or EIS data store. It then elaborates the JDO architecture and its relationship to the Connector architecture.

The document next describes two typical JDO scenarios, one managed (application server) and the other non-managed (local file storage). This chapter explains key roles and responsibilities involved in the development and deployment of portable Java applications that require persistent storage.

The document then details the prescriptive aspects of the architecture. It starts with the JDO instance which is the application programmer-visible part of the system. It then details the JDO `PersistenceManager` which is the primary interface between a persistence-aware application, focusing on the contracts between the application developer and JDO implementation provider. Finally, the contracts for connection and transaction management between the JDO vendor and application server vendor are defined.

1.5 Document Convention

A Palatino font is used for describing the JDO architecture.

```
A courier font is used for code fragments.
```

1.6 Terminology Convention

"Must" is used where the specified component is required to implement some interface or action in order to be compliant with the specification.

"Might" is used where there is an implementation choice whether or how to implement a method or function.

2 Overview

This chapter introduces key concepts that are required for an understanding of the JDO architecture. It lays down a reference framework to facilitate a formal specification of the JDO architecture in the subsequent chapters of this document.

2.1 Definitions

2.1.1 JDO common interfaces

JDO Instance

A JDO instance is a Java programming language instance of a Java class which implements the application functions, and represents data in a local file system or enterprise data store. Without limitation, the data might come from a single data store entity, or from a collection of entities. For example, an entity might be a single object from an object database, a single row of a relational database, the result of a relational database query consisting of several rows, a merging of data from several tables in a relational database, or the result of executing a data retrieval API from an ERP system.

JDO instances implement the `PersistenceCapable` interface, either explicitly by the class writer, or implicitly by the results of the enhancer. The objective of JDO is that most user-written classes, including both entity-type classes and utility-type classes, might be persistence capable. The limitations are that the persistent state of the class must be represented entirely by the state of its Java fields, and that the class be enhanced (or otherwise be written to implement the `PersistenceCapable` interface) prior to being loaded into the execution environment of the Java Virtual Machine. Thus, system-type classes such as `System`, `Thread`, `Socket`, `File`, and the like cannot be JDO persistence capable, but common user-defined classes can be.

JDO Implementation

A JDO implementation is a collection of classes which implement the JDO contracts. The JDO implementation might be provided by an EIS vendor or by a third party vendor, collectively known as JDO vendor. The third party might provide an implementation that is optimized for a particular application domain, or might be a general purpose tool (such as a relational mapping tool, embedded object database, or enterprise object database).

The primary interface to the application is `PersistenceManager`, with interfaces `Query` and `Transaction` playing supporting roles for application control of the execution environment.

JDO Enhancer

A JDO enhancer, or byte code enhancer, is a program which modifies the byte codes of Java class files which implement an application component, to enable transparent loading and storing of the fields of the persistent instances. The JDO reference implementation (reference enhancement) contains an approach for the enhancement of Java class files to allow for enhanced class files to be shared among several co-resident JDO implementations.

Alternative approaches to byte code enhancement are preprocessing or code generation. If one of the alternatives is used instead of byte code enhancement, the `PersistenceCapable` contract must be implemented.

A JDO implementation is free to extend the Reference Enhancement contract with implementation-specific methods and fields, which might be used by its runtime environment.

Binary Compatibility Requirement: classes enhanced by the reference enhancer must be usable by any JDO compliant implementation; classes enhanced by a JDO compliant implementation must be usable by the reference implementation; and classes enhanced by a JDO compliant implementation must be usable by any other JDO compliant implementation.

The following table determines which interface is used by a JDO implementation based on

Table 1: Which Enhancement Interface is Used

	Reference Runtime	Vendor A Runtime	Vendor B Runtime
Reference Enhancer	Reference Enhancement	Reference Enhancement	Reference Enhancement
Vendor A Enhancer	Reference Enhancement	Vendor A Enhancement	Reference Enhancement
Vendor B Enhancer	Reference Enhancement	Reference Enhancement	Vendor B Enhancement

the enhancement of the persistence-capable class. For example, if Vendor A runtime detects that the class was enhanced by its own enhancement, then the runtime will use its enhancement contract. Otherwise, it will use the Reference Enhancement contract.

2.1.2 JDO in a managed environment

Readers who are primarily interested in JDO as a local persistence mechanism might ignore the following section, as it details architectural features not relevant to local environments.

This discussion provides a bridge to the Connector architecture which JDO uses for transaction and connection management in application server environments.

Enterprise Information System (EIS)

An EIS provides the information infrastructure for an enterprise. An EIS offers a set of services to its clients. These services are exposed to clients as local and / or remote interfaces. Examples of EIS include:

- relational database system;
- object database system;
- ERP system; and
- mainframe transaction processing system.

EIS Resource

An EIS resource provides EIS-specific functionality to its clients. Examples are:

- a record or set of records in a database system;
- a business object in an ERP system; and
- a transaction program in a transaction processing system

Resource Manager (RM)

A resource manager manages a set of shared resources. A client requests access to a resource manager to use its managed resources. A transactional resource manager can par-

ticipate in transactions that are externally controlled and coordinated by a transaction manager.

Connection

A connection provides connectivity to a resource manager. It enables an application client to connect to a resource manager, perform transactions, and access services provided by that resource manager. A connection can be either transactional or non-transactional. Examples include a database connection and a SAP R/3 connection.

Application Component

An application component can be a server-side component, such as an EJB, JSP, or servlet, that is deployed, managed and executed on an application server. It can be a component executed on the web-client tier but made available to the web-client by an application server, such as a Java applet, or DHTML page. It might also be an embedded component executed in a small footprint device using flash memory for persistent storage.

Session Beans

Session objects are EJB application components which execute on behalf of a single client, might be transaction aware, update data in an underlying data store, and do not directly represent data in the data store.

Entity Beans

Entity objects are EJB application components which provide an object view of transactional data in an underlying data store, allow shared access from multiple users, including session objects and remote clients, and directly represent data in the data store.

Helper objects

Helper objects are application components which provide an object view of data in an underlying data store, allow transactionally consistent view of data in multiple transactions, are usable by local session and entity beans, but do not have a remote interface.

Container

A container is a part of an application server that provides deployment and runtime support for application components. It provides a federated view of the underlying application server services for the application components. For more details on different types of standard containers, refer to Enterprise JavaBeans (EJB) [see Appendix A reference 1], Java Server Pages (JSP), and Servlets specifications.

2.2 Rationale

There is no existing Java platform specification which proposes a standard architecture for storing the state of Java objects persistently in transactional data stores.

The JDO architecture offers a Java solution to the problem of presenting a consistent view of data from the large number of application programs and enterprise information systems already in existence. By using the JDO architecture, it is not necessary for application component vendors to customize their products for each type of data store.

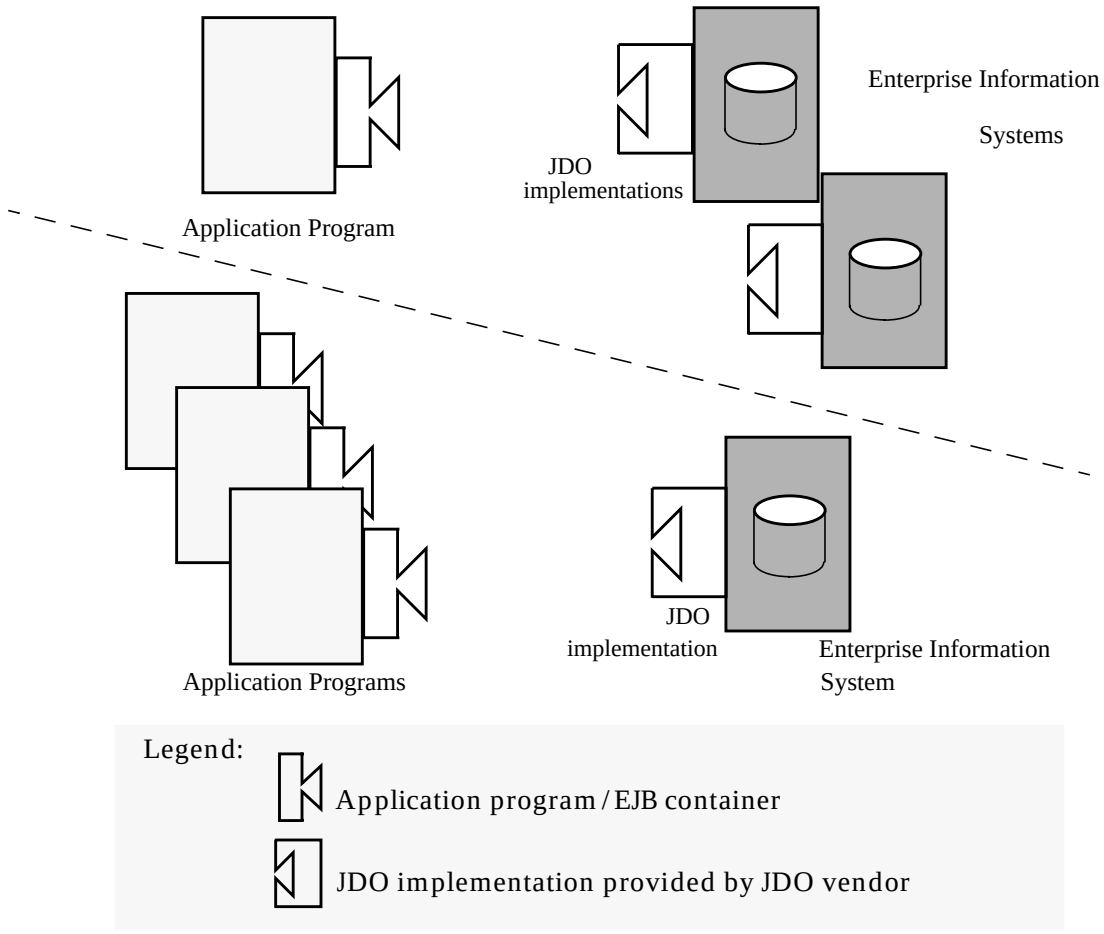
This architecture enables an EIS vendor to provide a standard data access interface for its EIS. The JDO implementation is plugged into an application server and provides underlying infrastructure for integration between the EIS and application components.

Similarly, a third party vendor can provide a standard data access interface for locally managed data such as would be found in an embedded device.

An application component vendor extends its system only once to support the JDO architecture and then exploits multiple data sources. Likewise, an EIS vendor provides one standard JDO implementation and it has the capability to work with any application component that uses the JDO architecture.

The Figure 1.0 on page 19 shows that an application component can plug into multiple JDO implementations. Similarly, multiple JDO implementations for different EISes can plug into an application component. This standard plug-and-play is made possible through the JDO architecture.

Figure 1.0 Standard plug-and-play between application programs and EISes using JDO



2.3 Goals

The JDO architecture has been designed with the following goals:

- The JDO architecture provides a transparent interface for application component and helper class developers to store data without learning a new data access language for each type of persistent data storage.

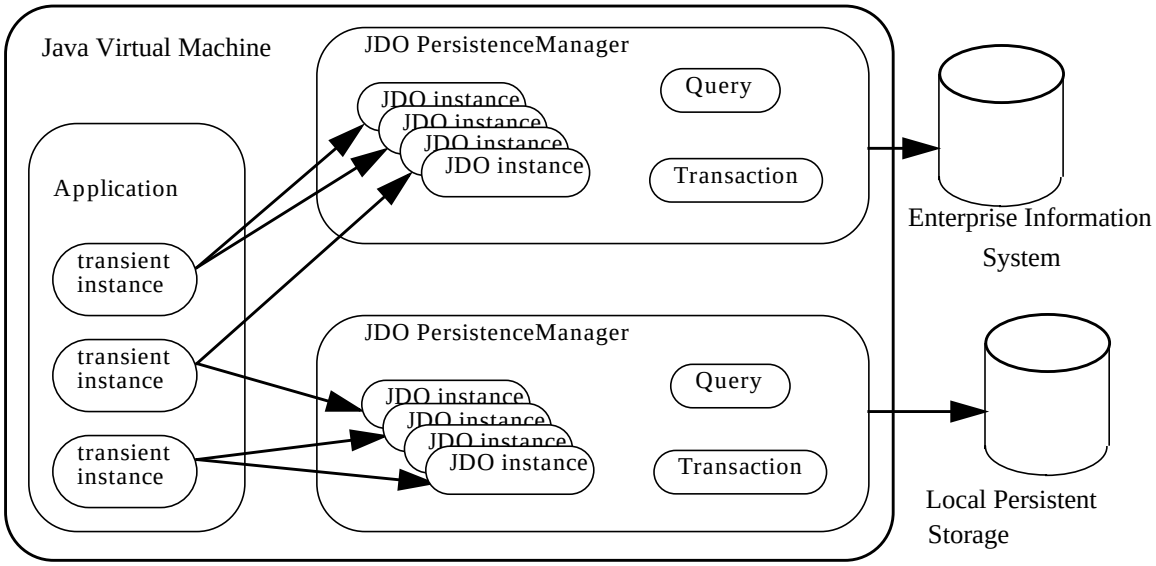
- The JDO architecture simplifies the development of scalable, secure and transactional JDO implementations for a wide range of EISes — ERP systems, database systems, mainframe-based transaction processing systems.
- The JDO architecture is implementable for a wide range of heterogeneous local file systems and EISes. The intent is that there will be various implementation choices for different EIS—each choice based on possibly application-specific characteristics and mechanisms of a mapping to an underlying EIS.
- The JDO architecture is suitable for a wide range of uses from embedded small footprint systems to large scale enterprise application servers. This architecture provides for exploitation of critical performance features from the underlying EIS, such as query evaluation and relationship management.
- The JDO architecture uses the J2EE Connector Architecture to make it applicable to all J2EE platform compliant application servers from multiple vendors.
- The JDO architecture makes it easy for application component developers to use the Java programming model to model the application domain and transparently retrieve and store data from various EIS systems.
- The JDO architecture defines contracts and responsibilities for various roles that provide pieces for standard connectivity to an EIS. This enables a standard JDO implementation from a EIS or third party vendor to be pluggable across multiple application servers.
- The connector architecture also enables an application programmer in a non-managed application environment to directly use the JDO implementation to access the underlying file system or EIS. This is in addition to a managed access to an EIS with the JDO implementation deployed in the middle-tier application server. In the former case, application programmers will not rely on the services offered by a middle-tier application server for security, transaction, and connection management, but will be responsible for managing these system-level aspects by using the EIS connector.

3 JDO Architecture

3.1 Overview

Multiple JDO implementations - possibly multiple implementations per type of EIS or local storage - are pluggable into an application server or usable directly in a two tier or embedded architecture. This enables application components, deployed either on a middle-tier application server or on a client-tier, to access the underlying data stores using a consistent Java-centric view of data. The JDO implementation provides the necessary mapping from Java objects into the special data types and relationships of the underlying data store.

Figure 2.0 Overview of non-managed JDO architecture



In a non-managed environment, the JDO implementation hides the EIS specific issues such as data type mapping, relationship mapping, and data retrieval and storage. The application component sees only the Java view of the data organized into classes with relationships and collections presented as native Java constructs.

Managed environments additionally provide transparency for the application components' use of system-level mechanisms - distributed transactions, security, and connection management, by hiding the contracts between the application server and JDO implementations.

With both managed and non-managed environments, an application component developer focuses on the development of business and presentation logic for the application components without getting involved in the issues related to connectivity with a specific EIS.

3.2 JDO Architecture

3.2.1 Two tier usage

For simple two tier usage, JDO exposes to the application component two primary interfaces: `javax.jdo.PersistenceManager`, from which services are requested; and `javax.jdo.PersistenceCapable`, which provides the management view of user-defined persistence capable classes.

The `PersistenceManager` interface provides services such as query management, transaction management, and life cycle management for instances of persistence capable classes.

The `PersistenceCapable` interface provides services such as life cycle state management for instances of persistence capable classes.

3.2.2 Application server usage

For application server usage, the JDO architecture uses the J2EE Connector architecture which defines a standard set of system-level contracts between the application server and EIS connectors. These system-level contracts are implemented in a resource adapter from the EIS side.

The JDO persistence manager is a caching manager as defined by the J2EE Connector architecture, that might use either its own (native) resource adapter or a third party resource adapter. If the JDO `PersistenceManager` has its own resource adapter, then implementations ofthe system-level contracts specified in the J2EE Connector architecture must be provided by the JDO vendor. These contracts include `ResourceAdapter, ConnectionManager, ManagedConnectionFactory, ManagedConnection, and LocalTransaction` interfaces.

The JDO `Transaction` must implement the `Synchronization` interface so that transaction completion events can cause flushing of state through the underlying connector to the EIS.

The application components are unable to distinguish between JDO implementations that use native resource adapters and JDO implementations that use third party resource adapters. However, the deployer will need to understand that there are two configurable components: the JDO `PersistenceManager` and its underlying resource adapter.

For convenience, the `PersistenceManagerFactory` provides the interface necessary to configure the underlying resource adapter.

Resource Adapter

A resource adapter provided by the JDO vendor is called a native resource adapter, and the interface is specific to the JDO vendor. It is a system-level software driver that is used by an application server or an application client to connect to a resource manager.

The resource adapter plugs into a container (provided by the application server). The application components deployed on the container then use the client API exposed by `javax.jdo.PersistenceManager` to access the JDO `PersistenceManager`. The JDO implementation in turn uses the underlying resource adapter interface specific to the data store. The resource adapter and application server collaborate to provide the underlying mechanisms - transactions, security and connection pooling - for connectivity to the EIS.

The resource adapter is located within the same address space of the JDO implementation using it. Examples of JDO native resource adapters are:

- Object/Relational (O/R) products that use their own native drivers to connect to object relational databases
- Object Database (OODBMS) products that store Java objects directly in object databases

Examples of non-native resource adapter implementations are:

- O/R mapping products that use JDBC drivers to connect to relational databases
- Hierarchical mapping products that use mainframe connectivity tools to connect to hierarchical transactional systems

Pooling

There are two levels of pooling in the JDO architecture. JDO `PersistenceManagers` might be pooled, and the underlying connections to the data stores might be independently pooled.

Pooling of the connections is governed by the Connector Architecture contracts. Pooling of `PersistenceManagers` is an optional feature of the JDO Implementation, and is not standardized for two-tier applications. For managed environments, `PersistenceManager` pooling is required in order to maintain correct transaction associations with `PersistenceManagers`.

For example, a JDO `PersistenceManager` instance might be bound to a session running a long duration optimistic transaction. This instance cannot be used by any other user for the duration of the optimistic transaction.

During the execution of a business method associated with the session, a connection might be required to fetch data from the data store. The `PersistenceManager` will request a connection from the connection pool to satisfy the request. Upon termination of the business method, the connection is returned to the pool but the `PersistenceManager` remains bound to the session.

After completion of the optimistic transaction, the `PersistenceManager` instance might be returned to the pool and reused for a subsequent transaction.

Contracts

JDO specifies the application level contract between the application components and the JDO `PersistenceManager`.

The J2EE Connector architecture specifies the standard contracts between application servers and an EIS connector used by a JDO implementation. These contracts are required for a JDO implementation to be used in an application server environment. The Connector architecture defines important aspects of integration: connection management, transaction management, and security.

The connection management contracts are implemented by the EIS resource adapter (which might include a JDO native resource adapter).

The transaction management contract is between the transaction manager (logically distinct from the application server) and the connection manager. It supports distributed transactions across multiple application servers and heterogeneous data management programs.

The security contract is required for secure access by the JDO connection to the underlying data store.

Figure 3.0 Contracts between application server and native JDO resource adapter

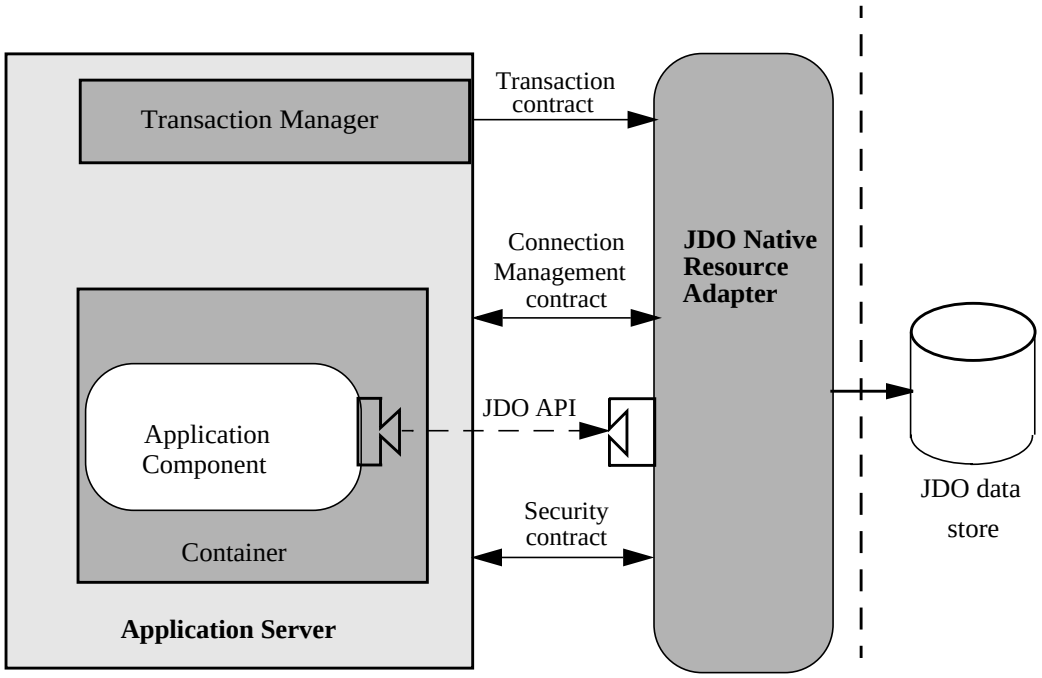
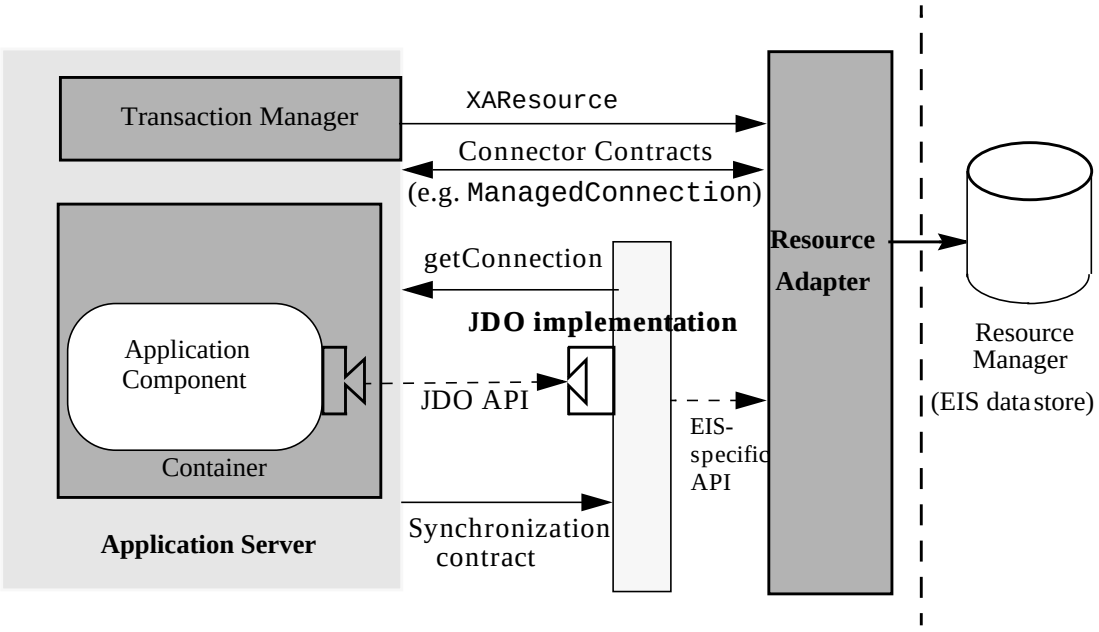


Figure 4.0 Contracts between application server and layered JDO implementation



The above diagram illustrates the relationship between a JDO implementation provided by a third party vendor and an EIS-provided resource adapter.

4 Roles and Scenarios

4.1 Roles

This chapter describes roles required for the development and deployment of applications built using the JDO architecture. The goal is to identify the nature of the work specific to each role so that the contracts specific to each role can be implemented on each side of the contracts.

The detailed contracts are specified in other chapters of this specification. The specific intent here is to identify the primary users and implementors of these contracts.

4.1.1 JDO Vendor

The JDO vendor is an expert in the technology related to a specific data store and is responsible for providing a JDO implementation for that specific data store. Since this role is highly data store specific, a data store vendor will often provide the standard JDO implementation.

A vendor can also provide a JDO implementation and associated set of application development tools through a loose coupling with a specific third party data store. Such providers specialize in writing connectors and related tools for a specific EIS or might provide a more general tool for a large number of data stores.

The JDO vendor requires that the EIS vendor has implemented the J2EE Connector architecture and the role of the JDO implementation is that of a synchronization adapter to the connector architecture.

4.1.2 Connector Provider

The connector provider is typically the vendor of the EIS or data store, and is responsible for supplying a library of interface implementations which satisfy the resource adapter interface.

In the JDO architecture, the Connector is a separate component, supplied by either the JDO vendor or by an EIS vendor or third party.

4.1.3 Application Server Vendor

An application server vendor [see Appendix A reference 1], [see Appendix A reference 3] provides an implementation of a J2EE compliant application server that provides support for component-based enterprise applications. A typical application server vendor is an OS vendor, middleware vendor, or database vendor.

The role of application server vendor will typically be the same as that of the container provider.

4.1.4 Container Provider

For bean-managed persistence, the container provides deployed application components with transaction and security management, distribution of clients, scalable management of resources and other services that are generally required as part of a managed server platform.

4.1.5 Application Component Provider

The application component provider produces an application library that implements application functionality through Java classes with business methods which store data persistently in one or more EISes through the JDO API.

There are two types of application components that interact with JDO. JDO-transparent application components, typically helper classes, are those that use JDO to have their state stored in a transactional data store, and directly access other components by references of their fields. Thus, they do not need to use JDO APIs directly.

JDO-aware application components (bean-managed persistent entity beans and session beans) use services of JDO by directly accessing its API. These components use JDO query facilities to retrieve collections of JDO instances from the data store, make specific instances persistent in a particular data store, delete specific persistent instances from the data store, interrogate the cached state of JDO instances, or explicitly manage the cache of the JDO `PersistenceManager`. These application components are non-transparent users of JDO.

Session beans that use helper JDO classes interact directly with `PersistenceManager` and `PersistenceCapable`. Entity beans might be generated by tools that analyze `PersistenceCapable` classes and produce bean-managed persistence beans corresponding to some of them. These generated entity beans use the standard JDO `PersistenceManager` contracts to request services.

The output of the application component provider is a set of jar files containing application components.

4.1.6 Application Assembler

The application assembler is a domain expert who assembles application components from multiple sources including in-house developers and application library vendors. The application assembler can combine different types of application components, for example EJBs, servlets, or JSPs, into a single end-user-visible application.

The input of the application assembler is one or more jar files, produced by application component providers. The output is one or more jar files with deployment specific descriptions.

4.1.7 Deployer

The deployer is responsible for configuring assembled components into specific operational environments. The deployer resolves all external references from components to other components or to the operational system.

For example, the deployer will bind application components in specific operating environments to data stores in those environments, and will resolve references from one application component to another. This typically involves using container-provided tools.

The deployer must understand, and be able to define, security roles, transactions, and connection pooling protocols for multiple data stores, application components, and containers.

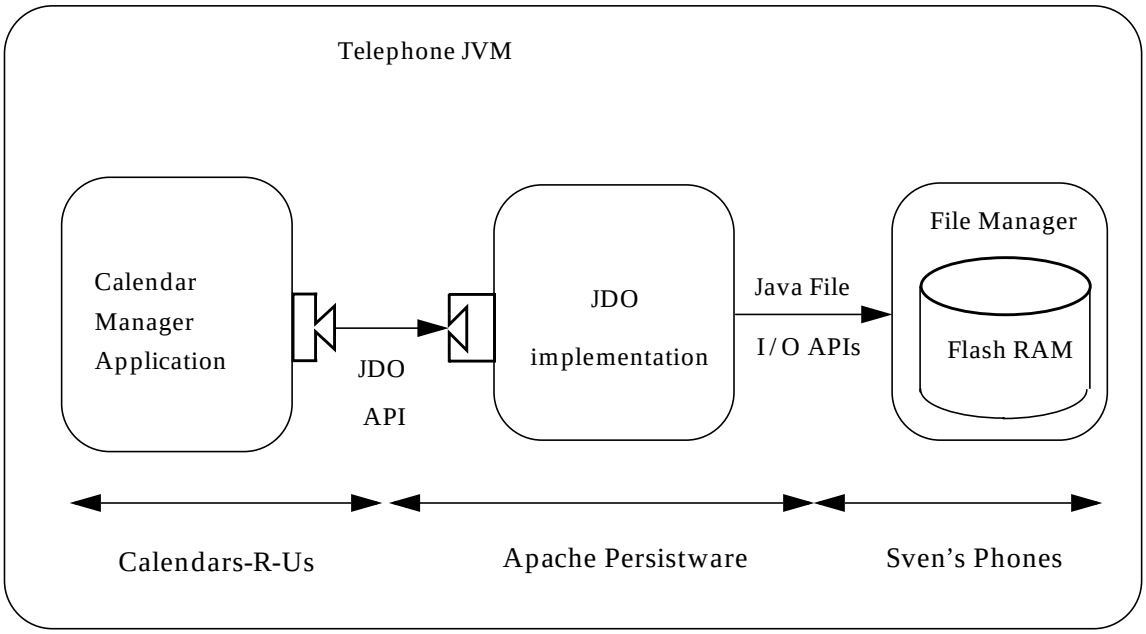
4.1.8 System Administrator

The system administrator manages the configuration and administration of multiple containers, resource adapters and EISs which combine into an operational system.

4.2 Scenario: Embedded calendar management system

This section describes a scenario to illustrate the use of JDO architecture in an embedded mobile device such as a personal information manager (PIM) or telephone.

Figure 5.0 Scenario: Embedded calendar manager



Sven's Phones is a manufacturer of high function telephones for the traveling businessperson. They have implemented a Java operating environment which provides persistence via a Java file I/O subsystem which writes to flash RAM.

Apache Persistware is a supplier of JDO software which has a small footprint and as such, is especially suited for embedded devices such as personal digital assistants and telephones. They use Java file I/O to store JDO instances persistently.

Calendars-R-Us is a supplier of appointment and calendar software that is written for several operating environments, from high function telephones to desktop workstations and enterprise application servers.

Calendars-R-Us uses the JDO API directly to manage calendar appointments on behalf of the user. The calendar application needs to insert, delete, and change calendar appointments based on the user's keypad input. It uses Java application domain classes: `Appointment, Contact, Note, Reminder, Location,` and `TelephoneNumber`. It employs JDK library classes: `Time, Date, ArrayList,` and `Calendar`.

Calendars-R-Us previously used Java file I/O APIs directly, but ran into several difficulties. The most efficient storage for some environments was an indexed file system, which was really required for management of thousands of entries. However, when they ported the application to the telephone, the indexed file system was too resource-intensive, and had to be abandoned.

They then wrote a data access manager for sequential files, but found that it burned out the flash RAM due to too much rewriting of data. They concluded that they needed to use the services of another software provider who specialized in persistence for flash RAM in embedded devices.

Apache Persistware developed a file access manager based on the Berkeley File System and successfully sold it to a range of Java customers from embedded devices to workstations. The interface was proprietary, which meant that every new sale was a challenge, because customers were loath to invest resources in learning a different interface for each environment they wanted to support. After all, Java was portable. Why wasn't file access?

Sven's Phones was a successful supplier of telephones to the mobile professional, but found themselves constrained by a lack of software developers. They wanted to offer a platform on which specially tailored software from multiple vendors could operate, and take advantage of external developers to write software for their telephones.

The solution to all of these issues was to separate the software into components that could be tailored by the domain expert for each component.

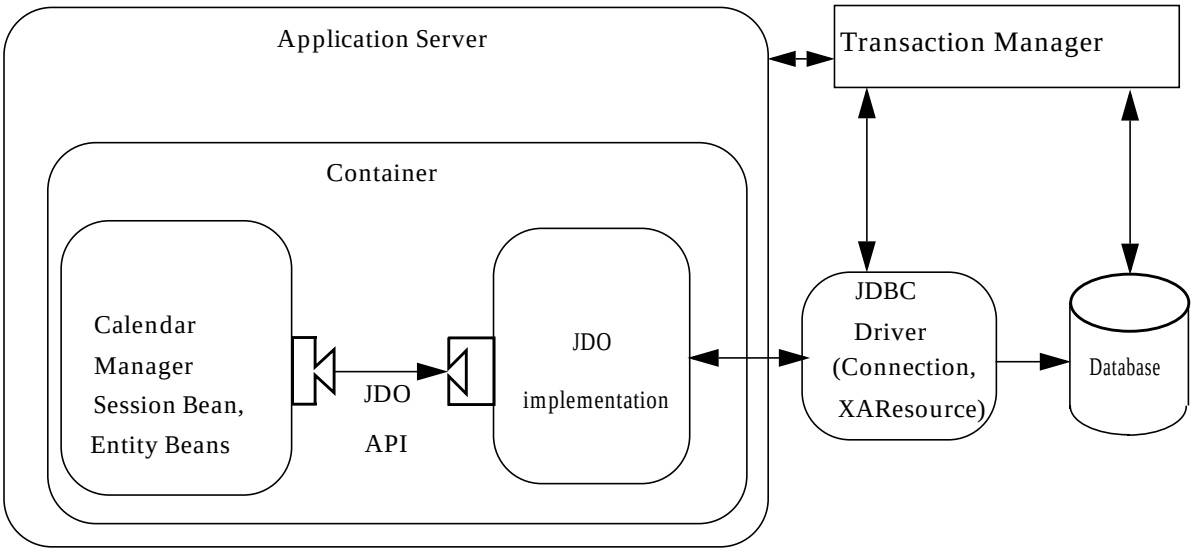
Sven's phones implemented the Java runtime environment for their phones, and wrote an efficient sequential file I/O manager that implemented the Java file I/O interface. This interface was used by Apache Persistware to build a JDO implementation, including a JDO instance handler and a JDO query manager.

Using the JDO interface, Calendars-R-Us rewrote just the query part of their software. The application classes did not have to be changed. Only the persistence interface that queried for specific instances needed to be modified.

4.3 Scenario: Enterprise Calendar Manager

Calendars-R-Us also supports workstations and enterprise mainframes with their calendar software, and they use the same interface for persistence in all environments. For enterprise environments, they simply need to use a different JDO implementation supplied by a different vendor to achieve persistence for their calendar objects.

Figure 6.0 Scenario: Enterprise Calendar Manager



In this scenario, the JDO implementation is provided by a vendor that maps Java objects to relational databases. The implementation uses standard JDBC to connect to the data store.

The JDO `PersistenceManager` is a caching manager, as defined by the Connector architecture, and it is configured to use a JDBC data source. The `PersistenceManager` instance might be cached when used with a Session Bean, and might be serially reused for multiple session beans.

Multiple JDO `PersistenceManager` instances might serially reuse connections from the same pool of JDBC drivers. Therefore, resource sharing is accomplished while maintaining state for each session.

5 Life Cycle of JDO Instances

This chapter specifies the life cycle for persistence capable class instances, hereinafter "JDO instances". The classes include behavior as specified by the class (bean) developer, and additional behavior as provided by the reference enhancer or JDO vendor's deployment tool. The enhancement of the classes allows application developers to treat JDO instances as if they were normal instances, with automatic fetching of persistent state from the JDO implementation.

5.1 Overview

JDO instances might be either transient or persistent. That is, they might represent the persistent state of data contained in a transactional data store. If a JDO instance is transient (and not transactional), then there is no difference between its behavior and the behavior of an instance of the unmodified (unenhanced) persistence capable class.

If a JDO instance is persistent, its behavior is linked to the transactional data store with which it is associated. The JDO implementation automatically tracks changes made to the values in the instance, and automatically refreshes values from the data store and saves values into the data store as required to preserve transactional integrity of the data.

During the life of a JDO instance, it transitions among various states until it is finally garbage collected by the JVM. During its life, the state transitions are governed by the behaviors executed on it directly as well as behaviors executed on the JDO `PersistenceManager` by both the application and by the execution environment (including the `TransactionManager`).

During the life cycle, instances at times might be inconsistent with the data store as of the beginning of the transaction. If instances are inconsistent, the notation for that instance in JDO is "dirty". Instances made newly persistent, deleted, or modified in the transaction are dirty.

At times, the JDO implementation might store the state of persistent instances in the data store. This process is called "flushing", and it does not affect the "dirty" state of the instances.

Under application control, transient JDO instances might observe transaction boundaries, in which the state of the instances is either preserved (on commit) or restored (on rollback). Transient instances that observe transaction boundaries are called transient transactional instances. Support for transient transactional instances is a JDO option; that is, a JDO compliant implementation is not required to implement the APIs that cause the state transitions associated with transient transactional instances.

Under application control, persistent JDO instances might not observe transaction boundaries. These instances are called persistent-nontransactional instances, and the life cycle of these instances is not affected by transaction boundaries. Support for nontransactional instances is a JDO option.

If a JDO instance is persistent or transactional, it contains a non-null reference to a JDO `StateManager` instance which is responsible for managing the JDO instance state changes and for interfacing with the JDO `PersistenceManager`.

5.2 Goals

The JDO instance life cycle has the following goals:

- The fact of persistence should be transparent to both JDO instance developer and application component developer
- JDO instances should be able to be used efficiently in a variety of environments, including managed (application server) and non-managed (two-tier) cases
- Several JDO `PersistenceManager`s might be co-resident and might share the same persistence capable classes (although any JDO instance can only be associated with zero or one `PersistenceManager` at a time)

5.3 Architecture:

JDO Instances

For transient JDO instances, there is no supporting infrastructure required. That is, transient instances will never make calls to methods to the persistence infrastructure. There is no requirement to instantiate objects outside the application domain. Aside from the additional methods available for life cycle state interrogation, there is no difference in behavior between transient instances of enhanced classes and transient instances of the same non-enhanced classes.

Persistent JDO instances execute in an environment which contains an instance of the JDO `PersistenceManager` responsible for its persistent behavior. The JDO instance contains a reference to an instance of the JDO `StateManager` responsible for the state transitions of the instance as well as for managing the contents of the fields of the instance. The `PersistenceManager` and the `StateManager` might be implemented by the same instance, but their interfaces are distinct.

The contract between the persistence capable class and other application components extends the contract between the associated non-persistence capable class and application components. These contract extensions support interrogation of the life cycle state of the instances and are intended for use by management parts of the system.

JDO State Manager

Persistent and transactional JDO instances contain a reference to a JDO `StateManager` instance to which all of the JDO interrogatives are delegated. The associated JDO `StateManager` instance maintains the state changes of the JDO instance and interfaces with the JDO `PersistenceManager` to manage the values of the data store.

5.4 JDO Identity

Java defines two concepts for determining if two instances are the same instance (identity), or represent the same data (equality). JDO extends these concepts to determine if two instances represent the same data store object.

Java object identity is entirely managed by the Java Virtual Machine. Instances are identical if and only if they occupy the same storage location within the JVM.

Java object equality is determined by the class. Distinct instances are equal if they represent the same data, such as the same value for an integer, or equivalent bits in a bit array.

The interaction between Java object identity and equality is an important one for JDO developers. Java object equality is an application specific concept, and JDO implementations must not change the application's implementation of equality. Still, JDO implementations must manage the cache of JDO instances such that there is only one JDO instance associated with each JDO `PersistenceManager` representing the persistent state of each corresponding data store object. Therefore, JDO defines object identity different from both the Java VM object identity and the application equality.

Note that applications should implement equality for persistence capable classes different from the default implementation which simply uses the Java VM object identity. This is because the JVM object identity of a persistent instance cannot be guaranteed between `PersistenceManager`s and across space and time, except in very specific cases noted below.

Additionally, if persistence instances are stored in the data store and are queried using the `==` query operator, or are referred by a persistent collection that enforces equality (`Set`, `Map`) then the implementation of `equals` should exactly match the JDO implementation of equality, using the primary key or `ObjectId` as the key. This is not enforced, but if not correctly implemented, semantics of standard collections and JDO collections may differ.

To avoid confusion with Java object identity, this document refers to the JDO concept as JDO identity.

JDO defines three types of JDO identity: JDO identity managed by the application and enforced by the data store, often called the primary key; JDO identity managed by the data store without being tied to any JDO instance values; and JDO identity managed by the implementation to guarantee uniqueness in the JVM but not in the data store.

The type of JDO identity used is a property of a JDO `PersistenceCapable` class and is fixed at enhancement time.

The representation of JDO identity in the JVM is via a JDO object id. Every persistent instance (Java instance representing a persistent object) has a corresponding object id. There might be an instance representing the object id, or not. The instance of an object representing the object id corresponding to a persistent instance might be acquired by the application at run time, saved in durable storage (by serialization or other technique), and used later to obtain a reference to the same data store object.

The class representing the object id for non-application identity classes is defined by the JDO implementation. The implementation might choose to use any class that satisfies the requirements for the specific type of JDO identity for a class. It might choose the same class for several different JDO classes, or might use a different class for each JDO class.

The class representing the object id for application identity classes is defined by the application in the metadata, and might be provided by the application or by a JDO vendor tool.

The application-visible representation of the JDO identity is an instance that is completely under the control of the application. The object id instances used as parameters or returned by methods in the JDO interface (`getObjectId, getTransactionalObjectId,` and `getObjectById`) will never be saved internally; rather, they are copies of the internal representation or used to find instances of the internal representation.

Therefore, the object returned by any call to `getObjectId` might be modified by the user, but that modification does not affect the identity of the object which was originally referred. That is, the call to `getObjectId` returns only a copy of the object identity used internally by the implementation.

It is a requirement that the instance returned by a call to `getObjectById (Object)` of different `PersistenceManager` instances returned by the same `PersistenceManagerFactory` represent the same persistent object, but with different Java object identity.

Further, any instances returned by any calls to `getObjectById (Object)` with the same object id instance to the same `PersistenceManager` instance must be identical (assuming the instances were not garbage collected between calls).

The JDO identity of transient instances is not defined. Attempts to get the object id for a transient instance will return `null`.

Uniquing

JDO identity of persistent instances is managed by the implementation. For a managed JDO identity (data store or primary key), there is only one persistent instance associated with a specific data store object per `PersistenceManager` instance, regardless of how the persistent instance was put into the cache:

- `PersistenceManager.getObjectById (Object oid, boolean validate);`
- query via a `Query` instance associated with the `PersistenceManager` instance;
- navigation from a persistent instance associated with the `PersistenceManager` instance;
- `PersistenceManager.makePersistent(Object pc);`

Change of identity

JDO instances using primary key identity may change their identity during a transaction, if the application changes a primary key field. In this case, there is a new JDO Identity associated with the JDO instance immediately upon completion of the statement which changes a primary key field. If a JDO instance is already associated with the new JDO Identity, then a `JDOUserException` is thrown and the statement which attempted to change the primary key field does not complete.

Upon successful commit of the transaction, the existing datastore instance will have been updated with the changed values of the primary key fields.

JDO Identity Support

A JDO implementation is required to support either Application (primary key) identity or Datastore identity, and may optionally support Non-datastore identity.

5.4.1 Application (primary key) identity

This is the JDO identity type used for data stores in which the value(s) in the instance determine the identity of the object in the data store. Thus, JDO identity is managed by the application. The class provided by the application which implements the JDO object id has all of the characteristics of an RMI remote object, which makes it possible to use the JDO object id class as the EJB primary key class. Specifically:

- the `ObjectId` class must be public;
- the `ObjectId` class must implement `Serializable`;
- the `ObjectId` class must have a public constructor, which might be the default constructor or a no-arg constructor;
- the field types of all non-static fields in the `ObjectId` class must be serializable;
- all serializable non-static fields in the `ObjectId` class must be public;

- the names of the non-static fields in the `ObjectId` class must include the names of the primary key fields in the JDO class, and the types of the common fields must be identical;
- the `equals()` and `hashCode()` methods of the `ObjectId` class must use the value(s) of all the fields corresponding to the primary key fields in the JDO class;

These restrictions allow the application to construct an instance of the primary key class providing only the values for the primary key fields. The JDO implementation is permitted to extend the primary key class to use additional fields, not provided by the application, to further identify the instance in the data store. Thus, the JDO object id instance returned by an implementation might be a subclass of the user-defined primary key class. Any JDO implementation must be able to use the JDO object id instance from any other JDO implementation.

A primary key identity is associated with a specific set of fields. The fields associated with the primary key are a property of the persistence-capable class, and cannot be changed after the class is enhanced for use at runtime. When a transient instance is made persistent, the implementation uses the values of the fields associated with the primary key to construct the JDO identity.

Persistence-capable classes that use application identity have special considerations for inheritance. To be portable, the key class must be the same for all classes in the inheritance hierarchy, and key fields must be declared only in the least-derived (topmost) persistence-capable class in the hierarchy.

5.4.2 Data store identity

This is the JDO identity type used for data stores in which the identity of the data in the data store does not depend on the values in the instance. The implementation guarantees uniqueness for all instances.

A JDO implementation may choose one of the primitive wrapper classes as the `ObjectId` class (`Short, Integer, Long,` or `String`), or may choose an implementation-specific class. Implementation-specific classes used as JDO `ObjectId` have the following characteristics:

- the `ObjectId` class must be public;
- the `ObjectId` class must implement `Serializable`;
- the `ObjectId` class must have a public constructor, which might be the default constructor or a no-arg constructor;
- all serializable fields in the `ObjectId` class must be public;
- the field types of all non-static fields in the `ObjectId` class must be serializable;

Note that, unlike primary key identity, data store identity `ObjectId` classes are **not** required to support equality with `ObjectId` classes from other JDO implementations. Further, the application cannot change the JDO identity of an instance of a class using data store identity.

5.4.3 Non-data store JDO identity

The primary usage for non-data store JDO identity is for log files, history files, and other similar files, where performance is a primary concern. Objects are typically inserted into data stores with transactional semantics, but are not accessed by key. They may have references to instances elsewhere in the data store, but often have no keys or indexes themselves. They might be accessed by other attributes, and might be deleted in bulk.

Multiple objects in the data store might have exactly the same values, yet an application program might want to treat the objects individually. For example, the application should be able to count the persistent instances to determine the number of data store objects with the same values. Also, the application might change a single field of an instance with duplicate objects in the data store, and the expected result in the data store is that exactly one instance has its field changed. Similarly, if the application deletes one of multiple duplicate objects, only one of the objects in the data store should be deleted.

Because the JDO identity is not visible in the data store, there are special behaviors with regard to non-data store JDO identity:

- the `ObjectId` is not valid after making the associated instance hollow, and attempts to retrieve it will throw a `JDOUserException`;
- the `ObjectId` cannot be used in a different instance of `PersistenceManager` from the one which issued it, and attempts to use it even indirectly (e.g. `getObjectById` with a persistence-capable object as the parameter) will throw a `JDOUserException`;
- the persistent instance might transition to persistent-nontransactional or hollow but cannot transition to any other state afterward;
- attempts to access the instance in the hollow state will throw a `JDOUserException`;
- the results of a query in the data store will always return instances that are not already in the Java VM, so multiple queries that find the same objects in the data store will return additional JDO instances with the same values and different JDO identities;
- `makePersistent` will succeed even though another instance already has the same values for all persistent fields.

For JDO identity which is not managed by the data store, the class which implements JDO `ObjectId` has the following characteristics:

- the `ObjectId` class must be public;
- the `ObjectId` class must have a public constructor, which might be the default constructor or a no-arg constructor;
- all fields in the `ObjectId` class must be public;
- the field types of all fields in the `ObjectId` class must be serializable.

5.5 Life Cycle States

There are ten states defined by this specification. Seven states are required, and three states are optional. If an implementation does not support certain operations, then these three states are not reachable.

Data Store Transactions

The following descriptions apply to data store transactions with retainValues=false. Optimistic transaction and retainValues=true state transitions are covered later in this chapter.

5.5.1 Transient (Required)

JDO instances created by using a developer-written constructor do not involve the persistence environment. These JDO instances behave exactly like instances of the unenhanced class.

There is no JDO Identity associated with a transient instance.

There is no intermediation to support fetching or storing values for fields. There is no support for demarcation of transaction boundaries. Indeed, there is no transactional behavior of these instances, unless they are referenced by transactional instances at commit time.

When a persistent instance is committed to the data store, instances referenced by persistent fields of the flushed instance become persistent. This behavior propagates to all instances in the closure of instances through persistent fields. This behavior is sometimes called persistence by reachability.

No methods of transient instances throw exceptions except those defined by the class developer.

A transient instance transitions to persistent-new if it is the parameter of `makePersistent` or if it is referenced by a persistent field of a persistent instance when that instance is committed. The values of persistent and transactional non-persistent fields are saved during this transition for use during rollback.

5.5.2 Persistent-new (Required)

JDO instances that are newly persistent in the current transaction are persistent-new. This is the state of an instance that has been requested by the application component to become persistent, by using the `PersistenceManager makePersistent` method on the instance.

During the transition from transient to persistent-new, the associated `PersistenceManager` becomes responsible to implement state interrogation and further state transitions. The values of persistent and transactional non-persistent fields are saved for use during rollback.

Also during the transition from transient to persistent-new, a JDO identity is assigned to the instance by the JDO implementation. This identity uniquely identifies the instance inside the `PersistenceManager`, and might uniquely identify the instance in the data store. An instance of the JDO identity will be returned by the `PersistenceManager` method `getObjectId (Object)`.

A persistent-new instance transitions to persistent-new-deleted if it is the parameter of `deletePersistent`.

A persistent-new instance transitions to hollow when it is flushed to the data store during `commit`. This transition is not visible during `beforeCompletion`, and is visible during `afterCompletion`.

It transitions to transient during rollback, and its state is restored as of the call to `makePersistent`.

5.5.3 Persistent-dirty (Required)

JDO instances that represent persistent data that was changed in the current transaction are persistent-dirty.

A persistent-dirty instance transitions to persistent-deleted if it is the parameter of `deletePersistent`.

Persistent-dirty instances transition to hollow during commit or rollback.

If an application modifies a field, but the new value is identical to the old value, then it is an implementation choice whether the JDO instance is modified or not. If no modification to any field was made by the application, then the implementation must not mark the instance as dirty. If a modification was made to any field that changes the value of the field, then the implementation must mark the instance as dirty.

5.5.4 Hollow (Required)

JDO instances that represent specific persistent data in the data store but whose values are not in the JDO instance are hollow. The hollow state provides for the guarantee of uniqueness for transactional instances between transactions.

This is permitted to be the state of instances committed from a previous transaction, acquired by the method `getObjectById`, returned by iterating an `Extent`, returned in the result of a query execution, or navigating a persistent field reference. However, the JDO implementation may choose to return instances in a different state reachable from hollow.

A JDO implementation is permitted to effect a legal state transition of a hollow instance at any time, as if a field were read. Therefore, the hollow state might not be visible to the application.

During the commit of the transaction in which a dirty persistent instance has had its values changed (including a new persistent instance), the underlying data store is changed to have the transactionally consistent values from the JDO instance, and the instance transitions to hollow.

Requests by the application for an instance with the same JDO identity (query, navigation, or lookup by ObjectId), in a subsequent transaction using the same `PersistenceManager` instance, will return the identical JDO instance. If the application does not hold a strong reference to a hollow instance, the hollow instance might be garbage collected, as the `PersistenceManager` must not hold a strong reference to any hollow instance.

The hollow JDO instance maintains its JDO identity and its association with the JDO `PersistenceManager`. If the instance is of a class using application identity, the hollow instance maintains its primary key fields.

A hollow instance transitions to persistent-deleted if it is the parameter of `deletePersistent`.

A hollow instance transitions to persistent-dirty if a change is made to any field. It transitions to persistent-clean if a read access is made to any persistent field other than one of the primary key fields.

5.5.5 Persistent-clean (Required)

JDO instances that represent specific transactional persistent data in the data store, and whose values have not been changed in the current transaction, are persistent-clean. This is the state of an instance whose values have been requested in the current data store transaction, and whose values have not been changed by the current transaction.

A persistent-clean instance transitions to persistent-dirty if a change is made to any persistent field.

A persistent-clean instance transitions to persistent-deleted if it is the parameter of `deletePersistent`.

A persistent-clean instance transitions to persistent-nontransactional if it is the parameter of `makeNontransactional`.

A persistent-clean instance transitions to hollow at commit or rollback. It retains its identity and its association with the `PersistenceManager`.

5.5.6 Persistent-deleted (Required)

JDO instances that represent specific persistent data in the data store, and which have been deleted in the current transaction, are persistent-deleted.

Read access to primary key fields is permitted but any access to other persistent fields will throw a `JDOUserException`.

A persistent-deleted instance transitions to transient at commit. During the transition, the instance is cleared of its state (the persistent fields are written with their default values), and loses its JDO Identity and its association with the `PersistenceManager`.

A persistent-deleted instance transitions to hollow at rollback. It retains its identity and its association with the `PersistenceManager`.

5.5.7 Persistent-new-deleted (Required)

JDO instances that represent instances which have been newly made persistent and deleted in the current transaction are persistent-new-deleted.

Read access to primary key fields is permitted but any access to other persistent fields will throw a `JDOUserException`.

A persistent-new-deleted instance transitions to transient at commit. During the transition, the instance is cleared of its state, and loses its JDO Identity and its association with the `PersistenceManager`.

A persistent-new-deleted instance transitions to transient at rollback. The values of persistent and transactional non-persistent fields in the instance are restored to their state as of the call to `makePersistent`, and the instance loses its JDO Identity and its association with the `PersistenceManager`.

5.6 Nontransactional (Optional)

Management of nontransactional instances is an optional feature of a JDO implementation. Usage is primarily for slowly changing data or for optimistic transaction management, as the values in nontransactional instances are not guaranteed to be transactionally consistent.

The use of this feature is governed by the `PersistenceManager` options `NontransactionalRead`, `NontransactionalWrite`, `Optimistic`, and `RetainValues`. An implementation might support any or all of these options. For example, an implementation might support only `NontransactionalRead`.

If a `PersistenceManager` does not support this optional feature, an operation that would result in an instance transitioning to the persistent-nontransactional state, or a request to set the `NontransactionalRead`, `NontransactionalWrite`, or `RetainValues` option to `true`, throws a `JDOUnsupportedOptionException`.

`RetainValues`, `NontransactionalRead`, `NontransactionalWrite`, and `Optimistic` are independent options.

With `NontransactionalRead` set to `true`:

- Navigation and queries are valid outside of a transaction. It is a JDO implementation decision whether the instances returned are in the hollow or persistent-nontransactional state.
- When a field of a hollow instance is read outside of a transaction, the instance transitions to persistent-nontransactional.
- If a persistent-clean instance is the parameter of `makeNontransactional`, the instance transitions to persistent-nontransactional.

With `NontransactionalWrite` set to `true`:

- Modification of persistent-nontransactional instances is permitted outside a transaction. The changes do not participate in any subsequent transaction.

With `RetainValues` set to `true`:

- At commit, persistent-clean, persistent-new, and persistent-dirty instances transition to persistent-nontransactional. Fields defined in the XML metadata as containing second-class types are replaced with new SCO instances. These include `java.util.Date`, and most collection types.
- At rollback, persistent-clean instances transition to persistent-nontransactional. Persistent-dirty instances transition to persistent-nontransactional and the state of each persistent and transactional field is restored to its state as of the beginning of the transaction.

5.6.1 Persistent-nontransactional (Optional)

NOTE: The following discussion applies only to datastore transactions. See section 5.8 for a discussion on how optimistic transactions change this behavior.

JDO instances that represent specific persistent data in the data store, whose values are currently loaded but not transactionally consistent, are persistent-nontransactional. There is a JDO Identity associated with these instances, and transactional instances can be obtained from them.

The persistent-nontransactional state allows persistent instances to be managed as a shadow cache of instances that are updated asynchronously.

As long as a transaction is not in progress, the application might make changes to the values in the instance, and data store values might be retrieved from the data store by the `PersistenceManager`. There is no state change associated with either of these operations.

A persistent-nontransactional instance transitions to persistent-clean if it is the parameter of a `makeTransactional` method executed when a transaction is in progress. The state of the instance in memory is discarded (cleared) and the state is loaded from the data store.

A persistent-nontransactional instance transitions to persistent-clean if any field is accessed when a data store transaction is in progress. The state of the instance in memory is discarded and the state is loaded from the data store.

A persistent-nontransactional instance transitions to persistent-dirty if any field is written when a transaction is in progress. The state of the instance in memory is saved for use during rollback, and the state is loaded from the data store. Then the change is applied.

A persistent-nontransactional instance transitions to persistent-deleted if it is the parameter of `deletePersistent`. The state of the instance in memory is saved for use during rollback.

5.7 Transient Transactional (Optional)

Management of transient transactional instances is an optional feature of a JDO implementation. The following sections describe the additional states and state changes when using transient transactional behavior.

A transient instance transitions to transient-clean if it is the parameter of `makeTransactional`.

5.7.1 Transient-clean (Optional)

JDO instances that represent transient transactional instances whose values have not been changed in the current transaction are transient-clean. This state is not reachable if the JDO `PersistenceManager` does not implement `makeTransactional`.

Changes made outside of a transaction are allowed without a state change. A transient-clean instance transitions to transient-dirty if any field is changed in a transaction. During the transition, values of persistent and transactional non-persistent fields are saved by the `PersistenceManager` for use during rollback.

A transient-clean instance transitions to transient if it is the parameter of `makeNontransactional`.

5.7.2 Transient-dirty (Optional)

JDO instances that represent transient transactional instances whose values have been changed in the current transaction are transient-dirty. This state is not reachable if the JDO `PersistenceManager` does not implement `makeTransactional`.

A transient-dirty instance transitions to transient-clean at commit. The values of persistent and transactional non-persistent fields saved (for rollback processing) at the time the transition was made from transient-clean to transient-dirty are discarded. None of the values of fields in the instance are modified as a result of commit.

A transient-dirty instance transitions to transient-clean at rollback. The values of persistent and transactional non-persistent fields saved at the time the transition was made from transient-clean to transient-dirty are restored.

5.8 Optimistic Transactions (Optional)

Optimistic transaction management is an optional feature of a JDO implementation.

The `Optimistic` flag set to `true` changes the state transitions of persistent instances:

- If a field is read, a hollow instance transitions to persistent-nontransactional instead of persistent-clean. Subsequent reads of fields do not cause a transition from persistent-nontransactional.
- At commit, with `RetainValues true`, persistent-new, persistent-clean, and persistent-dirty instances transition to persistent-nontransactional.
- At rollback, with `RetainValues true`, persistent-clean instances transition to persistent-nontransactional. A persistent-dirty instance transitions to persistent-nontransactional, and values of persistent and transactional non-persistent fields are restored as of the beginning of the transaction.
- A persistent-nontransactional instance transitions to persistent-deleted if it is a parameter of `deletePersistent`. The state of the instance in memory is saved for use during rollback, and for verification during commit. The state is not loaded from the datastore. If fresh values need to be loaded from the datastore, then the user should first call `refresh` on the instance.
- A persistent-nontransactional instance transitions to persistent-clean if it is a parameter of a `makeTransactional` method executed when an optimistic transaction is in progress. The state of the instance in memory is preserved. If fresh values need to be loaded from the datastore, then the user should first call `refresh` on the instance.

- A persistent-nontransactional instance transitions to persistent-dirty if a field is modified when an optimistic transaction is in progress. The state of the instance in memory is saved for use during rollback, and for verification during commit. The state is not loaded from the datastore. If fresh values need to be loaded from the datastore, then the user should first call `refresh` on the instance.

Table 2: State Transitions

method \ current state	Transient	P-new	P-clean	P-dirty	Hollow
makePersistent	P-new	unchanged	unchanged	unchanged	unchanged
deletePersistent	error	P-new-del	P-del	P-del	P-del
makeTransactional	T-clean	unchanged	unchanged	unchanged	unchanged
makeNontransactional	n/a	n/a	P-nontrans	n/a	unchanged
makeTransient	unchanged	error	Transient	error	Transient
commit	unchanged	Hollow	Hollow	Hollow	unchanged
commit retainValues	unchanged	P-nontrans	P-nontrans	P-nontrans	unchanged
rollback	unchanged	Transient	Hollow	Hollow	unchanged
rollback retainValues	unchanged	Transient	P-nontrans	P-nontrans	unchanged
refresh with active Datastore transaction	n/a	unchanged	unchanged	P-clean	unchanged
refresh with active Optimistic transaction	n/a	unchanged	unchanged	P-nontrans	unchanged
evict	n/a	unchanged	Hollow	unchanged	unchanged
read field outside or with Optimistic transaction	unchanged	unchanged	unchanged	unchanged	P-nontrans
read field with active Datastore transaction	unchanged	unchanged	unchanged	unchanged	P-clean
write field with active transaction	unchanged	unchanged	P-dirty	unchanged	P-dirty
write field outside transaction	unchanged	n/a	n/a	n/a	P-nontrans

method \ current state	T-clean	T-dirty	P-new-del	P-del	P-nontrans
makePersistent	P-new	P-new	unchanged	unchanged	unchanged
deletePersistent	error	error	unchanged	unchanged	P-del
makeTransactional	unchanged	unchanged	unchanged	unchanged	P-clean
makeNontransactional	Transient	error	error	error	unchanged

method \ current state	T-clean	T-dirty	P-new-del	P-del	P-nontrans
makeTransient	unchanged	unchanged	error	error	Transient
commit retainValues=false	unchanged	T-clean	Transient	Transient	unchanged
commit retainValues=true	unchanged	T-clean	Transient	Transient	unchanged
rollback retainValues=false	unchanged	T-clean	Transient	Hollow	unchanged
rollback retainValues=true	unchanged	T-clean	Transient	P-nontrans	unchanged
refresh	n/a	n/a	unchanged	unchanged	unchanged
evict	unchanged	unchanged	unchanged	unchanged	Hollow
read field outside or with Optimistic transaction	unchanged	unchanged	error	error	unchanged
read field with active Datastore transaction	unchanged	unchanged	error	error	P-clean
write field outside transaction	unchanged	n/a	error	error	unchanged
write field with active transaction	T-dirty	unchanged	error	error	P-dirty

error: a `JDOUserException` is thrown; the state does not change
unchanged: no state change takes place; no exception is thrown due to the state change
n/a: not applicable; if this instance is an explicit parameter of the method, a `JDOUserException` is thrown; if this instance is an implicit parameter, it is ignored.

Figure 7.0 Life Cycle: New Persistent Instances

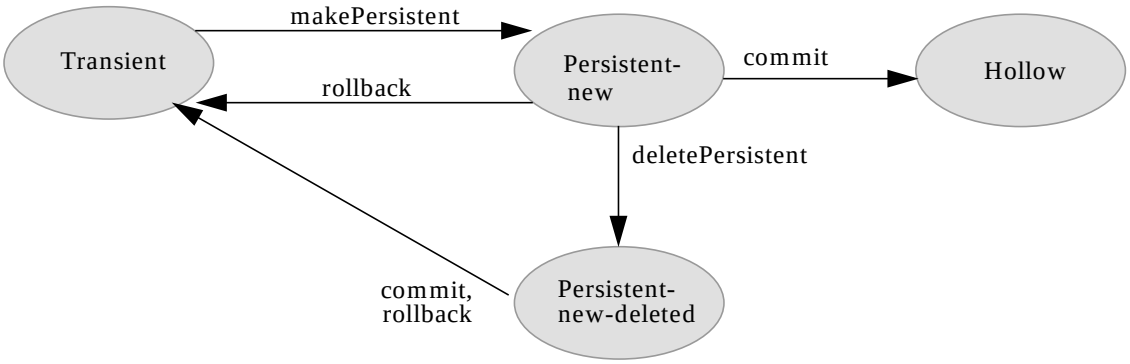


Figure 8.0 Life Cycle: Transactional Access

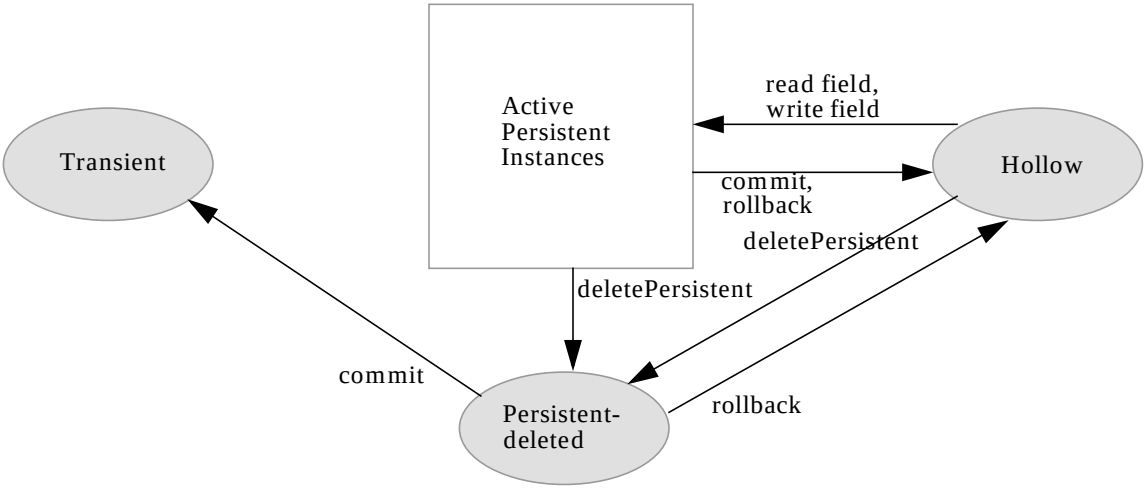


Figure 9.0 Life Cycle: Data Store Transactions

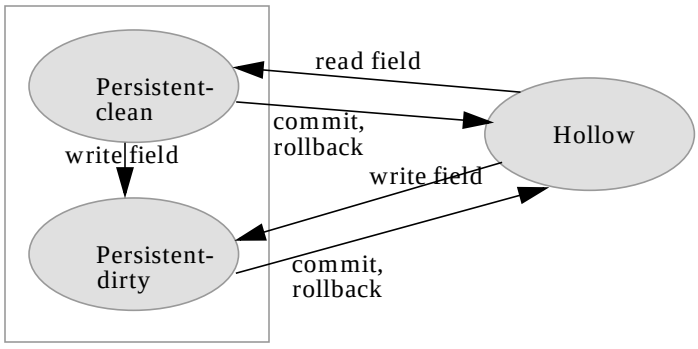


Figure 10.0 Life Cycle: Optimistic Transactions

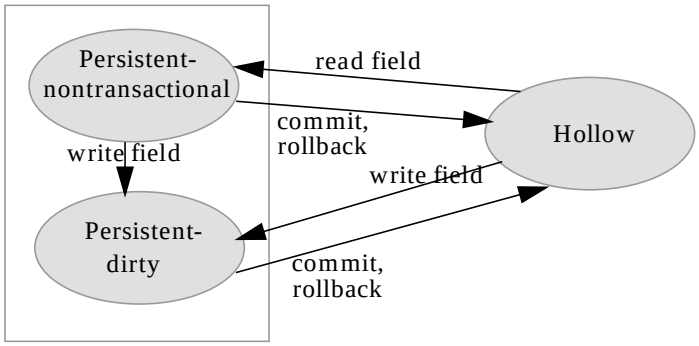


Figure 11.0 Life Cycle: Access Outside Transactions

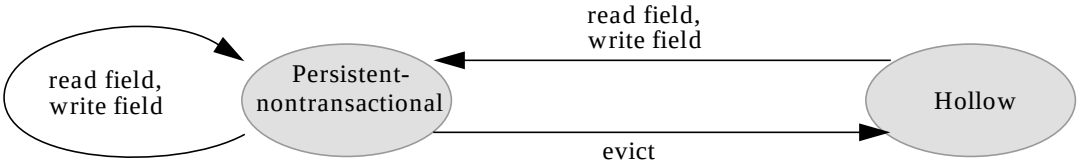


Figure 12.0 Life Cycle: Transient Transactional

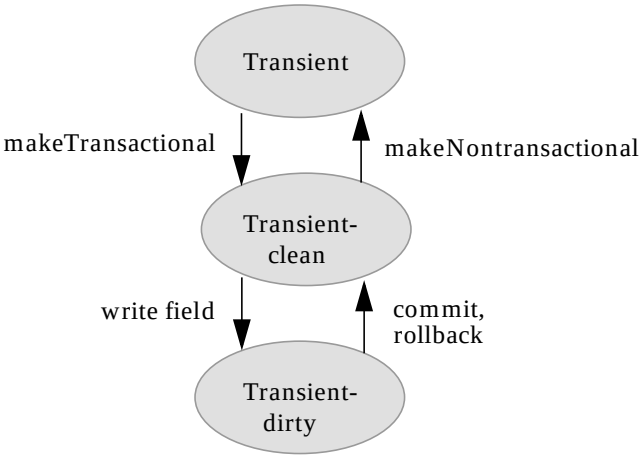
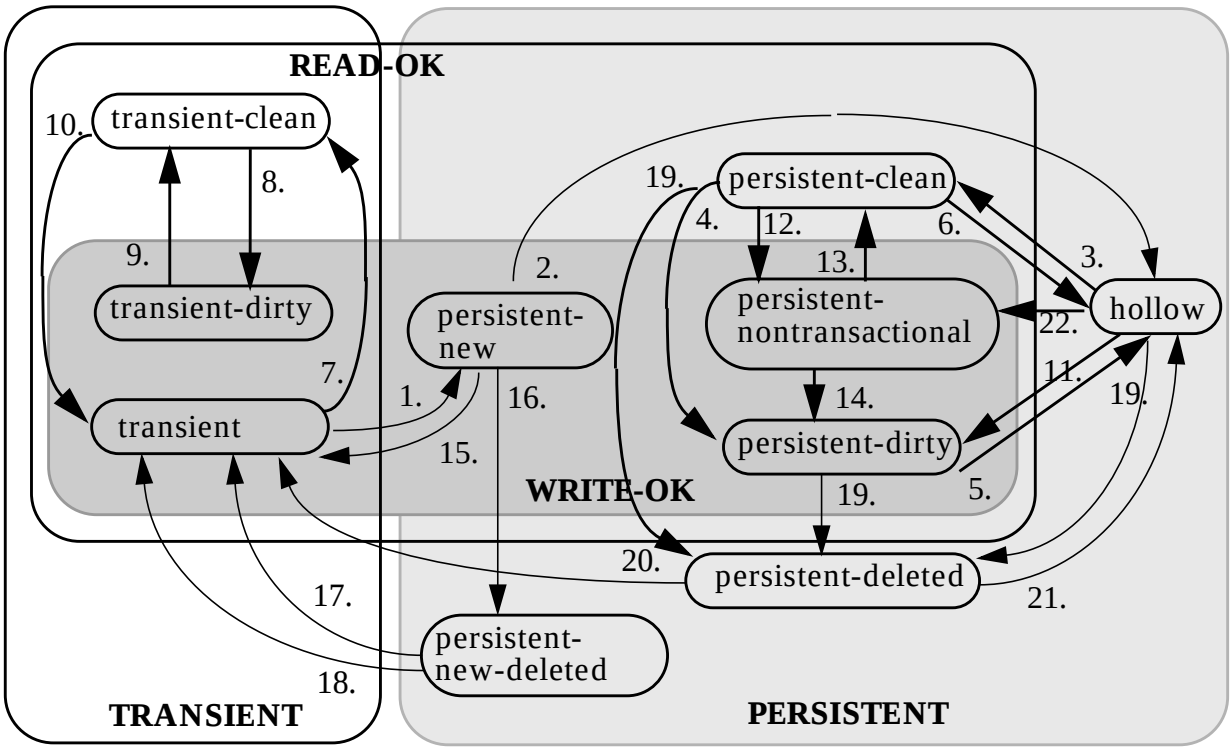


Figure 13.0 JDO Instance State Transitions



NOTE: Not all possible state transitions are shown in this diagram.

1. A transient instance transitions to persistent-new when the instance is the parameter of a `makePersistent` method.
2. A persistent-new instance transitions to hollow when the transaction in which it was made persistent commits.
3. A hollow instance transitions to persistent-clean when a field is read.
4. A persistent-clean instance transitions to persistent-dirty when a field is written.
5. A persistent-dirty instance transitions to hollow at commit or rollback.
6. A persistent-clean instance transitions to hollow at commit or rollback.
7. A transient instance transitions to transient-clean when it is the parameter of a `makeTransactional` method.
8. A transient-clean instance transitions to transient-dirty when a field is written.
9. A transient-dirty instance transitions to transient-clean at commit or rollback.
10. A transient-clean instance transitions to transient when it is the parameter of a `makeNontransactional` method.
11. A hollow instance transitions to persistent-dirty when a field is written.
12. A persistent-clean instance transitions to persistent-nontransactional when it is the parameter of a `makeNontransactional` method.

13. A persistent-nontransactional instance transitions to persistent-clean when it is the parameter of a `makeTransactional` method.
14. A persistent-nontransactional instance transitions to persistent-dirty when a field is written in a transaction.
15. A persistent-new instance transitions to transient on rollback.
16. A persistent-new instance transitions to persistent-new-deleted when it is the parameter of `deletePersistent`.
17. A persistent-new-deleted instance transitions to transient on rollback. The values of the fields are restored as of the `makePersistent` method.
18. A persistent-new-deleted instance transitions to transient on commit. The instance is cleared of values.
19. A hollow, persistent-clean, or persistent-dirty instance transitions to persistent-deleted when it is the parameter of `deletePersistent`.
20. A persistent-deleted instance transitions to transient when the transaction in which it was deleted commits.
21. A persistent-deleted instance transitions to hollow when the transaction in which it was deleted rolls back.
22. A hollow instance transitions to persistent-nontransactional when the `NontransactionalRead` option is set to `true,` a field is read, and there is either an optimistic transaction or no transaction active.

6 The Persistent Object Model

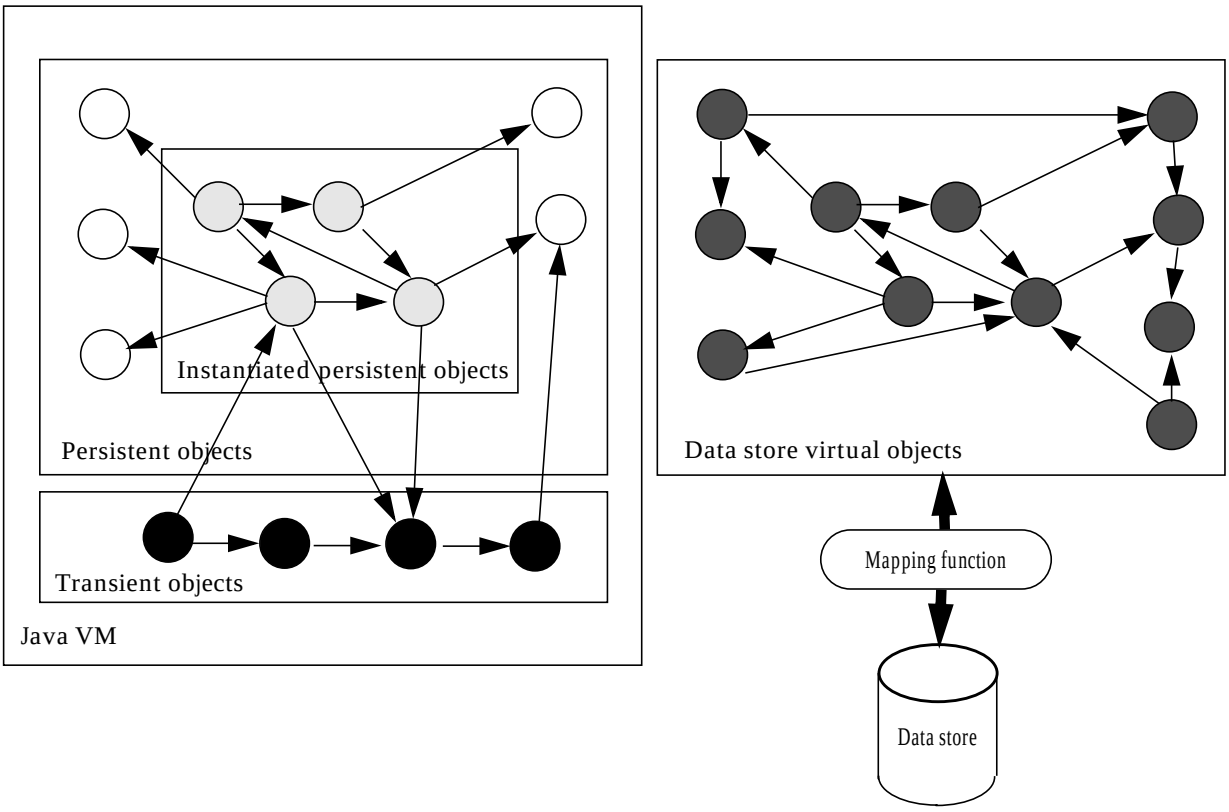
This chapter specifies the object model for persistence capable classes. To the extent possible, the object model is the same as the Java object model. Differences between the Java object model and the JDO object model are highlighted.

6.1 Overview

The Java execution environment supports different kinds of classes that are of interest to the developer. The classes that model the application and business domain are the primary focus of JDO. In a typical application, application classes are highly interconnected, and the graph of instances of those classes includes the entire contents of the data store.

Applications typically deal with a small number of persistent instances at a time, and it is the function of JDO to allow the illusion that the application can access the entire graph of connected instances, while in reality only small subset of instances needs to be instantiated in the JVM. This concept is called transparent data access, transparent persistence, or simply transparency.

Figure 14.0 Instantiated persistent objects



Within a JVM, there may be multiple independent units of work that must be isolated from each other. This isolation imposes requirements on JDO to permit the instantiation of the same data store object into multiple Java instances. The connected graph of Java instances is only a subset of the entire contents of the data store. Whenever a reference is followed from one persistent instance to another, the JDO implementation transparently instantiates the required instance into the JVM.

The storage of objects in data stores might be quite different from the storage of objects in the JVM. Therefore, there is a mapping between the Java instances and the objects in the data store. This mapping is performed by the JDO implementation, using metadata that is available at runtime. The metadata is generated by a JDO vendor-supplied tool, in cooperation with the deployer of the system. The mapping is not standardized by JDO.

JDO instances are stored in the data store and retrieved, possibly field by field, from the data store at specific points in their life cycle. The class developer might use callbacks at certain points to make a JDO instance ready for execution in the JVM, or make a JDO instance ready to be removed from the JVM. While executing in the JVM, a JDO instance might be connected to other instances, both persistent and transient.

There is no restriction on the types of non-persistent fields of persistence-capable classes. These fields behave exactly as defined by the Java language. Persistent fields of persistence-capable classes have restrictions in JDO, based on the characteristics of the types of the fields in the class definition.

6.2 Goals

The JDO Object Model has the following objectives:

- All field types supported by the Java language, including primitive types, reference types and interface types should be supported by JDO instances.
- All class and field modifiers supported by the Java language including private, public, protected, static, transient, abstract, final, synchronized, and volatile, should be supported by JDO instances.
- All user-defined classes should be allowed to be persistence-capable.
- Some system-defined classes (especially those for modeling state) should be persistence-capable.

6.3 Architecture

In Java, variables (including fields of classes) have types. Types are either primitive types or reference types. Reference types are either classes or interfaces. Arrays are treated as classes.

Instances are of a specific class, determined when the instance is constructed. Instances may be assigned to variables if they are assignment compatible with the variable type.

The JDO Object Model distinguishes between two kinds of classes: those that are persistence-capable, and those that are not. User-defined classes might be persistence-capable unless their state depends on the state of inaccessible or remote objects (e.g. they extend java.net.SocketImpl, or implement their behavior by using native calls).

System-defined classes (those defined in java.lang, java.io, java.net, etc.) are not persistence-capable, nor are they allowed to be types of persistent fields, unless they are addressed by the JDO specification.

First Class Objects

First Class Objects are represented by instances of persistence-capable classes that have JDO Identity and are stored in a data store together with their associated Second Class Objects and primitive values. They support uniquing: whenever a First Class Object is instantiated into memory, there is guaranteed to be only one instance representing that First Class Object managed by the same `PersistenceManager` instance. They are passed as arguments by reference.

First Class Objects can be shared among multiple First Class Objects, and if a First Class Object is changed (and the change is committed to the data store), then the changes are visible to all other First Class Objects which refer to it.

Second Class Objects

Second Class Objects are instances of either `PersistenceCapable` classes or classes that are provided by the JDO implementation to extend system-defined classes. They have no JDO Identity of their own. They track changes made to themselves and notify their owning First Class Object that they have changed, and the change is reflected as a change to that First Class Object (e.g. the owning instance might change state from persistent-clean to persistent-dirty). They are stored in the data store only as part of a First Class Object. They do not support uniquing, and their Java object identity is lost when the owning First Class Object is flushed to the data store. They are passed as arguments by reference.

Second Class Objects might be owned by a First Class Object or unowned. Sharing of owned Second Class Objects among multiple First Class Objects is not permitted. If an unowned Second Class Object shared among multiple First Class Objects is changed (and the change is committed to the data store), then the changes are seen only by those First Class Objects that are marked dirty.

It is possible for an application to assign the same instance of an unowned Second Class Object to multiple First Class Objects, but after the First Class Objects are committed to the data store, the Java object identity of the owned Second Class Objects might change.

When a First Class Object is instantiated in the JVM, the JDO implementation assigns to fields with a Second Class Object type a new instance that tracks changes made to itself, and notifies the owning First Class Object of the change.

Therefore, the application cannot assume that it knows the actual class of instances assigned to Second Class Object fields, although it is guaranteed that the actual class is assignment compatible with the type.

The only difference visible to the application between a field mapped to a First Class Object and a Second Class Object is in sharing. If a First Class Object “F” is assigned to a persistent field in two different First Class Objects “A” and “B”, then any changes at any time to instance “F” will be visible from instances “A” and “B”.

If a Second Class Object “S” is assigned to a persistent field in two different First Class Objects “A” and “B”, then any changes to instance “S” will be visible from instances “A” and “B” only until instances “A” and “B” are committed. After commit, instance “S” might not be referenced by either “A” or “B”, and any changes made to “S” might not be reflected in either “A” or “B”.

Arrays

Arrays are system-defined classes that do not necessarily have any JDO Identity of their own. They might be stored in the data store as part of a First Class Object. They do not support uniquing, and their Java object identity is lost when the owning First Class Object is flushed to the data store. They are passed as arguments by reference.

Tracking changes to Arrays is not required to be done by a JDO implementation. If an Array owned by a First Class Object is changed, then the changes might not be flushed to the data store. Portable applications must not require that these changes be tracked. In order for changes to arrays to be tracked, the application must explicitly notify the owning First Class Object of the change to the Array by calling the `jdoMakeDirty` method of the `PersistenceCapable` interface (or `makeDirty` of the `JDOHelper` class).

Furthermore, an implementation is permitted, but not required to, track changes to Arrays passed as references outside the body of methods of the owning class. There is a method defined on interface `PersistenceCapable` that allows the application to mark the field containing such an Array to be modified so its changes can be tracked. Portable applications must not require that these changes be tracked automatically.

It is possible for an application to assign the same instance of an Array to multiple First Class Objects, but after the First Class Object is flushed to the data store, the Java object identity of the Array might change.

When a First Class Object is instantiated in the JVM, the JDO implementation assigns to fields with an Array type a new instance with a different Java object identity from the instance stored.

Therefore, the application cannot assume that it knows the identity of instances assigned to Array fields, although it is guaranteed that the actual value is the same as the value stored.

Primitives

Primitives are types defined in the Java language and comprise `boolean, byte, short, int, long, char, float,` and `double`. They might be stored in the data store only as part of a First Class Object. They have no Java identity and no data store identity of their own. They are passed as arguments by value.

Interfaces

Interfaces are types whose values may be instances of any class that declare that they implement that interface.

6.4 Field types of persistent-capable classes

6.4.1 Nontransactional non-persistent fields

There are no restrictions on the types of nontransactional non-persistent fields. These fields are managed entirely by the application, not by the JDO implementation. Their state is not preserved by the JDO implementation, although they might be modified during execution of user-written callbacks defined in interface `InstanceCallbacks` at specific points in the life cycle, or any time during the instance's existence in the JVM.

6.4.2 Transactional non-persistent fields

There are no restrictions on the types of transactional non-persistent fields. These fields are partly managed by the JDO implementation. Their state is preserved and restored by the JDO implementation during certain state transitions.

6.4.3 Persistent fields

Primitive types

JDO implementations must support fields of any of the primitive types

- `boolean, byte, short, int, long, char, float,` and `double`.

Primitive values are stored in the data store associated with their owning First Class Object. They have no JDO Identity.

Immutable Object Class types

JDO implementations must support fields of immutable object classes, and may choose to support them as Second Class Objects or First Class Objects:

- `package java.lang: Boolean, Character, Byte, Short, Integer, Long, Float, Double,` and `String;`
- `package java.util: Locale;`
- `package java.math: BigDecimal, BigInteger`.

Portable JDO applications must not depend on whether these fields are treated as Second Class Objects or First Class Objects.

Mutable Object Class types

JDO implementations must support fields of the following mutable object classes, and may choose to support them as Second Class Objects or First Class Objects:

- `package java.util: Date, HashSet`.

JDO implementations may optionally support fields of the following mutable object classes, and may choose to support them as Second Class Objects or First Class Objects:

- `package java.util:ArrayList, HashMap, Hashtable, LinkedList, TreeMap, TreeSet`, and `Vector`.

Because the treatment of these fields may be as Second Class Object, the behavior of these mutable object classes when used in a persistent instance is not identical to their behavior in a transient instance.

Portable JDO applications must not depend on whether these fields are treated as Second Class Objects or First Class Objects.

PersistenceCapable Class types

JDO implementations must support fields of `PersistenceCapable` class types as First Class Objects, and are permitted, but not required, to support these field types as Second Class Objects.

Portable JDO applications must not depend on whether these fields are treated as Second Class Objects or First Class Objects.

Object Class type

JDO implementations must support fields of `Object` class type as First Class Objects. The implementation is permitted, but is not required, to allow any class to be assigned to the field. If an implementation restricts instances to be assigned to the field, a `ClassCastException` must be thrown at the time of any incorrect assignment.

Portable JDO applications must not depend on whether these fields are treated as Second Class Objects or First Class Objects.

Collection Interface types

JDO implementations must support fields of interface types, and may choose to support them as Second Class Objects or First Class Objects: `package java.util`: `Collection, Map, Set,` and `List`. `Collection` and `Set` are required; `Map` and `List` are optional.

Portable JDO applications must not depend on whether these fields are treated as Second Class Objects or First Class Objects.

Other Interface types

JDO implementations must support fields of interface types other than `Collection` interface types as First Class Objects. The implementation is permitted, but is not required, to allow any class that implements the interface to be assigned to the field. If an implementation further restricts instances that can be assigned to the field, a `ClassCastException` must be thrown at the time of any incorrect assignment.

Portable JDO applications should treat these fields as First Class Objects.

Arrays

JDO implementations may optionally support fields of array types, and may choose to support them as Second Class Objects or First Class Objects. If Arrays are supported by JDO implementations, they are permitted, but not required, to track changes made to Arrays that are fields of persistence capable classes in the methods of the classes. They need not track changes made to Arrays that are passed by reference as arguments to methods, including methods of persistence-capable classes.

Portable JDO applications must not depend on whether these fields are treated as Second Class Objects or First Class Objects.

6.5 Inheritance

A class might be persistence-capable even if its superclass is not persistence-capable. This allows users to extend classes that were not designed to be persistence-capable. If a class is persistence-capable, then its subclasses might or might not be persistence-capable themselves.

Further, subclasses of such classes that are not persistence-capable might be persistence-capable. That is, it is possible for classes in the inheritance hierarchy to be independently persistence-capable and not persistence-capable. It is not sufficient to test if a class implements `PersistenceCapable` (e.g. testing `anInstance instanceof PersistenceCapable`) to determine whether an instance is allowed to be stored.

Fields identified in the xml metadata as persistent or transactional in persistence-capable classes must be declared by the class, so that the values of the fields can be managed by behavior in the persistence-capable class, and not rely on behavior in the superclass in which the fields are visible.

Fields identified as persistent in persistence-capable classes will be persistent in subclasses; fields identified as transactional in persistence-capable classes will be transactional in subclasses; and fields identified as non-persistent in persistence-capable classes will be non-persistent in subclasses.

Of course, a class might define a new field with the same name as the field declared in the superclass, and might define it with a different persistence-modifier from the inherited field. But Java treats the declared field as a different field from the inherited field, so there is no conflict.

Any non-persistence-capable class which is a direct supertype of a persistence capable class must have an accessible no-args constructor.

Persistence-capable classes that use application identity have special considerations for inheritance. To be portable, the key class must be the same for all classes in the inheritance

hierarchy, and key fields must be declared only in the least-derived (topmost) persistence-capable class in the hierarchy.

7 PersistenceCapable

Every instance that is managed by a JDO `PersistenceManager` must be of a class that implements the public `PersistenceCapable` interface. This interface defines methods that allow the implementation to manage the instances. It also defines methods that allow a JDO aware application to examine the runtime state of instances, for example to discover whether the instance is transient, persistent, transactional, dirty, etc., and to discover its associated `PersistenceManager` if it has one.

The JDO Reference Enhancer modifies the class to implement `PersistenceCapable` prior to loading the class into the runtime environment. The enhancer additionally adds code to implement the methods defined by `PersistenceCapable`.

The `PersistenceCapable` interface is designed to avoid name conflicts in the scope of user-defined classes. All of its declared method names are prefixed with “jdo”.

Class implementors may explicitly declare that the class implements `PersistenceCapable`. If this is done, the implementor must implement the `PersistenceCapable` contract, and the enhancer will ignore the class instead of enhancing it.

The recommended approach for applications to interrogate the state of the instance is to use the class `JDOHelper` which provides static methods which delegate to the instance if it implements `PersistenceCapable`, and if not, returns the values that would have been returned by a transient instance.

7.1 Persistence Manager

```
PersistenceManager jdoGetPersistenceManager();
```

This method returns the associated `PersistenceManager` or `null` if the instance is transient.

7.2 Make Dirty

```
void jdoMakeDirty (String fieldName);
```

This method marks the specified field dirty so that its values will be modified in the data store when the transaction in which the instance is modified is committed. The `fieldName` is the name of the field to be marked as dirty, optionally including the fully qualified package name and class name of the field. This method returns with no effect if the instance is not managed by a `StateManager`.

If the same name is used for multiple fields (a class declares a field of the same name as a field in one of its superclasses) then the unqualified name refers to the most-derived class in which the field is declared to be persistent. The qualified name (className.fieldName) should always be used to identify the field to avoid ambiguity with subclass-defined fields.

The rationale for this is that a method in a superclass might call this method, and specify the name of the field that is hidden by a subclass. The `StateManager` has no way of

knowing which class called this method, and therefore assumes the Java rule regarding field names.

It is always safe to explicitly name the class and field referred to in the parameter to the method. The `StateManager` will resolve the scope of the name in the class named in the parameter.

For example, if class C inherits class B which inherits class A, and field X is declared in classes A and C, a method declared in class B may refer to the field in the method as "B.X" and it will refer to the field declared in class A. Field X is not declared in B; however, in the scope of class B, X refers to A.X.

7.3 JDO Identity

```
Object jdoGetObjectId();
```

This method returns the JDO identity of the instance. If the instance is transient, `null` is returned. If the identity is being changed in a transaction, this method returns the identity as of the beginning of the transaction.

```
Object jdoGetTransactionalObjectId();
```

This method returns the JDO identity of the instance. If the instance is transient, `null` is returned. If the identity is being changed in a transaction, this method returns the current identity in the transaction.

7.4 Status interrogation

The status interrogation methods return a boolean that represents the state of the instance:

7.4.1 Dirty

```
boolean jdoIsDirty();
```

Instances whose state has been changed in the current transaction return `true`. If the instance is transient, `false` is returned.

7.4.2 Transactional

```
boolean jdoIsTransactional();
```

Instances whose state is associated with the current transaction return `true`. If the instance is transient, `false` is returned.

7.4.3 Persistent

```
boolean jdoIsPersistent();
```

Instances that represent persistent objects in the data store return `true`. If the instance is transient, `false` is returned.

7.4.4 New

```
boolean jdoIsNew();
```

Instances that have been made persistent in the current transaction return `true`. If the instance is transient, `false` is returned.

7.4.5 Deleted

```
boolean jdoIsDeleted();
```

Instances that have been deleted in the current transaction return `true`. If the instance is transient, `false` is returned.

Table 3: State interrogation

	Persistent	Transactional	Dirty	New	Deleted
Transient					
Transient-clean		✓			
Transient-dirty		✓	✓		
Persistent-new	✓	✓	✓	✓	
Persistent-nontransactional	✓				
Persistent-clean	✓	✓			
Persistent-dirty	✓	✓	✓		
Hollow	✓				
Persistent-deleted	✓	✓	✓		✓
Persistent-new-deleted	✓	✓	✓	✓	✓

7.5 New instance

```
PersistenceCapable jdoNewInstance(StateManager sm);
```

This method creates a new instance of the class of the instance. It is intended to be used as a performance optimization compared to constructing a new instance by reflection using the constructor. It is intended to be used only by JDO implementations, not by applications. If the class is abstract, `null` is returned.

```
PersistenceCapable jdoNewInstance(StateManager sm, Object oid);
```

This method creates a new instance of the class of the instance, and copies key field values from the oid parameter instance. It is intended to be used as a performance optimization compared to constructing a new instance by reflection using the constructor, and copying values from the oid instance by reflection. It is intended to be used only by JDO implementations for classes that use application identity, not by applications. If the class is abstract, `null` is returned.

7.6 State Manager

```
void jdoReplaceStateManager (StateManager sm)
   throws IllegalAccessException;
```

This method sets the jdo`StateManager` field to the parameter. This method is normally used by the `StateManager` during the process of making an instance persistent, transactional, or transient. The caller of this method must have `JDOPermission` for the instance, otherwise `IllegalAccessException` is thrown.

7.7 Replace Flags

```
void jdoReplaceFlags ();
```

This method tells the instance to call the owning `StateManager`'s `replacingFlags` method to get a new value for the `jdoFlags` field.

7.8 Replace Fields

```
void jdoReplaceField (int fieldNumber);
```

This method gets a new value from the `StateManager` for the field specified in the parameter. The field number must refer to a field declared in this class or in a superclass.

```
void jdoReplaceFields (int[] fieldNumbers);
```

This method iterates over the array of field numbers and calls `jdoReplaceField` for each one.

7.9 Provide Fields

```
void jdoProvideField (int fieldNumber);
```

This method provides the value of the specified field to the `StateManager`. The field number must refer to a field declared in this class or in a superclass.

```
void jdoProvideFields (int[] fieldNumbers);
```

This method iterates over the array of field numbers and calls `jdoProvideField` for each one.

7.10 Copy Fields

```
void jdoCopyFields (Object other, int[] fieldNumbers);
```

This method copies fields from another instance of the same class. This method can only be invoked when both `this` and `other` are managed by the same `StateManager`.

7.11 Static Fields

The following fields define the permitted values for the `jdoFlags` field.

```
public static final byte READ_WRITE_OK = 0;
public static final byte READ_OK = -1;
public static final byte LOAD_REQUIRED = 1;
```

The following fields define the permitted values for the `jdoFieldFlags` elements.

```
public static final byte CHECK_WRITE = 0;
public static final byte MEDIATE_WRITE = 1;
public static final byte CHECK_READ_WRITE = 2;
```

```
public static final byte MEDIATE_READ_WRITE = 3;
```

7.12 JDO identity handling

```
public Object jdoNewObjectIdInstance();
```

This method creates a new instance of the class used for JDO identity. It is only intended for application identity. If the class has been enhanced for datastore identity, then `null` is returned. If the class is abstract, `null` is returned.

```
public void jdoCopyKeyFieldsToObjectId (PersistenceCapable pc, Object oid);
```

This method copies all key fields from the first parameter to the second parameter. The first parameter must be of the same class, and the second parameter must be an instance of the JDO identity class.

```
public void jdoCopyKeyFieldsToObjectId (ObjectIdFieldManager fm, Object oid);
```

This method copies fields from the field manager instance to the second parameter instance. Each key field in the `ObjectId` class matching a key field in the `PersistenceCapable` class is set by the execution of this method. For each key field, the method of the `ObjectIdFieldManager` is called for the corresponding type of field.

interface ObjectIdFieldManager

```
boolean fetchBooleanField (int fieldNumber);
char fetchCharField (int fieldNumber);
short fetchShortField (int fieldNumber);
int fetchIntField (int fieldNumber);
long fetchLongField (int fieldNumber);
float fetchFloatField (int fieldNumber);
double fetchDoubleField (int fieldNumber);
String fetchStringField (int fieldNumber);
Object fetchObjectField (int fieldNumber);
```

These methods all fetch one field from the field manager. The returned value is stored in the object id instance. The generated code in the PersistenceCapable class calls a method in the field manager for each key field in the object id. The field number is sequential, starting at zero for the first key field, in alphabetical order of key field name.

8 JDOHelper

`JDOHelper` is a class with static methods that is intended for use by persistence-aware classes. It contains methods that allow interrogation of the persistent state of an instance of a persistence-capable class.

Each method delegates to the instance, if it implements `PersistenceCapable`. Otherwise, if the method returns a value of reference type, it returns `null`; if the method returns a value of boolean type, it returns `false`; and if the method returns `void`, there is no effect.

8.1 Persistence Manager

```
static PersistenceManager getPersistenceManager (Object pc);
```

This method returns the associated `PersistenceManager` or `null` if the instance is transient.

See also `PersistenceCapable.jdoGetPersistenceManager()`.

8.2 Make Dirty

```
static void makeDirty (Object pc, String fieldName);
```

This method marks the specified field dirty so that its values will be modified in the data store when the instance is flushed. The `fieldName` is the name of the field to be marked as dirty, optionally including the fully qualified package name and class name of the field. This method is ignored if the instance is transient.

See also `PersistenceCapable.jdoMakeDirty(String fieldName)`.

8.3 JDO Identity

```
static Object getObjectId (Object pc);
```

This method returns the JDO identity of the instance. If the instance is transient, `null` is returned. If the identity is being changed in a transaction, this method returns the identity as of the beginning of the transaction.

See also `PersistenceCapable.jdoGetObjectId()` and `PersistenceManager.getObjectId(Object pc)`.

```
static Object getTransactionalObjectId (Object pc);
```

This method returns the JDO identity of the instance. If the instance is transient, `null` is returned. If the identity is being changed in a transaction, this method returns the current identity in the transaction.

See also `PersistenceCapable.jdoGetTransactionalObjectId()` and `PersistenceManager.getTransactionalObjectId(Object pc)`.

8.4 Status interrogation

The status interrogation methods return a `boolean` that represents the state of the instance:

8.4.1 Dirty

```
static boolean isDirty (Object pc);
```

Instances whose state has been changed in the current transaction return `true`. If the instance is transient, `false` is returned.

See also `PersistenceCapable.jdoIsDirty();`

8.4.2 Transactional

```
static boolean isTransactional (Object pc);
```

Instances whose state is associated with the current transaction return `true`. If the instance is transient, `false` is returned.

See also `PersistenceCapable.jdoIsTransactional()`.

8.4.3 Persistent

```
static boolean isPersistent (Object pc);
```

Instances that represent persistent objects in the data store return `true`. If the instance is transient, `false` is returned.

See also `PersistenceCapable.jdoIsPersistent();`

8.4.4 New

```
static boolean isNew (Object pc);
```

Instances that have been made persistent in the current transaction return `true`. If the instance is transient, `false` is returned.

See also `PersistenceCapable.jdoIsNew();`

8.4.5 Deleted

```
static boolean isDeleted (Object pc);
```

Instances that have been deleted in the current transaction return `true`. If the instance is transient, `false` is returned.

See also `PersistenceCapable.jdoIsDeleted();`

9 JDOImplHelper

This class is a public helper class for use by JDO implementations. It contains a registry of metadata by class. Use of the methods in this class avoids the use of reflection at runtime. `PersistenceCapable` classes register metadata with this class during class initialization.

```
public JDOImplHelper {
```

9.1 JDOImplHelper access

```
public static JDOImplHelper getInstance()
   throws IllegalAccessException;
```

This method returns an instance of the `JDOImplHelper` class if the caller is authorized for `JDOPermission("getMetadata")`. This instance gives access to all of the other methods, except for `registerClass`, which does not need any authorization.

9.2 Metadata access

```
public String[] getFieldNames (Class pcClass);
```

This method returns the names of persistent and transactional fields of the parameter class. If the class does not implement `PersistenceCapable`, or if it has not been enhanced correctly to register its metadata, a `JDOFatalUserException` is thrown.

Otherwise, the names of fields that are either persistent or transactional are returned, in order. The order of names in the returned array are the same as the field numbering. Relative field 0 refers to the first field in the array. The length of the array is the number of persistent and transactional fields in the class.

```
public Class[] getFieldTypes (Class pcClass);
```

This method returns the types of persistent and transactional fields of the parameter class. If the parameter does not implement `PersistenceCapable`, or if it has not been enhanced correctly to register its metadata, a `JDOFatalUserException` is thrown.

Otherwise, the types of fields that are either persistent or transactional are returned, in order. The order of types in the returned array is the same as the field numbering. Relative field 0 refers to the first field in the array. The length of the array is the number of persistent and transactional fields in the class.

```
public byte[] getFieldFlags (Class pcClass);
```

This method returns the field flags of persistent and transactional fields of the parameter class. If the parameter does not implement `PersistenceCapable`, or if it has not been enhanced correctly to register its metadata, a `JDOFatalUserException` is thrown.

Otherwise, the types of fields that are either persistent or transactional are returned, in order. The order of types in the returned array is the same as the field numbering. Relative

field 0 refers to the first field in the array. The length of the array is the number of persistent and transactional fields in the class.

```
public Class getPersistenceCapableSuperclass (Class pcClass);
```

This method returns the `PersistenceCapable` superclass of the parameter class, or `null` if there is none.

9.3 Persistence-capable instance factory

```
public PersistenceCapable newInstance (Class pcClass,
   StateManager sm);
public PersistenceCapable newInstance (Class pcClass, StateManager
sm, Object oid);
```

If the class does not implement `PersistenceCapable`, or if it has not been enhanced correctly to register its metadata, a `JDOFatalUserException` is thrown. If the class is abstract, a `JDOFatalInternalException` is thrown.

Otherwise, a new instance of the class is constructed and initialized with the parameter `StateManager`. The new instance has its `jdoFlags` set to `LOAD_REQUIRED` but has no defined state. The behavior of the instance is determined by the owning `StateManager`.

The second form of the method returns a new instance of `PersistenceCapable` that has had its key fields initialized by the `ObjectId` parameter instance.

See also `PersistenceCapable.jdoNewInstance (StateManager sm)` and `PersistenceCapable.jdoNewInstance (StateManager sm, Object oid)`.

9.4 Registration of PersistenceCapable classes

```
public static void registerClass
   (Class pcClass, String[] fieldNames,
      Class[] fieldTypes,
      byte[] fieldFlags,
      Class persistenceCapableSuperclass,
      PersistenceCapable pcInstance);
```

This method registers a `PersistenceCapable` class so that the other methods can return the correct information. The registration must be done in a static initializer for the persistence-capable class.

9.5 Application identity handling

```
public Object newObjectIdInstance(Class pcClass);
```

This method creates a new instance of the `Object Id` class for the `PersistenceCapable` class. If the class uses data store identity, then `null` is returned. If the class is abstract, a `JDOFatalInternalException` is thrown.

```
public void copyKeyFieldsToObjectId (Class pcClass, PersistenceCa-
pable pc, Object oid);
```

This method copies key fields from the `PersistenceCapable` instance pc to the `Object Id` instance oid. This is intended for use by the implementation to instantiate an object to

return from `getObjectId`. If the class is abstract, a `JDOFatalInternalException` is thrown.

```
public void copyKeyFieldsToObjectId (Class pcClass, PersistenceCa-
pable.ObjectIdFieldManager fm, Object oid);
```

This method copies key fields from the field manager to the `Object Id` instance oid. This is intended for use by the implementation to copy fields from a data store-specific representation to the `Object Id`. If the class is abstract, a `JDOFatalInternalException` is thrown.

10 InstanceCallbacks

Instance callbacks provide a mechanism for instances to take some action on specific JDO instance life cycle events. For example, classes which include non-persistent fields might use callbacks in order to correctly populate the values in these fields. Classes that affect the runtime environment might use callbacks to register and deregister themselves with other objects. This interface defines the methods executed by the `StateManager` for these life cycle events.

If the class implements `InstanceCallbacks`, it must explicitly declare it in the class definition in order for the `StateManager` to recognize that the callbacks should be made.

10.1 jdoPostLoad

```
public void jdoPostLoad();
```

This method is called after the values are loaded from the `StateManager` into the instance. Non-persistent fields should be initialized in this method. This method is not modified by the enhancer. Only fields that are in the default fetch group should be accessed by this method, as other fields are not guaranteed to be initialized. This method might register the instance with other objects in the runtime environment.

The context in which this call is made does not allow access to other persistent JDO instances.

10.2 jdoPreStore

```
public void jdoPreStore();
```

This method is called before the values are stored from the instance to the `StateManager`. Data store fields that might have been affected by modified non-persistent fields should be updated in this method. This method is modified by the enhancer so that changes to persistent fields will be reflected in the data store.

The context in which this call is made allows access to the `PersistenceManager` and other persistent JDO instances.

10.3 jdoPreClear

```
public void jdoPreClear();
```

This method is called before the values in the instance are cleared. This happens during the state transition to hollow. Non-persistent, non-transactional fields should be cleared in this method. Associations between this instance and others in the runtime environment should be cleared. This method is not modified by the enhancer.

10.4 jdoPreDelete

```
public void jdoPreDelete();
```

This method is called before the state transition to persistent-deleted or persistent-new-deleted. Access to field values within this call are valid. Access to field values after this call are disallowed. This method is modified by the enhancer so that fields referenced can be used in the business logic of the method.

To implement a containment aggregate, the user could implement this method to delete contained persistent instances.

11 PersistenceManagerFactory

This chapter details the `PersistenceManagerFactory`, which is responsible for creating `PersistenceManager` instances for application use.

11.1 Interface PersistenceManagerFactory

A JDO vendor must provide a class that implements `PersistenceManagerFactory` and is permitted to provide a `PersistenceManager` constructor[s].

A non-managed JDO application might choose to use a `PersistenceManager` constructor (JDO vendor specific) or use a `PersistenceManagerFactory` (provided by the JDO vendor). A portable JDO application must use the `PersistenceManagerFactory`.

In a managed environment, the JDO `PersistenceManager` instance is acquired by a two step process: the application uses JNDI lookup to retrieve an environment-named object, which is then cast to `javax.jdo.PersistenceManagerFactory`; and then calls one of the factory's `getPersistenceManager` methods.

In a non-managed environment, the JDO `PersistenceManager` instance is acquired by lookup as above; by constructing a `javax.jdo.PersistenceManager`; or by constructing a `javax.jdo.PersistenceManagerFactory`, configuring the factory, and then calling the factory's `getPersistenceManager` method. These constructors are not part of the JDO standard. However, the following is recommended to support portable applications.

Configuring the `PersistenceManagerFactory` follows the Java Beans pattern. Supported properties have a `get` method and a `set` method.

The following properties, if set in the `PersistenceManagerFactory`, are the default settings of all `PersistenceManager` instances created by the factory:

- `Optimistic`: the transaction mode that specifies concurrency control
- `RetainValues`: the transaction mode that specifies the treatment of persistent instances after commit
- `IgnoreCache`: the query mode that specifies whether cached instances are considered when evaluating the filter expression
- `NontransactionalRead`: the `PersistenceManager` mode that allows instances to be read outside a transaction
- `NontransactionalWrite`: the `PersistenceManager` mode that allows instances to be written outside a transaction
- `MaxPool`: the maximum number of `PersistenceManager` instances in the pool
- `MinPool`: the minimum number of `PersistenceManager` instances in the pool
- `MsWait`: the number of milliseconds to wait for an available `PersistenceManager` from the pool before `getPersistenceManager` throws a `JDODatastoreException`

- `Multithreaded`: the `PersistenceManager` mode that indicates that the application will invoke methods or access fields of managed instances from multiple threads.

The following properties are for convenience, if there is no connection pooling or other need for a connection factory:

- `ConnectionUserName`: the name of the user establishing the connection
- `ConnectionPassword`: the password for the user
- `ConnectionURL`: the URL for the data source
- `ConnectionDriverName`: the driver name for the connection

For a portable application, if any other connection properties are required, then a connection factory must be configured.

The following properties are for use when a connection factory is used, and override the connection properties specified in `ConnectionURL`, `ConnectionUserName`, or `ConnectionPassword`.

- `ConnectionFactory`: the connection factory from which data store connections are obtained
- `ConnectionFactoryName`: the name of the connection factory from which data store connections are obtained. This name is looked up with JNDI to locate the connection factory.

If multiple connection properties are set, then they are evaluated in order:

- if `ConnectionFactory` is specified (not `null`), all other properties are ignored;
- else if `ConnectionFactoryName` is specified (not `null`), all other properties are ignored.

For the application server environment, connection factories always return connections that are enlisted in the thread's current transaction context. To use optimistic transactions in this environment requires a connection factory that returns connections that are not enlisted in the current transaction context. For this purpose, the following two properties are used:

- `ConnectionFactory2`: the connection factory from which nontransactional data store connections are obtained
- `ConnectionFactory2Name`: the name of the connection factory from which nontransactional data store connections are obtained. This name is looked up with JNDI to locate the connection factory.

11.2 ConnectionFactory

For implementations that layer on top of standard `Connector` implementations, the configuration will typically support all of the associated `ConnectionFactory` properties.

When used in a managed environment, the `ConnectionFactory` will implement the contracts of `javax.resource: ConnectionManager, ManagedConnection, ManagedConnectionFactory, and LocalTransaction`.

The following properties of the `ConnectionFactory` should be used if the data source has a corresponding concept:

- `URL`: the URL for the data source

- `UserName`: the name of the user establishing the connection
- `Password`: the password for the user
- `DriverName`: the driver name for the connection
- `ServerName`: name of the server for the data source
- `PortNumber`: port number for establishing connection to the data source
- `MaxPool`: the maximum number of connections in the connection pool
- `MinPool`: the minimum number of connections in the connection pool
- `MsWait`: the number of milliseconds to wait for an available connection from the connection pool before throwing a `JDODatastoreException`
- `LogWriter`: the `PrintWriter` to which messages should be sent
- `LoginTimeout`: the number of seconds to wait for a new connection to be established to the data source

In addition to these properties, the `PersistenceManagerFactory` implementation class can support properties specific to the data source or to the `PersistenceManager`.

Aside from vendor-specific configuration APIs, there are three required methods for `PersistenceManagerFactory`.

11.3 PersistenceManager access

```
PersistenceManager getPersistenceManager();
```

```
PersistenceManager getPersistenceManager(String userid, String password);
```

Returns a `PersistenceManager` instance with the configured properties. The instance might have come from a pool of instances. The default values for option settings are reset to the value specified in the `PersistenceManagerFactory` before returning the instance.

After the first use of `getPersistenceManager`, none of the `set` methods will succeed. The settings of operational parameters might be modified dynamically during runtime via a vendor-specific interface.

If the method with the userid and password is used to acquire the `PersistenceManager`, then all accesses to the connection factory during the life of the `PersistenceManager` will use the userid and password to get connections. If `PersistenceManager` instances are pooled, then only `PersistenceManager` instances with the same userid and password will be used to satisfy the request.

11.4 Non-configurable Properties

The JDO vendor might store certain non-configurable properties and make those properties available to the application via a `Properties` instance. This method retrieves the `Properties` instance.

```
Properties getProperties();
```

The application is not prevented from modifying the instance.

Each key and value is a `String`. The keys defined for standard JDO implementations are:

- `VendorName`: The name of the JDO vendor.

- `VersionNumber`: The version number string.

Other properties are vendor-specific.

11.5 Optional Feature Support

```
Collection supportedOptions();
```

The JDO implementation might optionally support certain features, and will report the features that are supported. The supported query languages are included in the returned `Collection`.

This method returns a `Collection` of `String`, each `String` instance representing an optional feature of the implementation or a supported query language. The following are the values of the `String` for each optional feature in the JDO specification:

`javax.jdo.option.TransientTransactional`

`javax.jdo.option.NontransactionalRead`

`javax.jdo.option.NontransactionalWrite`

`javax.jdo.option.RetainValues`

`javax.jdo.option.Optimistic`

`javax.jdo.option.ApplicationIdentity`

`javax.jdo.option.DatastoreIdentity`

`javax.jdo.option.NonDatastoreIdentity`

`javax.jdo.option.ArrayList`

`javax.jdo.option.HashMap`

`javax.jdo.option.Hashtable`

`javax.jdo.option.LinkedList`

`javax.jdo.option.TreeMap`

`javax.jdo.option.TreeSet`

`javax.jdo.option.Vector`

`javax.jdo.option.Map`

`javax.jdo.option.List`

`javax.jdo.option.Array`

`javax.jdo.option.NullCollection`

The standard JDO query must be returned as the `String`:

`javax.jdo.query.JDOQL`

Other query languages are represented by a `String` not defined in this specification.

12 PersistenceManager

This chapter specifies the JDO `PersistenceManager` and its relationship to the application components, JDO instances, and J2EE Connector.

12.1 Overview

The JDO `PersistenceManager` is the primary interface for JDO-aware application components. It is the factory for the `Query` interface and contains methods for managing the life cycle of persistent instances.

The JDO `PersistenceManager` interface is architected to support a variety of environments and data sources, from small footprint embedded systems to large enterprise application servers. It might be a layer on top of a standard Connector implementation such as JDBC or JMS, or itself include connection management and distributed transaction support.

J2EE Connector support is optional . If it is not supported by a JDO implementation, then a constructor for the JDO `PersistenceManager` or `PersistenceManagerFactory` is required. The details of the construction of the `PersistenceManager` or `PersistenceManagerFactory` are not specified by JDO.

12.2 Goals

The architecture of the PersistenceManager has the following goals:

- No changes to application programs to change to a different vendor's `PersistenceManager` if the application is written to conform to the portability guidelines
- Application to non-managed and managed environments with no code changes

12.3 Architecture: JDO PersistenceManager

The JDO `PersistenceManager` instance is visible only to certain application components: those that explicitly manage the life cycle of JDO instances; and those that query for JDO instances. The JDO `PersistenceManager` is not required to be used by JDO instances.

There are three primary environments in which the JDO `PersistenceManager` is architected to work:

- non-managed (non-application server), minimum function, single transaction, single JDO `PersistenceManager` where compactness is the primary metric;
- non-managed but where extended features are desired, such as multiple `PersistenceManager` instances to support multiple data sources, XA coordinated transactions, or nested transactions; and
- managed, where the full range of capabilities of an application server is required.

Support for these three environments is accomplished by implementing transaction completion APIs on a companion JDO `Transaction` instance, which contains transaction policy options and local transaction support.

12.4 Threading

It is a requirement for all JDO implementations to be thread-safe. That is, the behavior of the implementation must be predictable in the presence of multiple application threads. Operations implemented by the `PersistenceManager` directly or indirectly via access or modification of persistent or transactional fields of persistence-capable classes must be treated as if they were serialized. The implementation is free to serialize internal data structures and thus order multi-threaded operations in any way it chooses. The only application-visible behavior is that operations might block indefinitely (but not infinitely) while other operations complete.

Since synchronizing the `PersistenceManager` is a relatively expensive operation, and not needed in many applications, the application must specify whether multiple threads might access the same `PersistenceManager` or instances managed by the `PersistenceManager` (persistent or transactional instances of `PersistenceCapable` classes; instances of `Transaction` or `Query`; query results, etc.).

If applications depend on serializing operations, then the applications must implement the appropriate synchronizing behavior, using instances visible to the application. This includes some instances of the JDO implementation (e.g. `PersistenceManager`, `Query`, etc.) and instances of persistence-capable classes.

The implementation must not use user-visible instances (instances of `PersistenceManagerFactory, PersistenceManager, Transaction, Query`, etc.) as synchronization objects, with one exception. The implementation must synchronize instances of `PersistenceCapable` during state transitions that replace the `StateManager`. This is to avoid race conditions where the application attempts to make the same instance persistent in multiple `PersistenceManagers`.

12.5 Interface PersistenceManager

A JDO `PersistenceManager` instance supports any number of JDO instances at a time. It is responsible for managing the identity of its associated JDO instances. A JDO instance is associated with either zero or one JDO `PersistenceManager`. It will be zero if and only if the JDO instance is transient. As soon as the instance is made persistent or transactional, it will be associated with exactly one JDO `PersistenceManager`.

A JDO `PersistenceManager` instance supports one transaction at a time, and uses one connection to the underlying data source at a time. The JDO `PersistenceManager` instance might use multiple transactions serially, and might use multiple connections serially.

Therefore, to support multiple concurrent connection-oriented data sources in an application, multiple JDO `PersistenceManager` instances are required.

In this interface, JDO instances passed as parameters and returned as values must implement `PersistenceCapable`. However, the interface defines these formal parameters as `Object` because casting user classes to `PersistenceCapable` is awkward.

```
public interface javax.jdo.PersistenceManager {
boolean isClosed();
```

```
void close();
```

The `isClosed` method returns `false` upon construction of the `PersistenceManager` instance, or upon retrieval of a `PersistenceManager` from a pool. It returns `true` only after the `close` method completes successfully. After being closed, the `PersistenceManager` instance might be returned to the pool or garbage collected, at the choice of the JDO implementation. Before being used again to satisfy a `getPersistenceManager` request, the options will be reset to their default values as specified in the `PersistenceManagerFactory`.

After the `close` method completes, all methods on the `PersistenceManager` instance except `isClosed` throw a `JDOFatalUserException`.

Null management

In the APIs that follow, `Object[]` and `Collection` are permitted parameter types. As these may contain nulls, the following rules apply.

Null arguments to APIs which take an `Object` parameter cause the API to have no effect. Null arguments to APIs which take `Object[]` or `Collection` will cause the API to throw `NullPointerException`. Non-null `Object[]` or `Collection` arguments that contain null elements will have the documented behavior for non-null elements, and the null elements will be ignored.

12.5.1 Cache management

Normally, cache management is automatic and transparent. When instances are queried, navigated to, or modified, instantiation of instances and their fields and garbage collection of unreferenced instances occurs without any explicit control. When the transaction in which persistent instances are created, deleted, or modified commits, eviction is automatically done by the transaction completion mechanisms. Therefore, eviction is not normally required to be done explicitly. However, if the application chooses to become more involved in the management of the cache, several methods are available.

The non-parameter version of these methods applies the operation to each appropriate JDO instance in the cache. If an inappropriate instance is the implicit parameter of the method, the inappropriate instance is unchanged by the method.

```
void evict (Object pc);
void evictAll ();
void evictAll (Object[] pcs);
void evictAll (Collection pcs);
```

Eviction is a hint to the PersistenceManager that the application no longer needs the parameter instances in the cache. Eviction allows the parameter instances to be subsequently garbage collected. Evicted instances will not have their values retained after transaction completion, regardless of the setting of the `retainValues` flag.

If `evictAll` with no parameters is called, then all persistent-nontransactional instances are evicted. If users wish to automatically evict transactional instances at transaction completion time, then they should set `RetainValues` to `false`.

For each persistent-clean and persistent-nontransactional instance that the JDO `PersistenceManager` evicts, it:

- calls the `jdoPreClear` method on each instance, if the class of the instance implements `InstanceCallbacks`

- clears persistent fields on each instance (sets the value of the field to its Java default value);
- changes the state of instances to hollow.

```
void refresh (Object pc);
void refreshAll ();
void refreshAll (Object[] pcs);
void refreshAll (Collection pcs);
```

The `refresh` method updates the values in the parameter instance[s] from the data in the data store. The intended use is for optimistic transactions where the state of the JDO instance is not guaranteed to reflect the state in the data store. This method can be used to minimize the occurrence of commit failures due to mismatch between the state of cached instances and the state of data in the data store.

The `refreshAll` method with no parameters causes all transactional instances to be refreshed.

Note that this method will cause loss of changes made to affected instances by the application due to refreshing the contents from the data store.

The JDO `PersistenceManager`:

- loads persistent values from the data store;
- loads persistent fields into the instance;
- calls the `jdoPostLoad` method on each persistent instance, if the class of the instance implements `InstanceCallbacks`; and
- changes the state of persistent-dirty instances to persistent-clean.

12.5.2 Transaction factory interface

```
Transaction currentTransaction();
```

The `currentTransaction` method returns the `Transaction` instance associated with the `PersistenceManager`. If transaction completion is managed by a transaction manager (through the `XAResource`), the `Transaction` instance returned cannot be used for transaction completion, but can be used to set flags.

12.5.3 Query factory interface

The query factory methods are detailed in the Query chapter .

```
void setIgnoreCache (boolean flag);
boolean getIgnoreCache ();
```

These methods get and set the value of the `IgnoreCache` option for all `Query` instances created by this `PersistenceManager` [see Query options]. The `IgnoreCache` option is a hint to the query engine that the user expects queries be optimized to return approximate results by ignoring changed values in the cache.

12.5.4 Extent Management

Extents are collections of data store objects managed by the data store, not by explicit user operations on collections. Extent capability is a boolean property of classes that are persistence capable. If an instance of a class that has a managed extent is made persistent via reachability, the instance is put into the extent implicitly.

```
Extent getExtent (Class PersistenceCapableClass, boolean subclass-
es);
```

The `getExtent` method returns an unmodifiable `Extent` that contains all of the instances in the parameter class, and if the subclasses flag is `true`, all of the instances of the parameter class and its subclasses.

It might be a common usage to iterate over the contents of the `Extent`, and the `Extent` should be implemented in such a way as to avoid out-of-memory conditions on iteration.

The primary use for the `Extent` returned as a result of this method is as a candidate collection parameter to a `Query` instance. For this usage, the elements in the `Extent` typically will not be instantiated in the Java VM; it is only used to identify the prospective data store instances.

12.5.5 Second Class Object Factory

Methods are provided for the application to obtain instances of second class objects for use as field values for maps, collections and dates .

```
Object newSCOInstance (Class type, Object owner, String fieldName);
```

This method returns a new Second Class Object instance of the type specified, with the owner and field name to notify upon changes to the value of any of its fields. If a collection class is created, then the class does not restrict the element types, allows nulls to be added as elements, and has an implementation default initial size and load factor.

This method immediately assigns the newly created instance to the owner field, and makes the appropriate state transition of the owner.

```
Collection newCollectionInstance (Class type, Object owner, String
fieldName, Class elementType, boolean allowNulls, Integer initial-
Capacity, Float loadFactor, Collection initialContents) throws Il-
legalArgumentException;
```

This method returns a new `Collection` instance that implements or extends the type (or interface) specified, with the owner and field name to notify upon changes to the value of any of its fields. The type parameter is the `Class` instance of the Java `Collection` type needed. The collection class restricts the element types allowed to the `elementType` or instances assignable to the `elementType`, and allows `null`s to be added as elements based on the setting of `allowNulls`.

- If the `initialContents` parameter is `null`, the `Collection` has an initial capacity as specified by the `initialSize` parameter, and a load factor as specified by the `loadFactor` parameter. Defaults as specified in the Java language specification are taken where possible for null values of `initialCapacity` and `loadFactor`. Otherwise, implementation defaults are taken.
- If the `initialContents` parameter is not `null`, the initial contents of the `Collection` are copied from the `initialContents` parameter using shallow copy semantics, and the `initialCapacity` and `loadFactor` are determined by the constructor semantics of the Java language specification for the specified `Collection` type.

This method immediately assigns the newly created instance to the owner field, and makes the appropriate state transition of the owner.

```
Map newMapInstance (Class type, Object owner, String fieldName,
Class keyType, Class valueType, boolean allowNulls, Integer ini-
```

```
tialCapacity, Float loadFactor, Map initialContents) throws Ille-
galArgumentException;
```

This method returns a new `Map` instance that implements or extends the type (or interface) specified, with the owner and field name to notify upon changes to the value of any of its fields. The type parameter is the `Class` instance of the Java `Map` type needed. The map class restricts the key and value types allowed to the specific types or instances assignable to the specific types, and allows nulls to be added as elements based on the setting of `allowNulls`.

- If the `initialContents` parameter is `null`, the `Map` has an initial capacity as specified by the `initialCapacity` parameter, and a load factor as specified by the `loadFactor` parameter. Defaults as specified in the Java language specification are taken where possible for null values of `initialCapacity` and `loadFactor`. Otherwise, implementation defaults are taken.
- If the `initialContents` parameter is not `null`, the initial contents of the `Map` are copied from the `initialContents` parameter using shallow copy semantics, and the `initialCapacity` and `loadFactor` are determined by the constructor semantics of the Java language specification for the specified `Map` type. For sorted maps, if the `initialContents` parameter is not `null`, then the comparator is copied from the `initialContents` parameter.

This method immediately assigns the newly created instance to the owner field, and makes the appropriate state transition of the owner.

12.5.6 JDO Identity management

```
Object getObjectById (Object oid, boolean validate);
```

The `getObjectById` method attempts to find an instance in the cache with the specified JDO identity. The `oid` parameter object might have been returned by an earlier call to `getObjectId` or `getTransactionalObjectId`, or might have been constructed by the application.

If the `PersistenceManager` is unable to resolve the `oid` parameter to an `ObjectId` instance, then it throws a `JDOUserException`.

- If the `validate` flag is `false`, and there is already an instance in the cache with the same JDO identity as the oid parameter, then this method returns it. There is no change made to the state of the returned instance.

  If there is not an instance already in the cache with the same JDO identity as the oid parameter, then this method creates an instance with the specified JDO identity and returns it. If there is no transaction in progress, the returned instance will be hollow or persistent-nontransactional, at the choice of the implementation.

  If there is a transaction in progress, the returned instance will be hollow, persistent-nontransactional, or persistent-clean, at the choice of the implementation.

  It is an implementation decision whether to access the data store, if required to determine the exact class. This will be the case of inheritance, where multiple `PersistenceCapable` classes share the same `Object Id` class.

  If the `validate` flag is `false`, and the instance does not exist in the data store, then this method might not fail. It is an implementation choice whether to fail immediately with a `JDODatastoreException`.But a subsequent access of the

fields of the instance will throw a `JDODatastoreException` if the instance does not exist at that time. Further, if a relationship is established to this instance, then the transaction in which the association was made will fail.

- If the `validate` flag is `true`, and there is already a transactional instance in the cache with the same jdo identity as the oid parameter, then this method returns it. There is no change made to the state of the returned instance.

  If there is an instance already in the cache with the same jdo identity as the oid parameter, but the instance is not transactional, then it must be verified in the data store. If the instance does not exist in the datastore, then a `JDODatastoreException` is thrown.

  If there is not an instance already in the cache with the same jdo identity as the oid parameter, then this method creates an instance with the specified jdo identity, verifies that it exists in the data store, and returns it. If there is no transaction in progress, the returned instance will be hollow or persistent-nontransactional, at the choice of the implementation.

  If there is a data store transaction in progress, the returned instance will be persistent-clean.

  If there is an optimistic transaction in progress, the returned instance will be persistent-nontransactional.

```
Object getObjectId (Object pc);
```

The `getObjectId` method returns an `ObjectId` instance that represents the object identity of the specified JDO instance. The identity is guaranteed to be unique only in the context of the JDO `PersistenceManager` that created the identity, and only for two types of JDO Identity: those that are managed by the application, and those that are managed by the data store.

If the object identity is being changed in the transaction, by the application modifying one or more of the application key fields, then this method returns the identity as of the beginning of the transaction. The value returned by `getObjectId` will be different following `afterCompletion` processing for successful transactions.

Within a transaction, the `ObjectId` returned will compare equal to the `ObjectId` returned by only one among all JDO instances associated with the `PersistenceManager` regardless of the type of `ObjectId`.

The `ObjectId` does not necessarily contain any internal state of the instance, nor is it necessarily an instance of the class used to manage identity internally. Therefore, if the application makes a change to the `ObjectId` instance returned by this method, there is no effect on the instance from which the `ObjectId` was obtained.

The `getObjectById` method can be used between instances of `PersistenceManager` of different JDO vendors only for instances of persistence capable classes using application-managed (primary key) JDO identity. If it is used for instances of classes using datastore identity, the method might succeed, but there are no guarantees that the parameter and return instances are related in any way.

```
Object getTransactionalObjectId (Object pc);
```

If the object identity is being changed in the transaction, by the application modifying one or more of the application key fields, then this method returns the current identity in the transaction. If there is no transaction in progress, or if none of the key fields is being modified, then this method will return the same value as `getObjectId`.

To get an instance in a PersistenceManager with the same identity as an instance from a different PersistenceManager, use the following: aPersistenceManager.getObjectById(pc.getPersistenceManager().getObjectId(pc), validate).

12.5.7 JDO Instance life cycle management

The following methods take either a single instance or multiple instances as parameters.

If a collection or array of instances is passed to any of the methods in this section, and one or more of the instances fail to complete the required operation, then all instances will be attempted, and a JDOUserException will be thrown which contains nested exceptions, each of which contains one of the failing instances. The succeeding instances will transition to the specified life cycle state, and the failing instances will remain in their current state.

Make instances persistent

```
void makePersistent (Object pc);
void makePersistentAll (Object[] pcs);
void makePersistentAll (Collection pcs);
```

These methods make a transient instance persistent directly. They must be called in the context of an active transaction. They will assign an object identity to the instance and transition it to persistent-new. During commit of the transaction in which this instance was made persistent, any transient instances reachable from this instance via persistent fields of this instance will become persistent as well.

They have no effect on persistent instances already managed by this PersistenceManager. They will throw a JDOUserException if the instance is managed by a different PersistenceManager.

Delete persistent instances

```
void deletePersistent (Object pc);
void deletePersistentAll (Object[] pcs);
void deletePersistentAll (Collection pcs);
```

These methods delete persistent instances from the data store. They must be called in the context of an active transaction. The representation in the data store will be deleted when this instance is flushed to the data store (via commit, or evict).

Note that this behavior is not exactly the inverse of makePersistent, due to the transitive nature of makePersistent. The implementation might delete dependent data store objects depending on implementation-specific policy options that are not covered by the JDO specification.

These methods have no effect on instances already deleted in the transaction.

If deleting an instance would violate datastore integrity constraints, it is implementation-defined whether an exception is thrown at commit time, or the delete operation is simply ignored. Portable applications should use this method to delete instances from the datastore, and not depend on any reachability algorithm to automatically delete instances.

These methods will throw a JDOUserException if the instance is managed by a different PersistenceManager.These methods will throw a JDOUserException if the instance is transient.

Make instances transient

```
void makeTransient (Object pc);
void makeTransientAll (Object[] pcs);
```

```
void makeTransientAll (Collection pcs);
```

These methods make persistent instances transient, so they are not associated with the `PersistenceManager` instance. They do not affect the persistent state in the data store. They can be used as part of a sequence of operations to move a persistent instance to another `PersistenceManager`. The instance transitions to transient, and it loses its JDO identity. If the instance has state (persistent-nontransactional or persistent-clean) the state in the cache is preserved unchanged. If the instance is dirty, a `JDOUserException` is thrown.

These methods will be ignored if the instance is transient.

Make instances transactional

```
void makeTransactional (Object pc);
void makeTransactionalAll (Object[] pcs);
void makeTransactionalAll (Collection pcs);
```

These methods make transient instances transactional and cause a state transition to transient-clean. They must be called in the context of an active transaction. Subsequently, the instance observes transaction boundaries. If the transaction in which this instance is made transactional commits, then the transient instance retains its values. If the transaction is rolled back, then the transient instance takes its values as of the call to `makeTransactional` if the call was made within an active transaction; or the beginning of the transaction, if the call was made prior to the beginning of the active transaction.

These methods are also used to mark a nontransactional persistent instance as being part of the read-consistency set of the transaction.

Make instances nontransactional

```
void makeNontransactional (Object pc);
void makeNontransactionalAll (Object[] pcs);
void makeNontransactionalAll (Collection pcs);
```

These methods make transient-clean instances nontransactional and cause a state transition to transient. They must be called in the context of an active transaction, or a `JDOUserException` is thrown. Subsequently, the instance does not observe transaction boundaries.

These methods make persistent-clean instances nontransactional and cause a state transition to persistent-nontransactional.

If this method is called with a dirty parameter instance, a `JDOUserException` is thrown.

12.6 Transaction completion

Transaction completion management is delegated to the associated `Transaction` instance .

12.7 Multithreaded Synchronization

The application might require the `PersistenceManager` to synchronize internally to avoid corruption of data structures due to multiple application threads. This synchronization is not required when the property `Multithreaded` is set to `false`.

```
void setMultithreaded (boolean flag);
```

```
boolean getMultithreaded();
```

NOTE: When the `Multithreaded` flag is set to `true`, there is a synchronization issue with `jdoFlags` values `READ_OK` and `READ_WRITE_OK`. Due to out-of-order memory writes, there is a chance that a value for a field in the default fetch group might be incorrect (stale) when accessed by a thread that has not synchronized with the thread that set the `jdoFlags` value. Therefore, it is recommended that a JDO implementation not use `READ_OK` or `READ_WRITE_OK` for `jdoFlags` if `Multithreaded` is set to `true`.

The application may choose to perform its own synchronization, and indicate this to the implementation by setting the `Multithreaded` flag to `false`. In this case, the JDO implementation is not required to implement any additional synchronizations, although it is permitted to do so.

12.8 User associated object

The application might manage `PersistenceManager` instances by using an associated object for bookkeeping purposes. These methods allow the user to manage the associated object.

```
void setUserObject (Object o);
```

```
Object getUserObject ();
```

The parameter is not inspected or used in any way by the JDO implementation.

12.9 PersistenceManagerFactory

The application might need to get the `PersistenceManagerFactory` that created this `PersistenceManager`. If the `PersistenceManager` was created using a constructor, then this call returns `null`.

```
PersistenceManagerFactory getPersistenceManagerFactory();
```

12.10 ObjectId class management

In order for the application to construct instances of the `ObjectId` class, there is a method that returns the `ObjectId` class given the persistence capable class.

```
Class getObjectIdClass (Class cls);
```

This method returns the class of the object id for the given class. This method is primarily useful for persistence capable classes that use application (primary key) JDO identity.

13 Transactions and Connections

This chapter describes the interactions among JDO instances, JDO Persistence Managers, data store transactions, and data store connections.

13.1 Overview

Operations on persistent JDO instances at the user's choice might be performed in the context of a transaction. That is, the view of data in the data store is transactionally consistent, according to the standard definition of ACID transactions:

- atomic --within a transaction, changes to values in JDO instances are all executed or none is executed
- consistent -- changes to values in JDO instances are consistent with changes to other values in the same JDO instance
- isolated -- changes to values in JDO instances are isolated from changes to the same JDO instances in different transactions
- durable -- changes to values in JDO instances survive the end of the VM in which the changes were made

13.2 Goals

The JDO transaction and connection contracts have the following goals.

- JDO implementations might span a range of small, embedded systems to large, enterprise systems
- Transaction management might be entirely hidden from class developers and application components, or might be explicitly exposed to class and application component developers.

13.3 Architecture: PersistenceManager, Transactions, and Connections

An instance of an object supporting the `PersistenceManager` interface represents a single user's view of persistent data, including cached persistent instances across multiple serial data store transactions.

There is a one-to-one relationship between the `PersistenceManager` and the `Transaction`. The `Transaction` interface is isolated because of separation of concerns. The methods could have been added to the `PersistenceManager` interface.

The `Transaction` interface provides for management of transaction options and, in the non-managed environment, for transaction completion.

Connection Management Scenarios

- single connection: In the simplest case, the PersistenceManager directly connects to the data store and manages transactional data. In this case, there is no reason to expose any Connection properties other than those needed to identify the user and the data source. During transaction processing, the Connection will be used to satisfy data read, write, and transaction completion requests from the `PersistenceManager`.
- connection pooling: In a slightly more complex situation, the `PersistenceManagerFactory` creates multiple `PersistenceManager` instances which use connection pooling to reduce resource consumption. The `PersistenceManager`s are used in single data store transactions. In this case, a pooling connection manager is a separate component used by the `PersistenceManager` instances to effect the pooling of connections. The `PersistenceManagerFactory` will include a reference to the connection pooling component, either as a JNDI name or as an object reference. The connection pooling component is separately configured, and the `PersistenceManagerFactory` simply needs to be configured to use it.
- distributed transactions: An even more complex case is where the `PersistenceManager` instances need to use connections that are involved in distributed transactions. This case requires coordination with a Transaction Manager, and exposure of the `XAResource` from the data store Connection. JDO does not specify how the application coordinates transactions among the `PersistenceManager` and the Transaction Manager.
- managed connections: The last case to consider is the managed environment, where the PersistenceManagerFactory uses a data store Connection whose transaction completion is managed by the application server. This case requires the data store Connection to implement the J2EE Connector Architecture and the PersistenceManager to use the architected interfaces to obtain a reference to a Connection.

The interface between the JDO implementation and the Connection component is not specified by JDO. In the non-managed environment, transaction completion is handled by the Connection managed internally by the Transaction. In the managed environment, transaction completion is handled by the `XAResource` associated with the Connection. In both cases, the `PersistenceManager` implementation is responsible for setting up the appropriate interface to the Connection infrastructure.

Native Connection Management

If the JDO implementation supplies its own Connection interface, this is termed native connection management. For use in a managed environment, the association between `Transaction` and Connection must be established using the J2EE Connection Architecture [see Appendix A reference 4]. This is done by the JDO implementation supplying the `javax.resource.ManagedConnectionFactory` and `javax.resource.ManagedConnection` interfaces.

When used in a non-managed environment, with non-distributed transaction management (local transactions) the application can use the `PersistenceManagerFactory`. But if distributed transaction management is required, the application needs to supply an implementation of `javax.resource.ConnectionManager` interface. This interface provides the infrastructure to enlist the `XAResource` with the Transaction Manager used in the application.

Non-native Connection Management

If the JDO implementation uses a third party Connection interface, then it can be used in a managed environment only if the third party Connection supports the J2EE Connector Architecture. In this case, the `PersistenceManagerFactory` property `ConnectionFactory` is used to allow the application server to manage connections.

In the non-managed case, non-distributed transaction management can use the `PersistenceManagerFactory`, as above. But if distributed transaction management is required, the application needs to supply an implementation of `javax.resource.ConnectionManager` interface to be used with the application's implementation of the Connection management.

Optimistic Transactions

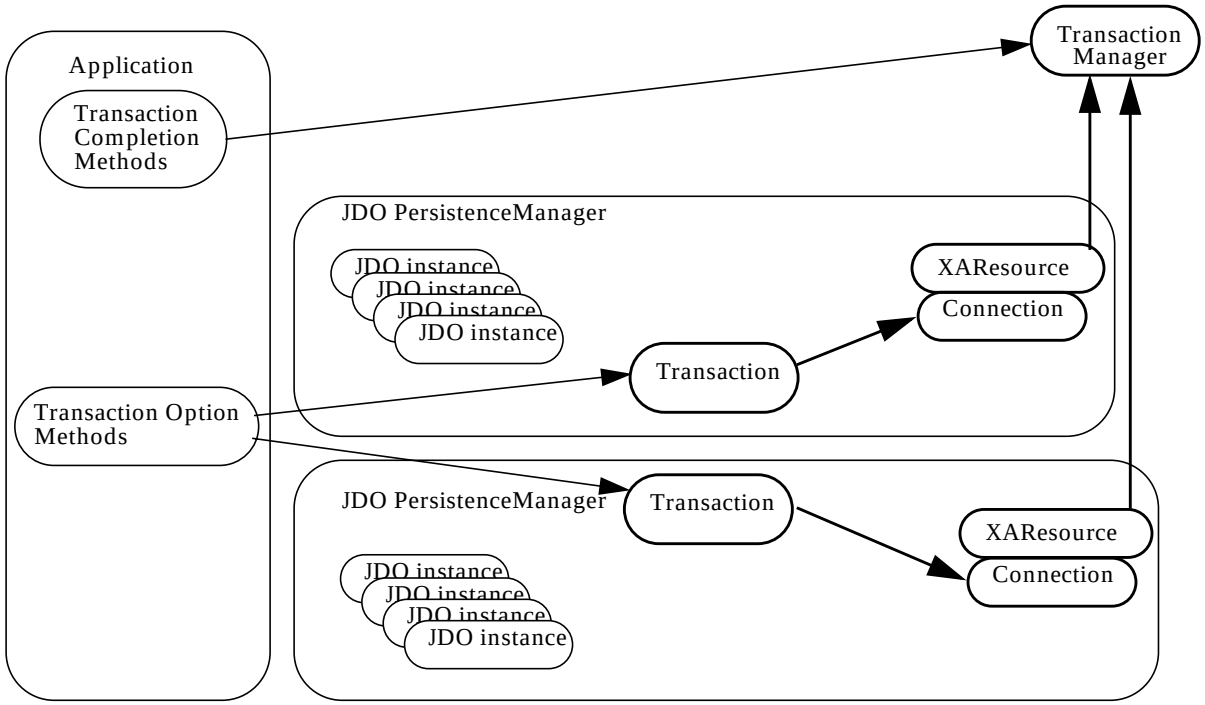
There are two types of transaction management strategies supported by JDO: "data store transaction management"; and "optimistic transaction management".

With data store transaction management, all operations performed by the application on persistent data are done using a data store transaction. This means that between the first data access until the commit, there is an active data store transaction.

With optimistic transaction management, operations performed by the application on persistent data outside of a transaction or before commit are done using a short local data store transaction. During flush, a data store transaction is used for the update operations, verifying that the proposed changes do not conflict with a parallel update by a different transaction.

Optimistic transaction management is specified by the `Optimistic` setting on `Transaction`.

Figure 15.0 Transactions and Connections



13.4 Interface Transaction

The JDO `Transaction` interface has the following methods.

13.4.1 PersistenceManager

```
PersistenceManager getPersistenceManager ();
```

This method returns the `PersistenceManager` associated with this `Transaction` instance.

```
boolean isActive ();
```

This method tells whether there is an active transaction. The transaction might be either a local transaction or a distributed transaction. If the transaction is local, then the `begin` method was executed and neither `commit` nor `rollback` has been executed. If the transaction is managed by `XAResource` with a `TransactionManager`, then this method indicates whether there is a distributed transaction active.

13.4.2 Transaction options

Transaction options are valid for both managed and non-managed environments. Flags are durable until changed explicitly by `set` methods. They are not changed by transaction demarcation methods.

If an implementation does not support the option, then an attempt to set the flag to an unsupported value will throw `JDOUnsupportedOptionException`.

Nontransactional access to persistent values

```
boolean getNontransactionalRead ();
```

```
void setNontransactionalRead (boolean flag);
```

These methods access the flag that allows persistent instances to be read outside of a transaction. If this flag is set to `true`, then queries and navigation are allowed without an active transaction. If this flag is set to `false`, then queries and navigation outside an active transaction throw a `JDOUserException`.

```
boolean getNontransactionalWrite ();
```

```
void setNontransactionalWrite (boolean flag);
```

These methods access the flag that allows non-transactional instances to be written in the cache. If this flag is set to `true`, then updates to non-transactional instances are allowed without an active transaction. If this flag is set to `false`, then updates to non-transactional instances outside an active transaction throw a `JDOUserException`.

Optimistic concurrency control

If this flag is set to `true`, then optimistic concurrency is used for managing transactions.

```
boolean getOptimistic ();
```

The optimistic setting currently active is returned.

```
void setOptimistic (boolean flag);
```

The optimistic setting passed replaces the optimistic setting currently active.

This method can only be used when there is not an active transaction. If it is used while there is an active transaction, a `JDOUserException` is thrown.

Retain values after transaction completion

If this flag is set to `true`, then eviction of persistent instances does not take place after transaction completion.

```
boolean getRetainValues ();
```

The retainValues setting currently active is returned.

```
void setRetainValues (boolean flag);
```

The `retainValues` setting passed replaces the `retainValues` setting currently active. If set to `true`, then `NontransactionalRead` is set to `true`.

13.4.3 Synchronization

The `Transaction` instance participates in synchronization in two ways: as a supplier of synchronization callbacks, and as a consumer of callbacks. As a supplier of callbacks, a user can register with the `Transaction` instance to be notified at transaction completion. As a consumer of callbacks, the user can explicitly notify the `Transaction` instance of externally-initiated transaction completion events. This is the technique used in a managed environment to cause flushing of changes to the data store prior to transaction completion.

For this latter purpose, `Transaction` implements `javax.transaction.Synchronization`.

Synchronization is supported for both managed and non-managed environments. A `Synchronization` instance registered with the `Transaction` remains registered until changed explicitly by another `setSynchronization`.

Only one `Synchronization` instance can be registered with the `Transaction`. If the application requires more than one instance to receive synchronization callbacks, then the application instance is responsible for managing them, and forwarding callbacks to them.

```
void setSynchronization (javax.transaction.Synchronization sync);
```

```
javax.transaction.Synchronization getSynchronization ();
```

The `Synchronization` instance is registered with the `Transaction` for transaction completion notifications. Any `Synchronization` instance already registered will be replaced. If the parameter is `null`, then no instance will be notified.

The `beforeCompletion` method will be called **before** the behavior specified for the transaction completion method `commit`. The `beforeCompletion` method will **not** be called before `rollback`.

The `afterCompletion` method will be called **after** the transaction completion methods are finished. The parameter for the `afterCompletion (int status)` method will be either `Status.STATUS_COMMITTED` or `Status.STATUS_ROLLEDBACK`.

These two methods allow the application control over the environment in which the transaction completion executes (for example, validate the state of the cache before completion) and to control the cache disposition once the transaction completes (for example, to change persistent instances to persistent-nontransactional state).

13.4.4 Transaction demarcation

If multiple parallel transactions are required, then multiple `PersistenceManager` instances must be used. If distributed transactions are required, then the Connector Architecture is used to coordinate transactions among the JDO `PersistenceManager`s.

Non-managed environment

In a non-managed environment, with a single JDO `PersistenceManager` per application, there is a `Transaction` instance representing a local transaction associated with the `PersistenceManager` instance.

```
void begin();
void commit();
void rollback();
```

The `commit` and `rollback` methods can only be used in a non-managed environment, or in a managed environment with Bean Managed Transactions. If one of these methods is executed in a managed environment with Container Managed Transactions, a `JDOUserException` is thrown.

The `commit` method performs the following operations:

- calls the `beforeCompletion` method of the `Synchronization` instance registered with the `Transaction`;
- flushes dirty persistent instances;
- notifies the underlying data store to commit the transaction;
- transitions persistent instances according to the life cycle specification;
- evicts persistent instances from the cache (subject to the `retainValues` flag);
- calls the `afterCompletion` method of the `Synchronization` instance registered with the `Transaction` with the results of the data store commit operation.

The `rollback` method performs the following operations:

- transitions persistent instances according to the life cycle specification;
- rolls back changes made in this transaction from the data store;
- evicts persistent instances from the cache (subject to the `retainValues` flag);
- calls the `afterCompletion` method of the `Synchronization` instance registered with the `Transaction`.

Managed environment

In a managed environment, there is either a user transaction or a local transaction associated with the `PersistenceManager` instance when executing method calls on JDO instances or on the `PersistenceManager`. Which of the two types of transactions is active is a policy issue for the managed environment.

If data store transaction management is being used with the `PersistenceManager` instance, and a Connection to the data store is required during execution of the `PersistenceManager` or JDO instance method, then the `PersistenceManager` will dynamically acquire a Connection. The call to acquire the Connection will be made with the calling thread in the appropriate transactional context, and the Connection acquired will be in the proper data store transaction.

If optimistic transaction management is being used with the `PersistenceManager` instance, and a Connection to the data store is required during execution of an instance method or a non-completion `PersistenceManager` method, then the `PersistenceManager` will use a local transaction Connection.

13.5 Optimistic transaction management

Optimistic transactions are an optional feature of a JDO implementation. They are useful when there are long-running transactions that rarely affect the same instances, and therefore the data store will exhibit better performance by deferring data store exclusion on modified instances until commit.

With datastore transactions, persistent instances accessed within the scope of an active transaction are guaranteed to be enlisted in the transaction. With optimistic transactions, persistent instances accessed within the scope of an active transaction are not enlisted in the transaction. The only time any instances are enlisted in the current transaction is during commit.

With optimistic transactions, instances queried or read from the data store will not be transactional unless they are modified, deleted, or marked by the application as transactional in the transaction.

At commit time, instances that have been made transactional will be verified against the current contents of the data store, to ensure that the state in the data store is the same as the "before image" of the instance in the transaction.

If any instance is found to have changed, a `JDOUserException` is thrown which contains an array of `JDOUserException`, one for each instance that failed the verification. The optimistic transaction is failed.

Details of the state transitions of persistent instances in optimistic transactions may be found in section .

14 Query

This chapter specifies the query contract between an application component and the JDO `PersistenceManager`.

The query facility consists of two parts: the query API, and the query language. The query language described in this chapter is “JDOQL”.

14.1 Overview

An application component requires access to JDO instances in order to invoke specific behavior on those instances. From a JDO instance, it might navigate to other associated instances, thereby operating on an application-specific closure of instances.

However, getting to the first JDO instance is a bootstrap issue. There are three ways to get an instance from JDO. First, if the users have or can construct a valid `ObjectId`, then they can get an instance via the persistence manager’s `getObjectById` method. Second, users can iterate a class extent by calling `getExtent`. Third, the JDO `Query` interface provides the ability to acquire access to JDO instances from a particular JDO persistence manager based on search criteria specified by the application.

The persistent manager instance is a factory for query instances, and queries are executed in the context of the persistent manager instance.

The actual query execution might be performed by the JDO `PersistenceManager` or might be delegated by the JDO `PersistenceManager` to its data store. The actual query executed thus might be implemented in a very different language from Java, and might be optimized to take advantage of particular query language implementations.

For this reason, methods in the query filter have semantics possibly different from those in the Java VM.

14.2 Goals

The JDO `Query` interface has the following goals:

- Query language neutrality. The underlying query language might be a relational query language such as SQL; an object database query language such as OQL; or a specialized API to a hierarchical database or mainframe EIS system.
- Optimization to specific query language. The `Query` interface must be capable of optimizations; therefore, the interface must have enough user-specified information to allow for the JDO implementation to exploit data source specific query features.
- Accommodation of multi-tier architectures. Queries might be executed entirely in memory, or might be delegated to a back end query engine. The JDO `Query` interface must provide for both types of query execution strategies.

- Large result set support. Queries might return massive numbers of JDO instances that match the query. The JDO `Query` architecture must provide for processing the results within the resource constraints of the execution environment.
- Compiled query support. Parsing queries may be resource-intensive, and in many applications can be done during application development or deployment, prior to execution time. The query interface allows for compiling queries and binding run-time parameters to the bound queries for execution.

14.3 Architecture: Query

The JDO `PersistenceManager` instance is a factory for JDO `Query` instances which implement the JDO `Query` interface. Multiple JDO `Query` instances might be active simultaneously in the same JDO `PersistenceManager` instance. Multiple queries might be executed simultaneously by different threads, but the implementation might choose to execute them serially. In either case, the execution must be thread safe.

There are three required elements in any query:

- the class of the candidate instances. The class is used to scope the names in the query filter. All of the candidate instances will be of this class or subclass.
- the collection of candidate JDO instances. The collection of candidate instances is either a `java.util.Collection`, or an `Extent` of instances in the data store. Candidate instances that are not of the required class or subclass will be silently ignored. The `Collection` might be a previous query result, allowing for subqueries.
- the query filter. The query filter is a Java `boolean` expression, which tells whether instances in the candidate collection are to be returned in the result.

Other elements in queries include:

- parameter declarations. The parameter declaration is a `String` containing one or more query parameter declarations separated with commas. It follows the syntax for formal parameters in the Java language. Each parameter named in the parameter declaration must be bound to a value when the query is executed.
- parameter values to bind to parameters. Values are specified as Java `Objects`, and might include simple wrapper types or more complex object types. The values are passed to the execute methods and are not preserved after a query executes.
- unbound variable declarations: Variables might be used in the filter, and these variables must be declared with their type. The unbound variable declaration is a `String` containing one or more unbound variable declarations separated with semicolons. It follows the syntax for local variables in the Java language.
- import statements: Parameters and unbound variables might come from a different class from the candidate class, and the names might need to be declared in an import statement to eliminate ambiguity. Import statements are specified as a `String` with semicolon-separated statements. The syntax is the same as in the Java language import statement.
- ordering specification. The ordering specification includes a list of expressions with the ascending / descending indicator. The expression's type must be one of:
  - primitive types except `boolean`;

- wrapper types except `Boolean`;
- `BigDecimal`;
- `BigInteger`;
- `String`;
- `Date`.

The class implementing the `Query` interface must be serializable. The serialized fields include the candidate class, the filter, parameter declarations, variable declarations, imports, and ordering specification. If a serialized instance is restored, it loses its association with its former `PersistenceManager`.

14.4 Namespaces in queries

The query namespace is modeled after methods in Java:

- `setClass` corresponds to the class definition
- `declareParameters` corresponds to formal parameters of a method
- `declareVariables` corresponds to local variables of a method
- `setFilter` and `setOrdering` correspond to the method body

There are two namespaces in queries. Type names have their own namespace that is separate from the namespace for fields, variables and parameters.

The method `setClass` introduces the name of the candidate class in the type namespace. The method `declareImports` introduces the names of the imported class or interface types in the type namespace. Imported type names must be unique. When used (e.g. in a parameter declaration, cast expression, etc.) a type name must be the name of the candidate class, the name of a class or interface imported by method `declareImports`, or denote a class or interface from the same package as the candidate class.

The method `setClass` introduces the names of the candidate class fields.

The method `declareParameters` introduces the names of the parameters. A name introduced by `declareParameters` hides the name of a candidate class field if equal. Parameter names must be unique.

The method `declareVariables` introduces the names of the variables. A name introduced by `declareVariables` hides the name of a candidate class field if equal. Variable names must be unique and must not conflict with parameter names.

A hidden field may be accessed using the '`this`' qualifier: `this.fieldName`.

14.5 Query Factory in PersistenceManager interface

The `PersistenceManager` interface contains `Query` factory methods.

```
Query newQuery();
```

Construct an empty query instance.

```
Query newQuery (Object query);
```

Construct a query instance from another query. The parameter might be a serialized / restored `Query` instance from the same JDO vendor but a different execution environment, or the parameter might be currently bound to a `PersistenceManager` from the same JDO vendor. Any of the elements Class, Filter, Import declarations, Variable declarations, Parameter declarations, and Ordering from the parameter `Query` are copied to the new `Query` instance, but a candidate `Collection` or `Extent` element is discarded.

```
Query newQuery (String language, Object query);
```

Construct a query instance using the specified language and the specified query. The query instance will be of a class defined by the query language. The language parameter for the JDO Query language as herein documented is "javax.jdo.query.JDOQL". Other languages' parameter is not specified.

```
Query newQuery (Class cls);
```

Construct a query instance with the candidate class specified.

```
Query newQuery (Class cls, Extent cln);
```

Construct a query instance with the candidate class and candidate `Extent` specified.

```
Query newQuery (Class cls, Collection cln);
```

Construct a query instance with the candidate class and candidate `Collection` specified.

```
Query newQuery (Class cls, String filter);
```

Construct a query instance with the candidate class and filter specified.

```
Query newQuery (Class cls, Collection cln, String filter);
```

Construct a query instance with the candidate class, the candidate `Collection`, and filter specified.

```
Query newQuery (Class cls, Extent cln, String filter);
```

Construct a query instance with the candidate class, the candidate `Extent`, and filter specified.

14.6 Query Interface

```
interface Query extends Serializable
```

Persistence Manager

```
PersistenceManager getPersistenceManager();
```

Return the associated `PersistenceManager` instance. If this `Query` instance was restored from a serialized form, then `null` is returned.

Query element binding

The `Query` interface provides methods to bind required and other elements prior to execution.

All of these methods replace the previously set query element, by the parameter. [The methods are not additive.] For example, if multiple variables are needed in the query, all of them must be specified in the same call to `declareVariables`.

```
void setClass (Class candidateClass);
```

Bind the candidate class to the query instance.

```
void setCandidates (Collection candidateCollection);
```

Bind the candidate `Collection` to the query instance. If the user adds or removes elements from the `Collection` after this call, it is not determined whether the added/removed elements take part in the `Query`, or whether a `NoSuchElementException` is thrown during execution of the `Query`.

```
void setCandidates (Extent candidateExtent);
```

Bind the candidate `Extent` to the query instance.

```
void setFilter (String filter);
```

Bind the query filter to the query instance.

```
void declareImports (String imports);
```

Bind the import statements to the query instance. All imports must be declared in the same method call, and the imports must be separated by semicolons.

```
void declareVariables (String variables);
```

Bind the unbound variable statements to the query instance. This method defines the types and names of variables that will be used in the filter but not provided as values by the `execute` method.

```
void declareParameters (String parameters);
```

Bind the parameter statements to the query instance. This method defines the parameter types and names which will be used by a subsequent `execute` method.

```
void setOrdering (String ordering);
```

Bind the ordering statements to the query instance.

Query options

```
void setIgnoreCache (boolean flag);
```

```
boolean getIgnoreCache ();
```

The `IgnoreCache` option is a hint to the query engine that the user expects queries be optimized to return approximate results by ignoring changed values in the cache. This option is only useful for optimistic transactions and allows the data store to return results that do not take modified cached instances into account. An implementation may choose to ignore the setting of this flag, and always return exact results reflecting current cached values.

Query compilation

The `Query` interface provides a method to compile queries for subsequent execution.

```
void compile();
```

This method requires the `Query` instance to validate any elements bound to the query instance and report any inconsistencies by throwing a `JDOUserException`. It is a hint to the `Query` instance to prepare and optimize an execution plan for the query.

14.6.1 Query execution

The `Query` interface provides methods which execute the query based on the parameters given. They return an unmodifiable `Collection` which the user can iterate to get results. Executing any operation on the `Collection` that might change it throws `UnsupportedOperationException`. For future extension, the signature of the `execute` methods specifies that they return an `Object` which must be cast to `Collection` by the user.

Any parameters passed to the `execute` methods are used only for this execution, and are not remembered for future execution.

```
Object execute ();
```

```
Object execute (Object p1);
```

```
Object execute (Object p1, Object p2);
```

```
Object execute (Object p1, Object p2, Object p3);
```

The `execute` methods execute the query using the parameters and return the result, which is an unmodifiable collection of instances that satisfy the boolean filter. The result may be a large `Collection`, which should be iterated, or possibly passed to another `Que-`

ry. The `size()` method might return `Integer.MAX_VALUE` if the actual size of the result is not known (for example, the `Collection` represents a cursored result).

Each parameter of the `execute` method(s) is an `Object` which is either the value of the corresponding parameter or the wrapped value of a primitive parameter. The parameters associate in order with the parameter declarations in the `Query` instance.

```
Object executeWithMap (Map m);
```

The `executeWithMap` method is similar to the `execute` method, but takes its parameters from a `Map` instance. The `Map` contains key / value pairs, in which the key is the declared parameter name, and the value is the value to use in the query for that parameter. Unlike `execute`, there is no limit on the number of parameters.

```
Object executeWithArray (Object[] a);
```

The `executeWithArray` method is similar to the `execute` method, but takes its parameters from an array instance. The array contains `Objects`, in which the positional `Object` is the value to use in the query for that parameter. Unlike `execute`, there is no limit on the number of parameters.

14.6.2 Filter specification

The filter specification is a `String` containing a boolean expression that is to be evaluated for each of the instances in the candidate collection. If the filter is not specified, then it defaults to "`true`", which has the effect of filtering the input `Collection` only for class type.

An element of the candidate collection is returned in the result if:

- it is assignment compatible to the candidate `Class` of the `Query`; and
- for all variables there exists a value for which the filter expression evaluates to `true`. The user may denote uniqueness in the filter expression by explicitly declaring an expression (for example, `e1 != e2`).

Rules for constructing valid expressions follow the Java language, except for these differences:

- Equality and ordering comparisons between primitives and instances of wrapper classes are valid.
- Equality and ordering comparisons of `Date` fields and `Date` parameters are valid.
- White space (non-printing characters space, tab, carriage return, and line feed) is a separator and is otherwise ignored.
- The assignment operators =, +=, etc. and pre- and post-increment and -decrement are not supported. Therefore, there are no side effects from evaluation of any expressions.
- Methods, including object construction, are not supported, except for `Collection.contains(Object o)`, `Collection.isEmpty()`, `String.startsWith (String s)`, and `String.endsWith (String e)`. Implementations might choose to support non-mutating method calls as non-standard extensions.
- Navigation through a null-valued field, which would throw `NullPointerException`, is treated as if the filter expression returned `false` for the evaluation of the current set of variable values. Other values for variables might still qualify the candidate instance for inclusion in the result set.

- Navigation through multi-valued fields (`Collection` types) is specified using a variable declaration and the `Collection.contains (Object o)` method.

Identifiers in the expression are considered to be in the name space of the specified class, with the addition of declared imports, parameters and variables. As in the Java language, `this` is a reserved word which means the element of the collection being evaluated.

Navigation through single-valued fields is specified by the Java language syntax of `field_name.field_name….field_name`.

A JDO implementation is allowed to reorder the filter expression for optimization purposes.

The following are minimum capabilities of the expressions that every implementation must support:

- operators applied to all types where they are defined in the Java language:

Table 4: Query Operators

Operator	Description
==	equal
!=	not equal
>	greater than
<	less than
>=	greater than or equal
<=	less than or equal
&	boolean logical AND (not bitwise)
&&	conditional AND
|	boolean logical OR (not bitwise)
||	conditional OR
~	integral unary bitwise complement
+	binary or unary addition or String concatenation
-	binary subtraction or numeric sign inversion
*	times
/	divide by
!	logical complement

- parentheses to explicitly mark operator precedence

- cast operator (class)
- promotion of numeric operands for comparisons.
- equality and ordering comparison and arithmetic operations on object-valued fields of wrapper types (`Boolean, Byte, Short, Integer, Long, Float, and Double`), and numeric types (`BigDecimal and BigInteger`) use the wrapped values as comparands or operands.
- equality comparison of object-valued fields of `PersistenceCapable` types use the JDO Identity comparison of the references. Thus, two objects will compare equal if they have the same JDO Identity.
- equality comparison of object-valued fields of non-`PersistenceCapable` types uses the `equals` method of the field type.
- `String` methods `startsWith` and `endsWith` support wild card queries. JDO does not define any special semantic to the argument passed to the method; in particular, it does not define any wild card characters.
- `Null`-valued fields of `Collection` types are treated as if they were empty if a method is called on them. In particular, they return `true` to `isEmpty` and return `false` to all `contains` methods. For datastores that support `null` values for `Collection` types, it is valid to compare the field to `null`. Datastores that do not support `null` values for `Collection` types, will return `false` if the query compares the field to `null`.

14.6.3 Parameter declaration

The parameter declaration is a `String` containing one or more parameter type declarations separated by commas. This follows the Java syntax for method signatures.

14.6.4 Import statements

The import statements follow the Java syntax for import statements.

14.6.5 Variable declaration

The type declarations follow the Java syntax for local variable declarations.

If the variable is not named in a contains clause, that variable's scope while evaluating the filter expression is the `Extent` (including subclasses) of the class of the variable. If the class does not manage an `Extent`, then no results will satisfy the query.

A portable query will constrain all variables with a `contains` clause in each "OR" expression of the filter where the variable is used. Further, the `contains` clause must be the left expression of an "AND" expression where the variable is used in the right expression. That is, for each occurrence of an expression in the filter using the variable, there is a `contains` clause "ANDed" with the expression that constrains the possible values by the elements of a collection.

A variable that is not constrained with an explicit `contains` clause is constrained by the extent of the persistence capable class in the database.

14.6.6 Ordering statement

The ordering statement is a `String` containing one or more ordering declarations separated by commas. Each ordering declaration is a Java expression of an orderable type:

- primitives except `boolean`;
- wrappers except `Boolean`;

- `BigDecimal`;
- `BigInteger`;
- `String`;
- `Date`

followed by one of the following words: "`ascending`" or "`descending`".

Ordering might be specified including navigation. The name of the field to be used in ordering via navigation through single-valued fields is specified by the Java language syntax of `field_name.field_name….field_name`.

14.6.7 Closing Query results

When the application has finished with the query results, it might optionally close the results, allowing the JDO implementation to release resources that might be engaged, such as database cursors or iterators. The following methods allow early release of these resources.

```
void close (Object queryResult);
```

This method closes the result of one `execute(...)` method, and releases resources associated with it. After this method completes, the query result can no longer be used, for example to iterate the returned elements. Any elements returned previously by iteration of the results remain in their current state. Any iterators acquired from the queryResult will return `false` to `hasNext()` and will throw `NoSuchElementException` to `next()`.

```
void closeAll ();
```

This method closes all results of `execute(...)` methods on this `Query` instance, as above. The `Query` instance is still valid and can still be used.

14.7 Examples:

The following class definitions for persistence capable classes are used in the examples:

```
package com.xyz.hr;
class Employee {
String name;
Float salary;
Department dept;
Employee boss;
}
package com.xyz.hr;
class Department {
String name;
Collection emps;
}
```

14.7.1 Basic query.

This query selects all Employee instances from the candidate collection where the salary is greater than the constant 30000.

Note that the `float` value for `salary` is unwrapped for the comparison with the literal `int` value which is promoted to `float` using numeric promotion. If the value for the `salary` field in a candidate instance is `null`, then it cannot be unwrapped for the comparison, and the candidate instance is rejected.

```
Class empClass = Employee.class;
Extent clnEmployee = pm.getExtent (empClass, false);
String filter = “salary > 30000”;
Query q = pm.newQuery (empClass, clnEmployee, filter);
Collection emps = q.execute ();
```

14.7.2 Basic query with ordering.

This query selects all Employee instances from the candidate collection where the salary is greater than the constant 30000, and returns a `Collection` ordered based on employee salary.

```
Class empClass = Employee.class;
Extent clnEmployee = pm.getExtent (empClass, false);
String filter = “salary > 30000”;
Query q = pm.newQuery (empClass, clnEmployee, filter);
q.setOrdering (“salary ascending”);
Collection emps = q.execute ();
```

14.7.3 Parameter passing.

This query selects all Employee instances from the candidate collection where the salary is greater than the value passed as a parameter.

If the value for the `salary` field in a candidate instance is `null`, then it cannot be unwrapped for the comparison, and the candidate instance is rejected.

```
Class empClass = Employee.class;
Extent clnEmployee = pm.getExtent (empClass, false);
String filter = “salary > sal”;
Query q = pm.newQuery (empClass, clnEmployee, filter);
String param = “Float sal”;
q.declareParameters (param);
Collection emps = (Collection) q.execute (new Float (30000.));
```

14.7.4 Navigation through single-valued field.

This query selects all Employee instances from the candidate collection where the value of the name field in the Department instance associated with the Employee instance is equal to the value passed as a parameter.

If the value for the `dept` field in a candidate instance is `null`, then it cannot be navigated for the comparison, and the candidate instance is rejected.

```
Class empClass = Employee.class;
Extent clnEmployee = pm.getExtent (empClass, false);
String filter = “dept.name == dep”;
String param = “String dep”;
```

```
Query q = pm.newQuery (empClass, clnEmployee, filter);
q.declareParameters (param);
String rnd = “R&D”;
Collection emps = (Collection) q.execute (rnd);
```

14.7.5 Navigation through multi-valued field.

This query selects all Department instances from the candidate collection where the collection of Employee instances contains at least one Employee instance having a salary greater than the value passed as a parameter.

```
Class depClass = Department.class;
Extent clnDepartment = pm.getExtent (depClass, false);
String vars = “Employee emp”;
String filter = “emps.contains (emp) & emp.salary > sal”;
String param = “float sal”;
Query q = pm.newQuery (depClass, clnDepartment, filter);
q.declareParameters (param);
q.declareVariables (vars);
Collection deps = (Collection) q.execute (new Float (30000.));
```

15 Extent

This chapter specifies the `Extent` contract between an application component and the JDO implementation.

15.1 Overview

An application needs to provide a candidate collection of instances to a query. If the query filtering is to be performed in the datastore, then the application must supply the collection of instances to be filtered. This is the primary function of the `Extent` interface.

A query may be executed against either a `Collection` or an `Extent`. The `Extent` is used when the query is intended to be filtered by the data store, not by in-memory processing. There are no `Collection` methods in `Extent` except for `iterator()`. Thus, common `Collection` behaviors are not possible, including determining whether one `Extent` contains another, determining the size of the `Extent`, or determining whether a specific instance is contained in the `Extent`. Any such operations must be performed by executing a query against the `Extent`.

If the `Extent` is large, then an appropriate iteration strategy should be adopted by the JDO implementation.

15.2 Goals

The extent interface has the following goals:

- Large result set support. Queries might return massive numbers of JDO instances that match the query. The JDO `Query` architecture must provide for processing the results within the resource constraints of the execution environment.
- Application resource management. Iterating an Extent might use resources that should be released when the application has finished an iteration. The application should be provided with a means to release iterator resources.

15.3 Interface

```
public interface Extent
```

```
Iterator iterator();
```

This method returns an `Iterator` over all the instances in the `Extent`. If instances were made persistent in the transaction prior to the execution of this method, the returned `Iterator` will contain the instances. If instances were deleted in the transaction prior to the execution of this method, the returned `Iterator` will not contain the instances.

If any mutating method, including the `remove` method, is called on the `Iterator` returned by this method, a `UnsupportedOperationException` is thrown.

```
boolean hasSubclasses();
```

This method returns an indicator of whether the extent is proper or includes subclasses.

```
Class getCandidateClass();
```

This method returns the class of the instances contained in it.

```
PersistenceManager getPersistenceManager();
```

This method returns the `PersistenceManager` which created it.

```
void close (Iterator i);
```

This method closes an iterator acquired from this `Extent`. After this call, the parameter iterator will return `false` to `hasNext()`, and will throw `NoSuchElementException` to `next()`. The `Extent` itself can still be used to acquire other iterators and can be used as the `Extent` for queries.

```
void closeAll ();
```

This method closes all iterators acquired from this `Extent`. After this call, all iterators acquired from this `Extent` will return `false` to `hasNext()`, and will throw `NoSuchElementException` to `next()`.

16 Enterprise Java Beans

Enterprise Java Beans (EJB) is a component architecture for development and deployment of distributed business applications. Java Data Objects is a suitable component for integration with EJB in these scenarios:

- Session Beans with JDO persistence-capable classes used to implement dependent objects;
- Entity Beans with JDO persistence-capable classes used as delegates for both Bean Managed Persistence and Container Managed Persistence.

16.1 Session Beans

A session bean should be associated with an instance of `PersistenceManagerFactory` that is established during a session life cycle event, and each business method should use an instance of `PersistenceManager` obtained from the `PersistenceManagerFactory`. The timing of when the `PersistenceManager` is obtained will vary based on the type of bean.

The bean class should contain instance variables that hold the associated `PersistenceManager` and `PersistenceManagerFactory`.

During activation of the bean, the `PersistenceManagerFactory` should be found via `JNDI` lookup. The `PersistenceManagerFactory` should be the same instance for all beans sharing the same datastore resource. This allows for the `PersistenceManagerFactory` to manage an association between the distributed transaction and the `PersistenceManager`.

When appropriate during the bean life cycle, the `PersistenceManager` should be acquired by a call to the `PersistenceManagerFactory`. The `PersistenceManagerFactory` should look up the transaction association of the caller, and return a `PersistenceManager` with the same transaction association. If there is no `PersistenceManager` currently enlisted in the caller's transaction, a new `PersistenceManager` should be created and associated with the transaction. The `PersistenceManager` should be registered for synchronization callbacks with the `TransactionManager`. This provides for transaction completion callbacks asynchronous to the bean life cycle.

The instance variables for a session bean of any type include:

- a reference to the `PersistenceManagerFactory`, which should be initialized by the method `setSessionContext`. This method looks up the `PersistenceManagerFactory` by `JNDI` access to the named object "`java:comp/env/jdo/<persistence manager factory name>`".
- a reference to the `PersistenceManager`, which should be acquired by each business method, and closed at the end of the business method; and
- a reference to the `SessionContext`, which should be initialized by the method `setSessionContext`.

16.1.1 Stateless Session Bean with Container Managed Transactions

Stateless session beans are service objects that have no state between business methods. They are created as needed by the container and are not associated with any one user. A business method invocation on a remote reference to a stateless session bean might be dispatched by the container to any of the available beans in the ready pool.

Each business method must acquire its own `PersistenceManager` instance from the `PersistenceManagerFactory`. This is done via the method `getPersistenceManager` on the `PersistenceManagerFactory` instance. This method must be implemented by the JDO vendor to find a `PersistenceManager` associated with the instance of `javax.transaction.Transaction` of the executing thread.

At the end of the business method, the `PersistenceManager` instance must be closed. This allows the transaction completion code in the `PersistenceManager` to free the instance and return it to the available pool in the `PersistenceManagerFactory`.

16.1.2 Stateful Session Bean with Container Managed Transactions

Stateful session beans are service objects that are created for a particular user, and may have state between business methods. A business method invocation on a remote reference to a stateful session bean will be dispatched to the specific instance created by the user.

The behavior of stateful session beans with container managed transactions is otherwise the same as for stateless session beans. All business methods in the remote interface must acquire a `PersistenceManager` at the beginning of the method, and close it at the end, since the transaction context is managed by the container.

16.1.3 Stateless Session Bean with Bean Managed Transactions

Bean managed transactions offer additional flexibility to the session bean developer, with additional complexity. Transaction boundaries are established by the bean developer, but the state (including the `PersistenceManager`) cannot be retained across business method boundaries. Therefore, the `PersistenceManager` must be acquired and closed by each business method.

The alternative techniques for transaction boundary demarcation are:

- `javax.transaction.UserTransaction`

If the bean developer directly uses `UserTransaction`, then the `PersistenceManager` must be acquired from the `PersistenceManagerFactory` only after establishing the correct transaction context of `UserTransaction`. During the `getPersistenceManager` method, the `PersistenceManager` will be enlisted in the `UserTransaction`. For example, if non-transactional access is required, a `PersistenceManager` must be acquired when there is no `UserTransaction` active. After beginning a `UserTransaction`, a different `PersistenceManager` must be acquired for transactional access. The user must keep track of which `PersistenceManager` is being used for which transaction.

- `javax.jdo.Transaction`

If the bean developer chooses to use the same `PersistenceManager` for multiple transactions, then transaction completion must be done entirely by using the `javax.jdo.Transaction` instance associated with the `PersistenceManager`. In this case, acquiring a `PersistenceManager` without beginning a `UserTransaction` results in the `PersistenceManager` being able to manage transaction boundaries via `be-`

gin, commit, and rollback methods on javax.jdo.Transaction. The PersistenceManager will automatically begin the UserTransaction during javax.jdo.Transaction.begin and automatically commit the UserTransaction during javax.jdo.Transaction.commit.

16.1.4 Stateful Session Bean with Bean Managed Transactions

Stateful session beans allow the bean developer to manage the transaction context as part of the conversational state of the bean. Thus, it is no longer required to acquire a PersistenceManager in each business method. Instead, the PersistenceManager can be managed over a longer period of time, and it might be stored as an instance variable of the bean.

The behavior of stateful session beans is otherwise the same as for stateless session beans. The user has the choice of using javax.transaction.UserTransaction or javax.jdo.Transaction for transaction completion.

16.2 Entity Beans

There are several components in the entity bean scenario that need to be considered to understand the JDO integration possibilities.

EJBObject: this is the class of non-transactional instances to which all remote proxies make reference in order to execute remote calls. When the EJB container receives a remote call on this instance, it looks up a transactional instance of the Entity Bean associated with the user's transaction and delegates the business method to it.

Entity Bean: this is the class of transactional instances whose life cycle is managed by the container in order to service remote method calls. At certain points in its life cycle an instance will be associated with an EntityContext instance, a PersistenceManager instance, and a JDO instance.

The Entity Bean might be constructed manually by the bean developer or automatically by a tool provided by a JDO vendor or third party.

JDO PersistenceCapable class: this is the class of instances that actually implement the business method by accessing and possibly modifying state. During execution of its business methods, it may associate with other JDO instances which themselves are in the transaction but do not have remote references. If during the execution of a method, a reference to a JDO instance needs to be returned to the client, then the Entity Bean will ask the Home Interface to construct an EJBObject using the primary key of the referred JDO instance.

JDO instances might be used to implement both the Entity Bean and the helper instances that are transactional but not returned to clients as remote references.

16.2.1 BMP Entity Bean life cycle

The Entity Bean which delegates to a JDO instance will contain a reference to an EntityContext instance, a PersistenceManager instance, and a JDO instance. All of these will be set to null at creation time.

The setEntityContext method copies the value of the EntityContext parameter to the instance variable entityContext, looks up the PersistenceManagerFactory using JNDI, and sets it into the persistenceManagerFactory variable.

The unsetEntityContext method clears the entityContext variable and the persistenceManagerFactory variable.

The `ejbCreate` method creates a new instance of the corresponding JDO class corresponding to the primary key value, sets the `jdoInstance` variable, and calls `makePersistent` with the instance as a parameter.

The `ejbRemove` method calls `deletePersistent` with the JDO instance as a parameter.

The `ejbActivate` method acquires a `PersistenceManager` from the `PersistenceManagerFactory`, and finds the JDO instance with the specific primary key by calling `getObjectById` on the `PersistenceManager` instance.

The `ejbPassivate` method sets the `jdoInstance` to `null` and closes the `PersistenceManager`.

The `ejbLoad` and `ejbStore` methods are used to acquire and close the `PersistenceManager`. Business methods operating on the JDO instance will access fields which will transparently cause the appropriate state transitions of the persistent instance. The state of the JDO instances will be synchronized with the datastore during transaction completion.

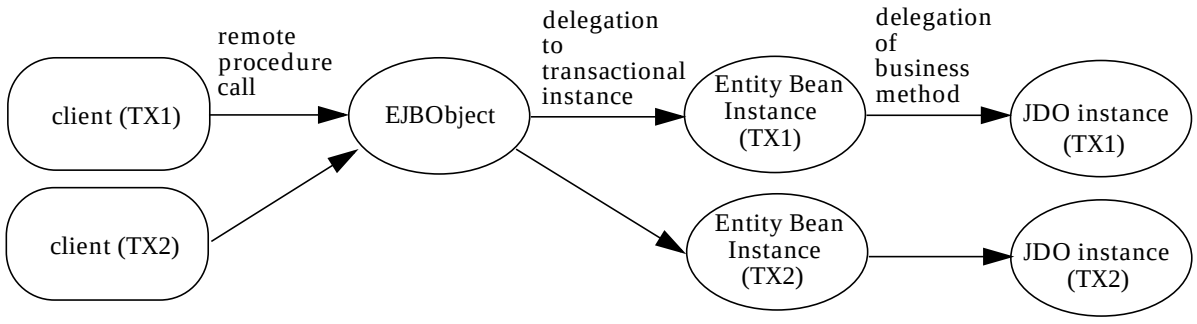
Business methods of the bean delegate to the JDO instance after performing parameter replacement operations. Parameters of Entity Bean reference types are transformed into parameters of JDO instances by using `getObjectById`.

During execution of business methods, persistent instances may be used without regard to EJB. The persistent instances might be of classes that have remote interfaces, but there is no requirement that an EJBObject be created for each persistent instance. Only those instances that need to be returned as a remote interface need to have an EJBObject created.

JDO instances may be returned as value types using standard Java Serialization. The main consideration is that the closure of instances via non-transient fields be appropriate. Considering that the closure of instances via persistent fields is potentially the entire database, care must be given to distinguishing persistent fields from serializable fields.

JDO instance return types are transformed into Entity Bean references by using `getObjectId` and looking up the EJB instance in the home interface.

Figure 16.0 Enterprise Java Beans: Entity Bean relationships



17 JDO Exceptions

The exception philosophy of JDO is to treat all exceptions as runtime exceptions. This preserves the transparency of the interface to the degree possible, allowing the user to choose to catch specific exceptions only when required by the application.

JDO implementations will often be built as layers on an underlying data store interface, which itself might use a layered protocol to another tier. Therefore, there are many opportunities for components to fail that are not under the control of the application.

Exceptions thus fall into several broad categories, each of which is treated separately:

- user errors that can be corrected and retried;
- user errors that cannot be corrected because the state of underlying components has been changed and cannot be undone;
- internal logic errors that should be reported to the JDO vendor's technical support;
- errors in the underlying data store that can be corrected and retried;
- errors in the underlying data store that cannot be corrected due to a failure of the data store or communication path to the data store;

Exceptions that are documented in interfaces that are used by JDO, such as the `Collection` interfaces, are used without modification by JDO. JDO exceptions that reflect underlying data store exceptions will wrap the underlying data store exceptions. JDO exceptions that are caused by user errors will contain the reason for the exception.

JDO `Exceptions` must be serializable.

17.1 JDOException

This is the base class for all JDO exceptions. It is a subclass of `RuntimeException`, and need not be declared or caught. It includes a descriptive String, an optional nested Exception array, and an optional failed Object.

Methods are provided to retrieve the nested exception array and failed object. If there are multiple nested exceptions, then each might contain one failed object. This will be the case where an operation requires multiple instances, such as commit, makePersistentAll, etc.

If the JDO `PersistenceManager` is internationalized, then the descriptive string should be internationalized.

17.1.1 JDOFatalException

This is the base class for errors that cannot be retried. It is a derived class of `JDOException`. This exception generally means that the transaction associated with the `PersistenceManager` has been rolled back, and the transaction should be abandoned.

17.1.2 JDOCanRetryException

This is the base class for errors that can be retried. It is a derived class of `JDOException`.

17.1.3 JDOUnsupportedOptionException

This class is a derived class of `JDOCanRetryException`. This exception is thrown by an implementation when it does not implement a JDO optional feature.

17.1.4 JDOUserException

This is the base class for user errors that can be retried. It is a derived class of `JDOCanRetryException`. Some of the reasons for this exception include:

- Object not `PersistenceCapable`. This exception is thrown when a method requires an instance of `PersistenceCapable` and the instance passed to the method does not implement `PersistenceCapable`. The failed Object has the failed instance.
- Extent not managed. This exception is thrown when `getExtent` is called with a class that does not have a managed extent.
- Object exists. This exception is thrown during flush of a new instance or an instance whose primary key changed where the primary key of the instance already exists in the data store. It might also be thrown during `makePersistent` if an instance with the same primary key is already in the `PersistenceManager` cache. The failed Object has the failed instance.
- Object does not exist. This exception is thrown when a hollow instance is being fetched and the object does not exist in the data store. The failed Object has the failed instance. This exception is also thrown when validation is requested for an instance retrieved by `getObjectById`.
- Object owned by another `PersistenceManager`. This exception is thrown when calling `makePersistent`, `makeTransactional`, `makeTransient`, `evict`, `refresh`, or `getObjectId` where the instance is already persistent or transactional in a different `PersistenceManager`. The failed Object has the failed instance.
- ObjectId not defined for transient instances. This exception is thrown when calling `getObjectId` on a transient instance.
- Non-unique ObjectId not valid after transaction completion. This exception is thrown when calling `getObjectId` on an object after transaction completion where the `ObjectId` is not managed by the application or data store.
- Object was changed or deleted by another transaction. This exception is thrown when committing an optimistic transaction, where a parallel transaction has changed the data store in an incompatible way. For example, an instance in the optimistic transaction was changed or deleted by another transaction.
- Unbound query variable. This exception is thrown during query compilation or execution if a variable not in the name scope of the class is referenced in the filter and is not named in the variable list.
- Unbound query parameter. This exception is thrown during query compilation or execution if there is an unbound query parameter.
- Query filter cannot be parsed. This exception is thrown during query compilation or execution if the filter cannot be parsed.

17.1.5 JDOFatalUserException

This is the base class for user errors that cannot be retried. It is a derived class of `JDOFatalException`.

- `PersistenceManager` was closed. This exception is thrown when any method is executed on the `PersistenceManager` instance except `isClosed()` after `close()` was called on that instance.

- Metadata unavailable. This exception is thrown if a request is made to the `JDOImplHelper` for metadata for a class, when the class has not been registered with the helper.

17.1.6 JDOFatalInternalException

This is the base class for JDO implementation failures. It is a derived class of `JDOFatalException`. This exception should be reported to the vendor for corrective action. There is no user action to recover.

17.1.7 JDODataStoreException

This is the base class for data store errors that can be retried. It is a derived class of `JDOCanRetryException`.

17.1.8 JDOFatalDataStoreException

This is the base class for fatal data store errors. It is a derived class of `JDOFatalException`. When this exception is thrown, the transaction has been rolled back.

- Transaction rolled back. This exception is thrown when the data store rolls back a transaction without the user asking for it. The cause may be a connection timeout, an unrecoverable media error, an unrecoverable concurrency conflict, or other cause outside the user's control.

18 XML Metadata

This chapter specifies the metadata that describes a persistence capable class. The metadata is stored in XML format. The information must be available when the class is enhanced.

The metadata associated with each persistence capable class must be contained within a file, and its format is as defined in the DTD. If the metadata is for only one class, then its file name should be `<class-name>.jdo`. If the metadata is for a package, then its file name should be `<package-name>.jdo`. For portability, files should be available via resources loaded by the same class loader as the class. These rules apply both to enhancement and to runtime. Hereinafter, the term "metadata" refers to the aggregate of all XML data for all packages and classes, regardless of their physical packaging.

The metadata is used both at enhancement time and at runtime. Information required at enhancement time is a subset of the information needed at runtime.

The metadata must declare all persistence-capable classes. If any field declarations are not provided in the metadata, then field metadata is defaulted for the missing field declarations. Therefore, the JDO implementation is able to determine based on the metadata whether a class is persistence-capable or not. And any class not known to be persistence-capable by the JDO specification (for example, `java.lang.Integer`) and not explicitly named in the metadata is not persistence-capable.

18.1 ELEMENT jdo

This element is the highest level element in the xml document. It is used to allow multiple packages to be described in the same document.

18.2 ELEMENT package

This element includes all classes in a particular package. The complete qualified package name is required.

18.3 ELEMENT class

This element includes fields declared in a particular class, and optional vendor extensions. The name of the class is required. The name is relative to the package name of the enclosing package.

Only persistence-capable classes may be declared. Non-persistence-capable classes must not be included in the metadata.

The identity type of classes is defaulted to datastore, unless explicitly specified. The `objectid-class` attribute is required only for application identity. The objectid class name uses Java rules for naming: if no package is included in the name, the package name is assumed to be the same package as the persistence-capable class. Inner classes are identified by the "$" marker.

The `requires-extent` attribute specifies whether an extent must be managed for this class. The `PersistenceManager.getExtent` method can only be executed for classes whose metadata includes `requires-extent = "true"`.

The `persistence-capable-superclass` attribute is the fully-qualified class name, and is required only for classes that have a persistence-capable superclass. If omitted, there is no superclass in the hierarchy that is persistence-capable.

18.4 ELEMENT field

The element `field` is optional, and the `name` attribute is the field name as declared in the class. If the field declaration is omitted in the xml, then the values of the attributes are defaulted.

The `persistence-modifier` attribute specifies whether this field is persistent, transactional, or none of these. There is special treatment for fields whose `persistence-modifier` is `persistent` or `transactional`. The default for the `persistence-modifier` attribute is based on the Java type and modifiers of the field:

- Fields with modifier `static`: `none`. No accessors or mutators will be generated for these fields during enhancement.
- Fields with modifier `transient`: `none`. Accessors and mutators will be generated for these fields during enhancement, but they will not delegate to the `StateManager`.
- Fields with modifier `final`: `none`. Accessors will be generated for these fields during enhancement, but they will not delegate to the `StateManager`.
- Fields of a type declared to be persistence-capable: `persistent`.
- Fields of the following types: `persistent`:
  - primitives: `boolean, byte, short, int, long, char, float, double`;
  - `java.lang` wrappers: `Boolean, Byte, Short, Integer, Long, Character, Float, Double`;
  - `java.lang: Locale, String`;
  - `java.math: BigDecimal, BigInteger`;
  - `java.util: Date, ArrayList, HashMap, HashSet, Hashtable, LinkedList, TreeMap, TreeSet, Vector, Collection, Set, List, and Map`;
  - Arrays of `java.lang` and `java.math` types specified above, arrays of `java.util.Date`, and arrays of persistence-capable types.
- Fields of types of user-defined classes and interfaces not mentioned above: `none`. No accessors or mutators will be generated for these fields.

The `primary-key` attribute is used to identify fields that have special treatment by the enhancer and by the runtime. The enhancer generates accessor methods for primary key fields that always permit access, regardless of the state of the instance. The mutator methods always delegate to the `jdoStateManager`, if it is non-`null`, regardless of the state of the instance.

The `null-value` attribute specifies the treatment of `null` values for persistent fields during storage in the data store. The default is `"none"`.

- `"none"`: store `null` values as `null` in the data store, and throw a `JDODataStoreException` if `null` values cannot be stored by the data store.

- "exception": always throw a JDOUserException if this field contains a null value at runtime when the instance must be stored;
- "default": convert the value to the data store default value if this field contains a null value at runtime when the instance must be stored.

The default-fetch-group attribute specifies whether this field is managed as a group with other fields. It defaults to "true" for fields of primitive types, java.util.Date, and fields of java.lang, java.math types specified above.

The embedded attribute specifies whether the field should be stored if possible as part of the instance instead of as its own instance in the datastore. It defaults to "true" for fields of primitive types, java.util.Date, and fields of java.lang, java.math, and array types specified above. This attribute is only a hint to the implementation. A compliant implementation is permitted to support these types as first class instances in the datastore. A portable application should not depend on the embedded treatment of persistent fields.

18.4.1 ELEMENT collection

This element specifies the element type and inverse of collection typed fields. The default is Collection typed fields are persistent, and the element type is Object. The element-type attribute specifies the type of the elements. The embedded-element attribute specifies whether the values of the elements should be stored if possible as part of the instance instead of as their own instances in the datastore.

18.4.2 ELEMENT map

This element specifies the treatment of keys and values of map typed fields. The default is map typed fields are persistent, and the key and value types are Object. The key-type and value-type attributes specify the types of the key and value, respectively. The embedded-key and embedded-value attributes specify whether the key and value should be stored if possible as part of the instance instead of as their own instances in the datastore. They default to "false".

18.4.3 ELEMENT array

This element specifies the element type of array typed fields. The default is array typed fields are persistent, according to the rules above. The embedded-element attribute specifies whether the values of the elements should be stored if possible as part of the instance instead of as their own instances in the datastore.

18.5 ELEMENT extension

This element specifies JDO vendor extensions. The vendor-name attribute is required. The vendor name "JDORI" is reserved for use by the JDO reference implementation. The key and value attributes are optional, and have vendor-specific meanings. They may be ignored by any JDO implementation.

18.6 The Document Type Descriptor

```
<?xml version="1.0" encoding="UTF-8"?>
<!ELEMENT jdo (package)+>
<!ELEMENT package ((class)+, (extension)*)>
<!ATTLIST package name CDATA #REQUIRED>
<!ELEMENT class (field|extension)*>
```

```
<!ATTLIST class name CDATA #REQUIRED>
<!ATTLIST class identity-type (application|datastore|none) 'datas-
tore'>
<!ATTLIST class objectid-class CDATA #IMPLIED>
<!ATTLIST class requires-extent (true|false) 'true'>
<!ATTLIST class persistence-capable-superclass CDATA #IMPLIED>
<!ELEMENT field ((collection|map|array)?, (extension)*)?>
<!ATTLIST field name CDATA #REQUIRED>
<!ATTLIST field persistence-modifier (persistent|transactional|none)
#IMPLIED>
<!ATTLIST field primary-key (true|false) 'false'>
<!ATTLIST field null-value (exception|default|none) 'none'>
<!ATTLIST field default-fetch-group (true|false) #IMPLIED>
<!ATTLIST field embedded (true|false) #IMPLIED>
<!ELEMENT collection (extension)*>
<!ATTLIST collection element-type CDATA #IMPLIED>
<!ATTLIST collection embedded-element (true|false) #IMPLIED>
<!ELEMENT map (extension)*>
<!ATTLIST map key-type CDATA #IMPLIED>
<!ATTLIST map embedded-key (true|false) #IMPLIED>
<!ATTLIST map value-type CDATA #IMPLIED>
<!ATTLIST map embedded-value (true|false) #IMPLIED>
<!ELEMENT array (extension)*>
<!ATTLIST array embedded-element (true|false) #IMPLIED>
<!ELEMENT extension (extension)*>
<!ATTLIST extension vendor-name CDATA #REQUIRED>
<!ATTLIST extension key CDATA #IMPLIED>
<!ATTLIST extension value CDATA #IMPLIED>
```

18.7 Example XML file

An example XML file for the query example classes follows. Note that all fields of both classes are persistent, which is the default for fields. The `emps` field in `Department` contains a collection of elements of type `Employee`, with an inverse relationship to the `dept` field in `Employee`.

In directory `com/xyz`, a file named `hr.jdo` contains:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE jdo SYSTEM "jdo.dtd">
<jdo>
<package name="com.xyz.hr">
<class name="Employee" identity-type="application" objectid-
class="EmployeeKey">
<field name="name" primary-key="true">
<extension vendor-name="sunw" key="index" value="btree"/>
</field>
<field name="salary" default-fetch-group="true"/>
<field name="dept">
<extension vendor-name="sunw" key="inverse" value="emps"/>
```

```
</field>
<field name="boss"/>
</class>
<class   name="Department"   identity-type="application"   objectid-
class="DepartmentKey">
<field name="name" primary-key="true"/>
<field name="emps">
<collection element-type="Employee">
<extension vendor-name="sunw" key="element-inverse" value="dept"/>
/collection>
</field>
</class>
</package>
</jdo>
```

19 Portability Guidelines

One of the objectives of JDO is to allow an application to be portable across multiple JDO implementations. This Chapter summarizes portability rules that are expressed elsewhere in this document. If all of these programming rules are followed, then the application will work in any JDO compliant implementation.

19.1 Optional Features

These features may be used by the application if the JDO vendor supports them. Since they are not required features, a portable application must not use them.

19.1.1 Optimistic Transactions

Optimistic transactions are enabled by the `PersistenceManagerFactory` or `Transaction` method `setOptimistic (true)`. JDO implementations that do not support optimistic transactions throw `JDOUnsupportedOptionException`.

19.1.2 Nontransactional Read

Nontransactional read is enabled by the `PersistenceManagerFactory` or `Transaction` method `setNontransactionalRead (true)`. JDO implementations that do not support nontransactional read throw `JDOUnsupportedOptionException`.

19.1.3 Nontransactional Write

Nontransactional write is enabled by the `PersistenceManagerFactory` or `Transaction` method `setNontransactionalWrite (true)`. JDO implementations that do not support nontransactional write throw `JDOUnsupportedOptionException`.

19.1.4 Transient Transactional

Transient transactional instances are created by the `PersistenceManager` `makeTransactional (Object)`. JDO implementations that do not support transient transactional throw `JDOUnsupportedOptionException`.

19.1.5 RetainValues

A portable application should run the same regardless of the setting of the `retainValues` flag.

19.2 Object Model

References among `PersistenceCapable` classes must be defined as First Class Objects in the model.

Second Class Object instances must not be shared among multiple persistent instances.

Arrays must not be shared among multiple persistent instances.

If arrays are passed by reference outside the defining class, the owning persistent instance must be notified via `jdoMakeDirty`.

The application must not depend on any sharing semantics of immutable class objects.

The application must not depend on knowing the exact class of a Second Class Object instance, as they may be substituted by a subclass of the type.

PersistenceCapable classes must not contain final non-static fields or methods or fields that start with "jdo".

19.3 JDO Identity

Applications must only construct `ObjectId` instances for classes that use application-defined JDO identity.

Applications must only compare `ObjectId` instances from different JDO implementations for classes that use application-defined JDO identity.

Applications must use `getObjectIdClass` to get the class for `ObjectId` types.

The equals method of any PersistenceCapable class using application identity must depend on all of the key fields.

Key fields can only be defined in the least-derived persistence-capable class in an inheritance hierarchy. All of the classes in the hierarchy use the same key class.

19.4 PersistenceManager

Instances of `PersistenceManager` must be obtained from a `PersistenceManagerFactory`, and not by construction.

19.5 Query

A query must constrain all variables used in any expressions with a contains clause referencing a persistent field of a `PersistenceCapable` class.

Not all datastores allow storing null-valued collections. Portable queries on these collections should use `isEmpty()` instead of comparing to `null`.

19.6 XML metadata

Portable applications will define all persistence-capable classes in the XML metadata.

19.7 Life cycle

Portable applications will not depend on requiring instances to be hollow or persistent-nontransactional, or to remain non-transactional in a transaction.

19.8 JDOHelper

Portable applications will use JDOHelper for state interrogations of instances of persistence-capable classes and for determining if an instance is of a persistence-capable class.

20 JDO Reference Enhancer

This chapter specifies the JDO Reference Enhancement, which specifies the contract between JDO persistence-capable classes and JDO StateManager in the runtime environment. The JDO Reference Enhancer modifies persistence-capable classes in order to run in the JDO environment and implement the required contract. The resulting classes, hereinafter referred to as enhanced classes, implement a contract used by the `JDOHelper`, the `JDOImplHelper`, and the `StateManager` classes.

The JDO Reference Enhancer is just one possible implementation of the JDO Reference Enhancement contract. Tools may instead preprocess or generate source code to create classes that implement this contract.

20.1 Overview

The JDO Reference Enhancer will be used to modify each persistence-capable class before using that persistence-capable class with the Reference Implementation `PersistenceManager` in the Java VM. It might be used before class loading or during the class loading process.

The JDO Reference Enhancer transforms the class by making specific changes to the class definition to enable the state of any persistent instances to be synchronized with the representation of the data in the data store.

Tools that generate source code or modify the java source code files must generate classes that meet the defined contract in this chapter.

The Reference Enhancer provides an implementation for the `PersistenceCapable` interface.

20.2 Goals

The following are the goals for the JDO Reference Enhancer:

- Binary compatibility and portability of application classes among JDO vendor implementations
- Binary compatibility between application classes enhanced by different JDO vendors at different times.
- Minimal intrusion into the operation of the class and class instances
- Provide metadata at runtime without requiring implementations to be granted reflect permission for non-private fields
- Values of fields can be read and written directly without wrapping code with accessors or mutators (`field1 += 13` is allowed, instead of requiring the user to code `setField1 (getField1() + 13)`)
- Navigation from one instance to another uses natural Java syntax without any requirement for explicit fetching of referenced instances

- Automatically track modification of persistent instances without any explicit action by the application or component developer
- Highest performance for transient instances of persistence-capable classes
- Support for all class and field modifiers
- Transparent operation of persistent and transient instances as seen by application components and persistent-capable classes
- Shared use of persistent-capable classes (utility components) among multiple JDO `PersistenceManager` instances in the same Java VM
- Preservation of the security of instances of `PersistenceCapable` classes from unauthorized access
- Support for debugging enhanced classes by line number

20.3 Enhancement: Architecture

The reference enhancement of `PersistenceCapable` classes has the primary objective of preserving transparency for the classes. Specifically, accesses to fields in the JDO instance are mediated to allow for initializing values of fields from the associated values in the data store and for storing the values of fields in the JDO instance into the associated values in the data store at transaction boundaries.

To avoid conflicts in the name space of the `PersistenceCapable` classes, all methods and fields added to the `PersistenceCapable` classes have the "`jdo`" prefix.

Enhancement might be performed at any time prior to use of the class by the application. During enhancement, special JDO class metadata must be available if any non-default actions are to be taken. The metadata is in XML format .

Specifically, the following will require access to special class metadata at class enhancement time, because these are not the defaults:

- classes are to use primary key or non-managed object identity;
- fields declared as transient in the class definition are to be persistent in the data store;
- fields not declared as transient in the class definition are to be non-persistent in the data store;
- fields are to be transactional non-persistent;
- fields with domains of references to `PersistenceCapable` classes are to be part of the default fetch group;
- fields with domains of primitive types (`char, byte, short, int, long, float, double`) or primitive wrapper types (`Char, Byte, Short, Integer, Long, Float, Double`) are not to be part of the default fetch group;
- fields with domains of `String` are not to be part of the default fetch group;
- fields with domains of array types are to be part of the default fetch group.

Enhancement makes changes to two categories of classes: `PersistenceCapable` and persistence aware. `PersistenceCapable` classes are those whose instances are allowed to be stored in a JDO-managed data store. Persistence aware classes are those which while not `PersistenceCapable` themselves, contain references to managed fields of classes that are `PersistenceCapable`.

To preserve the security of instances of `PersistenceCapable` classes, access restrictions to fields before enhancement will be propagated to accessor methods after enhancement. Further, in order to become the delegate of field access (`StateManager`) the caller must be authorized for `JDOPermission`.

A JDO implementation must interoperate with classes enhanced by the Reference Enhancer and with classes enhanced with other Vendor Enhancers. Additionally, classes enhanced by any Vendor Enhancers must interoperate with the Reference Implementation.

Name scope issues are minimized because the Reference Enhancer adds methods and fields that begin with "`jdo`", while methods and fields added by Vendor Enhancers must not begin with "`jdo`". Instead, they may begin with "`sunwjdo`", "`exlnjdo`" or other string that includes a vendor-identifying name and the "`jdo`" string.

Debugging by source line number must be preserved by the enhancement process. If any code is modified within method bodies which changes the byte code offsets within the method, then the line number references of the method must be updated to reflect the change.

The Reference Enhancer makes the following changes to `PersistenceCapable` classes:

- adds "`implements javax.jdo.PersistenceCapable`" to the class definition, and adds methods to implement its status query methods;
- adds a protected transient field named `jdoStateManager` in the least-derived persistence capable class, of type `javax.jdo.StateManager` to associate each instance with zero or one instance of JDO `StateManager`;
- adds a public method `jdoReplaceStateManager` to replace the value of the `jdoStateManager`, which invokes security checking for declared `JDOPermission`;
- adds a protected transient field named `jdoFlags` of type `byte` in the least-derived persistence capable class, to distinguish readable and writable instances from non-readable and non-writable instances;
- adds a public method `jdoReplaceFlags` to require the instance to request an updated value for the `jdoFlags` field from the `StateManager`;
- adds a protected default constructor if none exists, to be used by the implementation when a new persistent instance is required [no initialization of user-declared fields is done by this generated constructor];
- adds two public methods `jdoNewInstance`, one of which takes a parameter of type `StateManager`, to be used by the implementation when a new persistent instance is required (this method allows a performance optimization), and the other takes a parameter of type `StateManager` and a parameter of an `ObjectId` for key field initialization;
- adds public methods `jdoReplaceField` and `jdoReplaceFields` (with `int` and `int[]` parameters) to obtain values of specified fields from the `StateManager` and cache the values in the instance;
- adds public methods `jdoProvideFields` with `int` and `int[]` parameters to supply values of specific fields to the `StateManager`;
- adds a final static accessor method and mutator method for each field declared in the class, which delegates to the `StateManager` for values;
- changes the modifiers of all persistent fields to `private` to guarantee that all classes that access non-private fields be enhanced;

- adds code to the bodies of each method replacing getfield and putfield calls with calls to the generated accessor and mutator methods;
- adds a public method `void jdoCopyFields (Object other, int[] fieldNumbers)` to allow the `StateManager` to manage multiple images of the persistence capable instance;
- adds a method `protected static int jdoGetManagedFieldCount()` to manage the numbering of fields with respect to inherited managed fields.
- adds a field `private static int jdoInheritedFieldCount` which is set at class initialization time to the returned value of `super.jdoGetManagedFieldCount()`.
- adds final static fields `String[] jdoFieldNames`, `Class[] jdoFieldTypes`, and `byte[] jdoFieldFlags`, which contain the names, types, and flags of managed fields.
- adds final static field `Class jdoPersistenceCapableSuperclass`, which contains the `Class` of the `PersistenceCapable` superclass.
- adds a static initializer to register the class with the `JDOImplHelper`.
- adds a public method `jdoPreSerialize` to load all non-transient fields into the instance prior to serialization
- adds a public static field `serialVersionUID` if it does not already exist, and calculates its initial value based on the non-enhanced class definition.
- adds a public method `jdoNewObjectIdInstance()` which creates an instance of the jdo ObjectId for this class.
- adds public methods `jdoCopyKeyFieldsToObjectId (PersistenceCapable pc, Object oid)` and `jdoCopyKeyFieldsToObjectId (ObjectIdFieldManager fm, Object oid)`.

Enhancement makes the following changes to persistence aware classes:

- modifies bodies of each method that accesses public fields of `PersistenceCapable` classes, replacing getfield and putfield calls with calls to the generated accessor and mutator methods.

20.4 Inheritance

Enhancement allows a class to manage the persistent state only of declared fields. It is a future objective to allow a class to manage fields of a non-persistence capable superclass.

Fields that hide inherited fields (because they have the same name) are fully supported. The enhancer delegates accesses of inherited hidden fields to the appropriate class by referencing the appropriate method which is implemented in the declaring class.

All persistence capable classes in the inheritance hierarchy must use the same kind of JDO identity.

20.5 Field Numbering

Enhancement assigns field numbers to all managed (transactional or persistent) fields. Generated methods and fields that refer to fields (`jdoFieldNames, jdoFieldTypes, jdoFieldFlags, jdoGetManagedFieldCount, jdoCopyFields, jdo-`

`MakeDirty`, `jdoProvideField`, `jdoProvideFields`, `jdoReplaceField`, and `jdoReplaceFields`) are generated to include both transactional and persistent fields.

Relative field numbers are calculated at enhancement time. For each persistence capable class the enhancer determines the declared managed fields. To calculate the relative field number, the declared fields array is sorted by field name. Each managed field is assigned a relative field number, starting with zero.

Absolute field numbers are calculated at runtime, based on the number of inherited managed fields, and the relative field number. The absolute field number used in method calls is the relative field number plus the number of inherited managed fields.

The absolute field number is used in method calls between the `StateManager` and `PersistenceCapable`; and in the reference implementation, between the `StateManager` and `StoreManager`.

20.6 Serialization

Serialization of a transient instance results in writing an object graph of objects connected via non-transient fields. The explicit intent of JDO enhancement of serializable classes is to permit serialization of transient instances or persistent instances to a format that can be deserialized by either an enhanced or non-enhanced class.

When the `writeObject` method is called on a class in order to serialize it, all fields not declared as transient must be loaded into the instance. This function is performed by the enhancer-generated method `jdoPreSerialize`. This method simply delegates to the `StateManager` to ensure that all persistent non-transient fields are loaded into the instance. [Fields not declared as transient and not declared as persistent must have been loaded by the `PersistenceCapable` class an application-specific way.]

If a standard serialization is done to an enhanced class instance, the fields added by the enhancer will not be serialized because they are declared to be transient.

In order to allow a non-enhanced class to deserialize the stream, the `serialVersionUID` for the enhanced and non-enhanced classes must be identical. If the `serialVersionUID` field does not already exist in the non-enhanced class, the enhancer will calculate it (excluding any enhancer-generated fields) and add it to the enhanced class.

If a `PersistenceCapable` class is declared to implement `java.io.Serializable` but does not contain implementations of `writeObject`, `readObject`, `writeReplace`, or `readResolve`, then the enhancer will generate `writeObject` and `readObject`. Fields that are required to be present during serialization operations will be be explicitly instantiated by the generated method `jdoPreSerialize`, which will be called by the enhancer-generated `writeObject`.

If a `PersistenceCapable` class implements `java.io.Serializable`, then the non-transient fields might be instantiated prior to serialization. However, the closure of instances reachable from this instance might include a large part of instances in the data store.

The results of restoring a serialized persistent instance graph is a graph of interconnected transient instances. The method `readObject` is not enhanced, as it deals only with transient instances.

20.7 Cloning

If a standard clone is made of a persistent instance, then incorrect results may occur, as the `jdoFlags` and `jdoStateManager` fields will also be cloned. Therefore, the enhancer modifies the cloning behavior by modifying methods that invoke `super.clone()`, setting these two fields to indicate that the clone is a transient instance. Otherwise, all of the fields in the clone contain the standard shallow copy of the fields of the cloned instance.

The reference enhancer will modify methods that invoke `super.clone()` (including the `clone` method itself) to reset these two fields immediately after returning from `super.clone()`. If there is no `clone` method defined in the class, then one will be created, which calls `super.clone` and then resets the two fields. An optimized enhancer might create a `clone` method only in the topmost persistence capable class in the inheritance hierarchy of the class. The generated `clone` method will be ignored by the VM if the user does not declare the class to implement `Cloneable`.

20.8 Introspection (Java core reflection)

No changes are made to the behavior of introspection. The current state of all fields is exposed to the reflection APIs.

This is not at all what some users might expect. It is a future objective to more gracefully support introspection of fields in persistent instances of persistence capable classes.

20.9 Field Modifiers

Fields in `PersistenceCapable` classes are treated by the enhancer in one of several ways, based on their modifiers as declared in the Java language and their enhanced modifiers as declared by the `PersistenceCapable` MetaData.

These modifiers are orthogonal to the modifiers defined by the Java language. They have default values based on modifiers defined in the class for the fields. They may be specified in the XML metadata used at enhancement time.

20.9.1 Non-persistent

Non-persistent fields are ignored by the enhancer. They are assumed to lie outside the domain of persistence. They might be changed at will by any method based only on the private / protected / public modifiers. There is no enhancement of accesses to non-persistent fields.

The default modifier is non-persistent for fields identified as transient in the class declaration.

20.9.2 Transactional non-persistent

Transactional non-persistent fields are non-persistent fields whose values are saved and restored during rollback. Their values are not stored in the data store. There is no enhancement of read accesses to transactional non-persistent fields. Write accesses are always mediated (the `StateManager` is called on write).

20.9.3 Persistent

Persistent fields are fields whose values are synchronized with values in the data store. The synchronization is performed transparent to the methods in the `PersistenceCapable` class.

The default persistence-modifier is persistent for fields not declared as transient in the class declaration.

The modification to the class by the enhancer depends on whether the persistent field is a member of the default fetch group.

If the persistent field is a member of the default fetch group, then the enhanced code behaves as follows. The constant values `READ_OK`, `READ_WRITE_OK`, and `LOAD_REQUIRED` are defined in interface `PersistenceCapable`.

- for read access, `jdoFlags` is checked for READ_OK or READ_WRITE_OK. If it is then the value in the field is retrieved. If it is not, then the `StateManager` instance is requested to load the value of the field from the data store, which might cause the `StateManager` to populate values of all default fetch group fields in the instance, and other values as defined by the JDO vendor policy. This behavior is not required, but optional. If the `StateManager` chooses, it may simply populate the value of the specific field requested. Upon conclusion of this process, the `jdoFlags` value might be set by the `StateManager` to `READ_OK` and the value of the field is retrieved. If not all fields in the default fetch group were populated, the `StateManager` must not set the `jdoFlags` to be `READ_OK`.
- for write access, `jdoFlags` is checked for `READ_WRITE_OK`. If it is `READ_WRITE_OK`, then the value is stored in the field. If it is not `READ_WRITE_OK`, then the `StateManager` instance is requested to load the state of the values from the data store, which might cause the `StateManager` to populate values of all default fetch group fields in the instance. Upon conclusion of the load process, the `jdoFlags` value might be set by the `StateManager` to `READ_WRITE_OK` and the value of the field is stored.

If the persistent field is not a member of the default fetch group, then each read and write access to the field is delegated to the `StateManager`. For read, the value of the field is obtained from the `StateManager`, stored in the field, and returned to the caller. For write, the proposed value is given to the `StateManager`, and the returned value from the `StateManager` is stored in the field.

The enhanced code that fetches or modifies a field that is not in the default fetch group first checks to see if there is an associated `StateManager` instance and if not (the case for transient instances) the access is allowed without intervention.

20.9.4 PrimaryKey

Primary key fields are not part of the default fetch group; all changes to the field can be intercepted by the `StateManager`. This allows special treatment by the implementation if any primary key fields are changed by the application.

Primary key fields are always available in the instance, regardless of the state. Therefore, read access to these fields is never mediated.

20.9.5 Embedded

Fields identified as embedded in the XML metadata are treated as containing embedded instances. The default for `Collection` and `Map` types is embedded. This is to allow JDO implementations to map `PersistenceCapable` field types to embedded objects (aggregation by containment pattern).

20.9.6 Null-value

Fields of `Object` types might be mapped to data store elements that do not allow null values. The default behavior “none” is that no special treatment is done for null-valued fields.

In this case, null-valued fields throw a `JDOUserException` when the instance is flushed to the data store and the data store does not support null values.

However, the treatment of null-valued fields can be modified by specifying the behavior in the XML metadata. The null-value setting of "default" is used when the default value for the data store element is to be used for null-valued fields.

If the application requires non-null values to be stored in this field, then the setting should be "exception", which throws a `JDOUserException` if the value of the field is null at the time the instance is stored in the datastore.

For example, if a field of type Integer is mapped to a data store int value, committing an instance with a field value of null where the null-value setting is "default" will result in a zero written to the data store element. Similarly, a null-valued `String` field would be written to the data store as an empty (zero length) `String` where the null-value setting is "default".

20.10 Treatment of standard Java field modifiers

20.10.1 Static

Static fields are ignored by the enhancer. They are not initialized by JDO; accesses to values are not mediated.

20.10.2 Final

Final fields are treated as non-persistent and non-transactional by the enhancer. Final fields are initialized only by the constructor, and their values cannot be changed after construction of the instance. Therefore, their values cannot be loaded or stored by JDO; accesses are not mediated.

This treatment might not be what users expect; therefore, final fields are not supported as instance fields, only static fields are supported by ignoring them.

20.10.3 Private

Private fields are accessed only by methods in the class itself. JDO handles private fields according to the semantic that values are stored in private fields by the enhancement-generated `jdoSetXXX` methods or `jdoReplaceField` which become part of the class definition. Values are loaded from private fields by the enhancement-generated `jdoGetXXX` or `jdoProvideField` methods which become part of the class definition.

20.10.4 Public, Protected

Public fields are not recommended to be persistent in persistence capable classes. Classes that make reference to persistent public fields (persistence aware) must be enhanced themselves prior to execution. To guarantee that persistence aware classes are enhanced, the modifier for public persistent fields is changed during enhancement to be `private`. Therefore, if an unenhanced persistence aware class attempts to access a public field of a persistence capable class, an `IllegalAccessError` will be thrown by the JVM. Protected fields may be persistent; the assumption is that all classes in a package will be enhanced.

20.11 Fetch Groups

Fetch groups represent a grouping of fields that are retrieved from the data store together. Typically, a data store associates a number of data values together and efficiently retrieves these values. Other values require extra method calls to retrieve.

For example, in a relational database, the Employee table defines columns for Employee id, Name, and Position. These columns are efficiently retrieved with one data transfer request. The corresponding fields in the Employee class might be part of the default fetch group.

Continuing this example, there is a column for Department dept, defined as a foreign key from the Employee table to the Department table, which corresponds to a field in the Employee class named dept of type Department. The runtime behavior of this field depends on the mapping to the Department table. The reference might be to a derived class and it might be expensive to determine the class of the Department instance. Therefore, the dept field will not be defined as part of the default fetch group, even though the foreign key that implements the relationship might be fetched when the Employee is fetched. Rather, the value for the dept field will be retrieved from the `StateManager` every time it is requested. Similarly, the `StateManager` will be called for each modification of the value of dept.

The `jdoFlags` field is the indicator of the state of the default fetch group.

20.12 jdoFlags Definition

The value of the `jdoFlags` field is entirely determined by the `StateManager`. The `StateManager` calls the `jdoReplaceFlags` method to inform the persistence capable class to retrieve a new value for the `jdoFlags` field. The values permitted are constants defined in the interface `PersistenceCapable`: `READ_OK`, `READ_WRITE_OK`, and `LOAD_REQUIRED`.

During the transition from transient to a managed life cycle state, the `jdoFlags` field is set to `LOAD_REQUIRED` by the persistence capable instance, to indicate that the instance is not ready. During the transition from a managed state to transient, the `jdoFlags` field is set to `READ_WRITE_OK` by the persistence capable instance, to indicate that the instance is available for read and write of any field.

The `jdoFlags` field is a byte with three possible values and associated meanings:

- 0 - `READ_WRITE_OK`: the values in the default fetch group can be read or written without intermediation of the associated `StateManager` instance.
- -1 - `READ_OK`: the values in the default fetch group can be read but not written without intermediation of the associated `StateManager` instance.
- 1 - `LOAD_REQUIRED`: the values in the default fetch group cannot be accessed, either for read or write, without intermediation of the associated `StateManager` instance.

20.13 Modified field access

The enhancer modifies field accesses to guarantee that the values of fields are retrieved from the data store prior to application usage.

For any field access that reads the value of a field, the getfield byte code is replaced with a call to a generated local method, `jdoGetXXX`, which determines based on the kind of field (default fetch group or not) and the state of the `jdoFlags` whether to call the `StateManager` with the field number needed.

For any field access that stores the new value of a field, the putfield byte code is replaced with a call to a generated local method, `jdoSetXXX`, which determines based on the kind of field (default fetch group or not) and the state of the `jdoFlags` whether to call the `StateManager` with the field number needed. A JDO implementation might perform

field validation during this operation and might throw a `JDOUserException` if the value of the field does not meet the criterion.

Table 5: Field access mediation

field type	read access	write access	flags
transient transactional	not checked	checked	CHECK_WRITE
primary key	not checked	mediated	MEDIATE_WRITE
default fetch group	checked	checked	CHECK_READ_WRITE
non-default fetch group	mediated	mediated	MEDIATE_READ_WRITE

not checked: access is always granted

checked: the condition of `jdoFlags` is checked to see if access should be mediated

mediated: access is always mediated (delegated to the `StateManager`)

flags: the value in the `jdoFieldFlags` field

The flags is allowed to have one of four possible values as defined in `PersistenceCapable`:

```
0 - CHECK_WRITE
1 - MEDIATE_WRITE
2 - CHECK_READ_WRITE
3 - MEDIATE_READ_WRITE
```

20.14 Generated fields in `PersistenceCapable`

These fields are generated in the enhanced class.

```
protected transient javax.jdo.StateManager jdoStateManager;
```

This field contains the managing StateManager instance, if this instance is being managed.

```
protected transient byte jdoFlags;
```

This field contains an indicator of the state of the default fetch group fields.

```
private static int jdoInheritedFieldCount;
```

This field is initialized at class load time to be the number of fields managed by the superclasses of this class, or to zero if there is no persistence capable superclass.

```
private static String[] jdoFieldNames;
```

This field is initialized at class load time to an array of names of persistent and transactional fields. The position in the array is the relative field number of the field.

```
private static Class[] jdoFieldTypes;
```

This field is initialized at class load time to an array of types of persistent and transactional fields. The position in the array is the relative field number of the field.

```
private static byte[] jdoFieldFlags;
```

This field is initialized at class load time to an array of flags indicating the characteristics of each persistent and transactional field.

```
private static Class jdoPersistenceCapableSuperclass;
```

This field is initialized at class load time to the class instance of the `PersistenceCapable` superclass.

20.15 Generated methods in `PersistenceCapable`

These methods are declared in interface `PersistenceCapable`.

```
public boolean jdoIsPersistent();
public boolean jdoIsTransactional();
public boolean jdoIsNew();
public boolean jdoIsDirty();
public boolean jdoIsDeleted();
```

These methods check if the `jdoStateManager` field is `null`. If so, they return `false`. If not, they delegate to the corresponding method in `StateManager`.

```
public Object jdoGetObjectId();
public Object jdoGetTransactionalObjectId();
```

These methods check if the `jdoStateManager` field is `null`. If so, they return `null`. If not, they delegate to the corresponding method in `StateManager`.

```
public void jdoMakeDirty (String fieldName);
```

This method checks if the `jdoStateManager` field is `null`. If so, it returns silently. If not, it delegates to the corresponding method in `StateManager`.

```
public PersistenceManager jdoGetPersistenceManager();
```

This method checks if the `jdoStateManager` field is `null`. If so, it returns `null`. If not, it delegates to the corresponding method in `StateManager`.

```
public synchronized void jdoReplaceStateManager (StateManager sm);
```

This method is implemented as synchronized to resolve race conditions, if more than one `StateManager` attempts to acquire ownership of the same `PersistenceCapable` instance. This method asks the associated `StateManager` to return the new value for the `StateManager`. If the change was not requested by the `StateManager` , then a `JDOUserException` is thrown.

If the current value of the `jdoStateManager` field is `null`, then a security check is performed with the value of `JDOPermission`. Thus, only callers authorized for `JDOPermission` are allowed to set the `StateManager`. If the security check succeeds, the `jdoStateManager` field is set to the value of the parameter `sm`.

This method will be called by the `StateManager` on state changes when transitioning an instance from transient to persistent, transient to transactional, and from any state to transient.

During the execution of this method, if the `StateManager` is successfully set to a non-null value, the `jdoFlags` field is set to `LOAD_REQUIRED` to indicate that mediation is required; and if the `StateManager` is successfully set to null, the `jdoFlags` field is set to `READ_WRITE_OK` to indicate that no mediation is required.

```
public PersistenceCapable jdoNewInstance(StateManager sm);
```

This method uses the default constructor and assigns the `StateManager` argument. If the class is abstract, a `jdoUserException` is thrown.

```
public PersistenceCapable jdoNewInstance(StateManager sm, Object
objectid);
```

This method uses the default constructor and assigns the `StateManager` parameter and copies the key fields from the `objectid` parameter. If the class is abstract, a `jdoUserException` is thrown.

```
public void jdoReplaceFlags();
```

This method asks the `StateManager` for a new value for the `jdoFlags` field. This method is called by the StateManager when it wants to set a new value.

```
public void jdoReplaceField (int field);
```

```
public void jdoReplaceFields (int[] fields);
```

This method calls the `StateManager` to get a new value for the field(s) from the `StateManager`.

This method is used by the `StateManager` to store values from the data store in the instance.

For each field number in the fields parameter, the field number is examined to see if it is a declared field or an inherited field. If it is inherited, then the call is delegated to the superclass. If it is declared, then the appropriate `replacingXXXField` method is called, which in turn calls the `StateManager` to retrieve the new value for the field.

```
public void jdoProvideField (int field);
```

```
public void jdoProvideFields (int[] fields);
```

This method calls the `StateManager` `replacingXXXField` method to supply the value of the specified field to the `StateManager`.

This method is used by the `StateManager` to retrieve values from the instance, during flush to the data store or for in-memory query processing.

For each field number in the fields parameter, the field number is examined to see if it is a declared field or an inherited field. If it is inherited, then the call is delegated to the superclass. If it is declared, then the appropriate `giveXXXField` method is called, which in turn calls the `StateManager` with the value for the field.

```
protected int jdoGetManagedFieldCount();
```

This method returns the number of managed fields declared by this class and inherited from all superclasses. This method is generated in the class to allow the class to determine at runtime the number of inherited fields, without having introspection code in the enhanced class.

```
public void jdoCopyFields (Object other, int[] fieldNumbers);
```

This method copies the specified fields from the other instance, which must be the same class as this instance, and owned by the same `StateManager`. This method is called by the `StateManager` to create before images of instances for the purpose of rollback.

```
private void jdoPreSerialize();
```

This method is called by the generated or modified `writeObject` in order to allow the instance to fully populate serializable fields. This method delegates to the `StateManager` so that fields can be fetched by the JDO implementation prior to serialization.

```
public void jdoCopyKeyFieldsToObjectId (ObjectIdFieldManager fm,
Object oid)
```

This method is called by the JDO implementation (or implementation helper) to populate key fields in object id instances.

```
public void jdoCopyKeyFieldsToObjectId (PersistenceCapable pc, Ob-
ject oid)
```

This method is called by the JDO implementation (or implementation helper) to populate key fields in object id instances from persistence-capable instances. This might be used to implement getObjectId or getTransactionalObjectId.

20.16 Example class: Employee

The following class definitions for persistence capable classes are used in the examples:

```
package com.xyz.hr;
class Employee {
Employee boss; // relative field 0
Department dept; // relative field 1
int empid; // relative field 2, key field
String name; // relative field 3
}
package com.xyz.hr;
class Department {
Collection emps; // relative field 0
String name; // relative field 1
}
```

20.16.1 Generated fields

```
protected transient javax.jdo.StateManager jdoStateManager = null;
protected transient byte jdoFlags =
   javax.jdo.PersistenceCapable.READ_WRITE_OK;
// if no superclass, the following:
private static int jdoInheritedFieldCount = 0;
// otherwise,
private static int jdoInheritedFieldCount =
   <persistence-capable-superclass>.jdoGetManagedFieldCount();
private static String[] jdoFieldNames = {“boss”, “dept”, “empid”,
“name”};
private static Class[] jdoFieldTypes = {Employee.class, Depart-
ment.class, int.class, String.class};
private static byte[] jdoFieldFlags = {MEDIATE_READ_WRITE,
MEDIATE_READ_WRITE, MEDIATE_WRITE, CHECK_READ_WRITE};
// if no PersistenceCapable superclass, the following:
private static Class jdoPersistenceCapableSuperclass = null;
// otherwise,
private static Class jdoPersistenceCapableSuperclass = <pc-super>;
```

20.16.2 Generated interrogatives

```
public boolean jdoIsPersistent() {
   return jdoStateManager==null?false:
      jdoStateManager.isPersistent(this);
}
public boolean jdoIsTransactional(){
   return jdoStateManager==null?false:
      jdoStateManager.isTransactional(this);
}
public boolean jdoIsNew(){
   return jdoStateManager==null?false:
      jdoStateManager.isNew(this);
}
public boolean jdoIsDirty(){
   return jdoStateManager==null?false:
      jdoStateManager.isDirty(this);
}
public boolean jdoIsDeleted(){
   return jdoStateManager==null?false:
      jdoStateManager.isDeleted(this);
}
public void jdoMakeDirty (String fieldName){
   if (jdoStateManager==null) return;
   jdoStateManager.MakeDirty(this, fieldName);
}
public PersistenceManager jdoGetPersistenceManager(){
   return jdoStateManager==null?null:
      jdoStateManager.getPersistenceManager(this);
}
```

20.16.3 Generated static initializer

The generated static initializer constructs the values for `jdoFieldNames` and `jdoFieldTypes` and calls the static method in `JDOImplHelper` to register itself.

```
static {
JDOImplHelper.registerClass (
   Employee.class,
   jdoFieldNames,
   jdoFieldTypes,
   jdoFieldFlags,
   jdoPersistenceCapableSuperclass,
```

```
   new Employee());
}
```

20.16.4 Generated constructor

The enhancer will generate a constructor for non-abstract classes if a public or protected default (no-args) constructor does not exist. Using the same example as above, the enhancer by default will generate the constructor byte codes as if the method were declared as:

```
protected Employee () {
   <super>();
}
```

20.16.5 Generated jdoNewInstance helpers

The first generated helper assigns the value of the passed parameter to the `jdoStateManager` field.

```
public PersistenceCapable jdoNewInstance(StateManager sm) {
// if class is abstract, throw new JDOUserException() instead
   Employee pc = new Employee ();
   pc.jdoStateManager = sm;
   pc.jdoFlags = LOAD_REQUIRED;
   return pc;
}
```

The second generated helper assigns the value of the passed parameter to the `jdoStateManager` field, and initializes the values of the key fields from the `oid` parameter.

```
public PersistenceCapable jdoNewInstance(StateManager sm, Object oid) {
// if class is abstract, throw new JDOUserException() instead
   Employee pc = new Employee ();
   pc.jdoStateManager = sm;
   pc.jdoFlags = LOAD_REQUIRED;
   EmployeeKey pck = (EmployeeKey)oid;
   pc.empid = pck.empid;
   return pc;
}
```

20.16.6 Generated jdoGetManagedFieldCount

The generated method returns the number of managed fields in this class plus the number of inherited managed fields. This method is expected to be executed only during class loading of the subclasses.

```
protected static int jdoGetManagedFieldCount () {
   return jdoInheritedFieldCount + jdoFieldNames.length;
}
```

20.16.7 Generated jdoReplaceStateManager

The generated method asks the current `StateManager` to approve the change or validates the caller's authority to set the state.

```
void public jdoReplaceStateManager (javax.jdo.StateManager sm) {
// throws exception if current sm didn't request the change
if (jdoStateManager != null) {
   jdoStateManager = jdoStateManager.replacingStateManager (this,
      sm);
} else {
   SecurityManager sec = System.getSecurityManager();
   if (sec != null) {
      sec.checkPermission( // throws exception if not authorized
         new javax.jdo.JDOPermission("setStateManager"));
   }
   jdoStateManager = sm;
}
jdoFlags = javax.jdo.PersistenceCapable.LOAD_REQUIRED;
}
```

20.16.8 Generated jdoGetObjectId and jdoGetTransactionalObjectId

These generated methods delegate to the `StateManager`.

```
public Object jdoGetObjectId(){
   return jdoStateManager==null?null:
      jdoStateManager.getObjectId(this);
}
public Object jdoGetTransactionalObjectId((){
   return jdoStateManager==null?null:
      jdoStateManager.getTransactionalObjectId(this);
}
```

20.16.9 Generated jdoGetXXX methods (one per persistent field)

The generated `jdoGetXXX` methods have exactly the same semantics as the byte code getfield. They return the value of one specific field. The field returned was either cached in the instance or retrieved from the `StateManager`.

The name of the generated method is constructed from the fully qualified class name plus field name. This allows for hidden fields to be supported explicitly, and for classes to be enhanced independently.

The modifier (MMM in the examples) is the same modifier as the corresponding field definition. Therefore, access to the method is controlled by the same policy as for the corresponding field.

Using the same example as above, the enhancer by default will generate the byte codes as if the method were declared as:

```
MMM static String
```

```
jdoGetcom_xyz_hr_Employee_name(Employee x) {
   // this field is in the default fetch group
   if (x.jdoFlags <= READ_WRITE_OK) {
        // ok to read
        return x.name;
   }
   // field needs to be fetched from StateManager
   // this call might result in name being stored in instance
   if (x.jdoStateManager.isLoaded
           (x, x.jdoInheritedFieldCount + 3))
        return x.name;

    return x.jdoStateManager.getStringField(x,
                                      x.jdoInheritedFieldCount + 3,
                                          x.name);
   }

MMM static com.xyz.hr.Department
jdoGetcom_xyz_hr_Employee_dept(Employee x) {
    // this field is not in the default fetch group
   if (x.jdoStateManager != null) {
      if (x.jdoStateManager.isLoaded
         (x, x.jdoInheritedFieldCount + 1))
      return x.dept;
   return (com.xyz.hr.Department)
        x.jdoStateManager.getObjectField(x,
                                      x.jdoInheritedFieldCount + 1,
                                        x.dept);
   } else {
      return x.dept;
   }
}
```

20.16.10 Generated jdoSetXXX methods (one per persistent field)

The generated `jdoSetXXX` methods have exactly the same semantics as the byte code `setField`. They modify the value of one specific field. The field modified was either cached in the instance or notified the `StateManager`.

If the field is in the default fetch group, and the `jdoFlags` are set to `READ_WRITE_OK`, then this method stores the new value in the field and returns. If the `jdoFlags` is not set to `READ_WRITE_OK`, then this method calls the `StateManager` with the old value of the

field and the new value. The `StateManager` will return the value to be stored in the instance.

The name of the generated method is constructed from the fully qualified class name plus field name. This allows for hidden fields to be supported explicitly, and for classes to be enhanced independently.

Using the same example as above, the enhancer by default will generate the byte codes as if the method were declared as:

```
MMM static void
jdoSetcom_xyz_hr_Employee_name(Employee x, String newValue) {
   // this field is in the default fetch group
   if (x.jdoFlags == READ_WRITE_OK) {
        // ok to read, write
        x.name = newValue;
        return;
   }
   x.jdoStateManager.setStringField(x,
      jdoInheritedFieldCount + 3,
      x.name,
      newValue);
   }

MMM static void
jdoSetcom_xyz_hr_Employee_dept(Employee  x,  com.xyz.hr.Department
newValue) {
   // this field is not in the default fetch group
   if (x.jdoStateManager != null) {
      x.jdoStateManager.setObjectField(x,
         jdoInheritedFieldCount + 1,
         x.dept,
                                        newValue);
   } else {
      x.dept = newValue;
   }
}
```

20.16.11 Generated jdoReplaceField and jdoReplaceFields

The generated `jdoReplaceField` retrieves a new value from the `StateManager` for one specific field based on field number. This method is called by the `StateManager` whenever it wants to update the value of a field in the instance, for example to store values in the instance from the data store.

This may be used by the StateManager to clear fields and handle cleanup of the objects currently referred to by the fields (e. g., embedded objects).

The enhancer by default will generate the `jdoReplaceField` and `jdoReplaceFields` byte codes as if the methods were declared as:

```
void jdoReplaceField (int fieldNumber) {
int relativeField = fieldNumber - jdoInheritedFieldCount;
   switch (relativeField) {
      case (0): boss = (Employee)
         jdoStateManager.replacingObjectField (this,
         fieldNumber);
         break;
      case (1): dept = (Department)
         jdoStateManager.replacingObjectField (this,
         fieldNumber);
         break;
      case (2): empid =
         jdoStateManager.replacingIntField (this,
         fieldNumber);
         break;
      case (3): name =
         jdoStateManager.replacingStringField (this,
            fieldNumber);
         break;
      default:
         /* if there is a pc superclass, delegate to it
         if (relativeField < 0) {
            super.jdoReplaceField (fieldNumber);
         } else {
            throw new javax.jdo.JDOFatalInternalException();
         }
         */
         // if there is no pc superclass, throw an exception
         throw new javax.jdo.JDOFatalInternalException();
   } // switch
}
void jdoReplaceFields (int[] fieldNumbers) {
for (int i = 0; i < fieldNumbers.length; ++i) {
   int fieldNumber = fieldNumbers[i];
   jdoReplaceField (fieldNumber);
}
```

```
}
```

20.16.12 Generated jdoProvideField and jdoProvideFields

The generated `jdoProvideField` gives the current value of one field to the `StateManager`. This method is called by the `StateManager` whenever it wants to get the value of a field in the instance, for example to store the field in the data store.

The enhancer by default will generate the `jdoProvideField` and `jdoProvideFields` byte codes as if the methods were declared as:

```
void jdoProvideField (int fieldNumber) {
   int relativeField = fieldNumber - jdoInheritedFieldCount;
   switch (relativeField) {
      case (0): jdoStateManager.providedIntField(this,
         fieldNumber, boss);
         break;
      case (1): jdoStateManager.providedObjectField(this,
         fieldNumber, dept);
         break;
      case (2): jdoStateManager.providedIntField(this,
         fieldNumber, empid);
         break;
      case (3): jdoStateManager.providedStringField(this,
         fieldNumber, name);
         break;
      default:
         /* if there is a pc superclass, delegate to it
         if (relativeField < 0) {
            super.jdoProvideField (fieldNumber);
         } else {
            throw new javax.jdo.JDOFatalInternalException();
         }
         */
         // if there is no pc superclass, throw an exception
         throw new javax.jdo.JDOFatalInternalException();
   } // switch
}
void jdoProvideFields (int[] fieldNumbers) {
for (int i = 0; i < fieldNumbers.length; ++i) {
   int fieldNumber = fieldNumbers[i];
   jdoProvideField (fieldNumber);
}
}
```

20.17 Generated jdoCopyFields method

The generated `jdoCopyFields` copies fields from another instance to this instance. This method might be used by the `StateManager` to create before images of instances for roll-back.

This method copies values for fields declared in this class, and then calls the superclass (if there is a persistence-capable superclass) to handle the other values. This is done instead of calling another method (such as jdoCopyField) due to the overhead of checking and casting the parameter.

To avoid security exposure, `jdoCopyFields` can only be invoked when both instances are owned by the same `StateManager`. Thus, a malicious user cannot use this method to copy fields from a managed instance to a non-managed instance, or to an instance managed by a stealth `StateManager`.

```
public void jdoCopyFields (Object pc, int[] fieldNumbers){
   // make sure other class is the same class
   if (!pc instanceof <class>)
      throw new javax.jdo.JDOFatalInternalException();
   Employee other = (Employee) pc;
   // the other instance must be owned by the same StateManager
   if (other.jdoStateManager != this.jdoStateManager)
      throw new javax.jdo.JDOUserException();
   for (int i = 0; i < fieldNumbers.length; ++i) {
      int relativeField = fieldNumbers[i] - jdoInheritedFieldCount;
      switch (relativeField) {
      case (0): this.boss = other.boss;
         break;
      case (1): this.dept = other.dept;
         break;
      case (2): this.empid = other.empid;
         break;
      case (3): this.name = other.name;
         break;
      default: break; // other fields handled in superclass
      } // switch
   } // for loop
   /* this class has no superclass, so return; if it had a
      superclass, it would handle the fields as follows:
   super.jdoCopyFields (pc, fieldNumbers);
    */
} // jdoCopyFields
```

20.18 Generated writeObject method

If no user-written method `writeObject` exists, then one will be generated. The generated `writeObject` makes sure that all persistent and transactional serializable fields are loaded into the instance, and then the default output behavior is invoked on the output stream.

If the class is serializable (either by explicit declaration or by inheritance) then this code will guarantee that the fields are loaded prior to standard serialization. If the class is not serializable, then this code will never be executed.

Note that there is no modification of a user's `readObject`. During the execution of `readObject`, a new transient instance is created. This instance might be made persistent later, but while it is being constructed by serialization, it remains transient.

```
private void writeObject(java.io.ObjectOutputStream out)
      throws java.io.IOException{
   jdoPreSerialize();
   out.defaultWriteObject ();
}
```

20.19 Generated jdoPreSerialize method

The generated `jdoPreSerialize` method makes sure that all persistent and transactional serializable fields are loaded into the instance by delegating to the corresponding method in `StateManager`.

```
private void jdoPreSerialize() {
if (jdoStateManager != null)
   jdoStateManager.preSerialize(this);
}
```

20.20 Generated jdoCopyKeyFieldsToObjectId

The generated methods copy key field values from the `PersistenceCapable` instance or from the `ObjectIdFieldHandler`.

```
public void jdoCopyKeyFieldsToObjectId (ObjectIdFieldManager fm,
Object oid) {
   ((EmployeeKey)oid).empid = fm.fetchStringField (2);
}
public void jdoCopyKeyFieldsToObjectId (PersistenceCapable pc, Ob-
ject oid) {
   ((EmployeeKey)oid).empid = ((Employee)pc).empid;
}
```

21 Interface StateManager

This chapter specifies the `StateManager` interface, which is responsible for managing the state of fields of persistence-capable classes in order to run in the JDO environment.

21.1 Overview

A class which implements the JDO `StateManager` interface must be supplied by the JDO implementation. There is no user-visible behavior for this implementation; its only caller from the user's perspective is the `PersistenceCapable` class.

21.2 Goals

This interface allows the JDO implementation to completely control the behavior of the `PersistenceCapable` classes under management. In particular, the implementation may choose to exploit the caching capabilities of `PersistenceCapable` or not.

The architecture permits JDO implementations to have a singleton `StateManager` for all `PersistenceCapable` instances; a `StateManager` for all `PersistenceCapable` instances associated with a particular `PersistenceManager` or `PersistenceManagerFactory`; a `StateManager` for all `PersistenceCapable` instances of a particular class; or a `StateManager` for each `PersistenceCapable` instance. This list is not intended to be exhaustive, but simply to identify the cases which might be typical.

21.3 StateManager Management

The following methods provide for updating the corresponding `PersistenceCapable` fields. These methods are intended to be called only from the `PersistenceCapable` instance.

```
public StateManager replacingStateManager (Object pc, StateManager sm);
```

The current `StateManager` should be the only caller of `PersistenceCapable.replaceStateManager`, which calls this method. This method should only be called when the current `StateManager` wants to set the `jdoStateManager` field to `null` to transition the instance to transient.

The `jdoFlags` are completely controlled by the `StateManager`. The meaning of the values are the following:

```
0: READ_WRITE_OK
any negative number: READ_OK
any positive number: LOAD_REQUIRED
```

```
public byte replacingFlags(Object pc);
```

This method is called by the PersistenceCapable in response to the StateManager calling jdoReplaceFlags. The PersistenceCapable will store the returned value into its jdoFlags field.

21.4 PersistenceManager Management

The following method provides for getting the PersistenceManager. This method is intended to be called only from the PersistenceCapable instance.

```
public PersistenceManager getPersistenceManager (Object pc);
```

21.5 Dirty management

The following method provides for marking the PersistenceCapable instance dirty:

```
public void makeDirty (Object pc, String fieldName);
```

21.6 State queries

The following methods are delegated from the PersistenceCapable class, to implement the associated behavior of PersistenceCapable.

```
public boolean isPersistent (Object pc);
public boolean isTransactional (Object pc);
public boolean isNew (Object pc);
public boolean isDirty (Object pc);
public boolean isDeleted (Object pc);
```

21.7 JDO Identity

```
public Object getObjectId (Object pc);
```

This method returns the JDO identity of the instance.

```
public Object getTransactionalObjectId (Object pc);
```

This method returns the transactional JDO identity of the instance.

21.8 Serialization support

```
public void preSerialize (Object pc);
```

This method loads all non-transient persistent fields in the PersistenceCapable instance, as a precursor to serializing the instance. It is called by the generated jdoPreSerialize() method in the PersistenceCapable class.

21.9 Field Management

The StateManager completely controls the behavior of the PersistenceCapable with regard to whether fields are loaded or not. Setting the value of the jdoFlags field in the PersistenceCapable directly affects the behavior of the PersistenceCapable with regard to fields in the default fetch group.

- The StateManager might choose to never cache any field values in the PersistenceCapable, but rather to retrieve the values upon request. To implement this strategy, the StateManager will always use the LOAD_REQUIRED value for the jdoFlags, and will always return false to any call to isLoaded.
- The StateManager might choose to selectively retrieve and cache field values in the PersistenceCapable. To implement this strategy, the StateManager will always use the LOAD_REQUIRED value for jdoFlags, and will return true to calls to isLoaded that refer to fields that are cached in the PersistenceCapable.
- The StateManager might choose to retrieve at one time all field values for fields in the default fetch group, and to take advantage of the performance optimization in the PersistenceCapable. To implement this strategy, the StateManager will use the LOAD_REQUIRED value for jdoFlags only when the fields in the default fetch group are not cached. Once all of the fields in the default fetch group are cached in the PersistenceCapable, the StateManager will set the value of the jdoFlags to READ_OK. This will probably be done during the call to isLoaded made for one of the fields in the default fetch group, and before returning true to the method, the StateManager will call jdoReplaceFields with the field numbers of all fields in the default fetch group, and will call jdoReplaceFlags to set jdoFlags to READ_OK.
- The StateManager might choose to manage updates of fields in the default fetch group individually. To implement this strategy, the StateManager will not use the READ_WRITE_OK value for jdoFlags. This will result in the PersistenceCapable always delegating to the StateManager for any change to any field. In this way, the StateManager can reliably tell when any field changes, and can optimize the writing of data to the store.

The following method is used by the PersistenceCapable to determine whether the value of the field is already cached in the PersistenceCapable instance. If it is cached (perhaps during the execution of this method) then the value of the field is returned by the PersistenceCapable method without further calls to the StateManager.

```
boolean isLoaded (Object pc, int field);
```

21.9.1 User-requested value of a field

The following methods are used by the PersistenceCapable instance to inform the StateManager of a user-initiated request to access the value of a persistent field.

The pc parameter is the instance of PersistenceCapable making the call; the field parameter is the field number of the field; and the currentValue parameter is the current value of the field in the instance.

The current value of the field is passed as a parameter to allow the StateManager to cache values in the PersistenceCapable. If the value is cached in the PersistenceCapable, then the StateManager can simply return the current value provided with the method call.

```
public boolean getBooleanField (Object pc, int field, boolean currentValue);
public char getCharField (Object pc, int field, char currentValue);
public byte getByteField (Object pc, int field, byte currentValue);
```

```
public short getShortField (Object pc, int field, short currentVal-
ue);
public int getIntField (Object pc, int field, int currentValue);
public long getLongField (Object pc, int field, long currentValue);
public float getFloatField (Object pc, int field, float currentVal-
ue);
public double getDoubleField (Object pc, int field, double cur-
rentValue);
public String getStringField (Object pc, int field, String cur-
rentValue);
public Object getObjectField (Object pc, int field, Object cur-
rentValue);
```

21.9.2 User-requested modification of a field

The following methods are used by the `PersistenceCapable` instance to inform the `StateManager` of a user-initiated request to modify the value of a persistent field.

The `pc` parameter is the instance of `PersistenceCapable` making the call; the `field` parameter is the field number of the field; the `currentValue` parameter is the current value of the field in the instance; and the `newValue` parameter is the value of the field given by the user method.

```
public boolean setBooleanField (Object pc, int field, boolean cur-
rentValue, boolean newValue);
public char setCharField (Object pc, int field, char currentValue,
char newValue);
public byte setByteField (Object pc, int field, byte currentValue,
byte newValue);
public short setShortField (Object pc, int field, short currentVal-
ue, short newValue);
public int setIntField (Object pc, int field, int currentValue, int
newValue);
public long setLongField (Object pc, int field, long currentValue,
long newValue);
public float setFloatField (Object pc, int field, float currentVal-
ue, float newValue);
public double setDoubleField (Object pc, int field, double cur-
rentValue, double newValue);
public String setStringField (Object pc, int field, String cur-
rentValue, String newValue);
public Object setObjectField (Object pc, int field, Object cur-
rentValue, Object newValue);
```

21.9.3 StateManager-requested value of a field

The following methods inform the `StateManager` of the value of a persistent field requested by the `StateManager`.

The pc parameter is the instance of PersistenceCapable making the call; the field parameter is the field number of the field; and the currentValue parameter is the current value of the field in the instance.

```
public void providedBooleanField (Object pc, int field, boolean currentValue);
public void providedCharField (Object pc, int field, char currentValue);
public void providedByteField (Object pc, int field, byte currentValue);
public void providedShortField (Object pc, int field, short currentValue);
public void providedIntField (Object pc, int field, int currentValue);
public void providedLongField (Object pc, int field, long currentValue);
public void providedFloatField (Object pc, int field, float currentValue);
public void providedDoubleField (Object pc, int field, double currentValue);
public void providedStringField (Object pc, int field, String currentValue);
public void providedObjectField (Object pc, int field, Object currentValue);
```

21.9.4 StateManager-requested modification of a field

The following methods ask the StateManager for the value of a persistent field requested to be modified by the StateManager.

The pc parameter is the instance of PersistenceCapable making the call; and the field parameter is the field number of the field.

```
public boolean replacingBooleanField (Object pc, int field);
public char replacingCharField (Object pc, int field);
public byte replacingByteField (Object pc, int field);
public short replacingShortField (Object pc, int field);
public int replacingIntField (Object pc, int field);
public long replacingLongField (Object pc, int field);
public float replacingFloatField (Object pc, int field);
public double replacingDoubleField (Object pc, int field);
public String replacingStringField (Object pc, int field);
public Object replacingObjectField (Object pc, int field);
```

22 JDOPermission

A permission represents access to a system resource. In order for a resource access to be allowed for an applet (or an application running with a security manager), the corresponding permission must be explicitly granted to the code attempting the access.

The `JDOPermission` class provides a marker for the security manager to grant access to classes to perform privileged operations necessary for JDO implementations.

There are two JDO permissions defined:

- `setStateManager`: this permission allows an instance to manage an instance of Persiste, which allows the instance to access and modify any fields defined as persistent or transactional. This permission is similar to but allows access to only a subset of the broader `ReflectPermission (“suppressAccessChecks”)`. This permission is checked by the `PersistenceCapable.setStateManager` method.
- `getMetadata`: this permission allows an instance to access the metadata for any registered `PersistenceCapable` class. This permission allows access to a subset of the broader `RuntimePermission ("accessDeclaredMembers")`. This permission is checked by the `JDOImplHelper.getJDOImplHelper` method.

Use of `JDOPermission` allows the security manager to restrict potentially malicious classes from accessing information contained in instances of `PersistenceCapable`.

A sample policy file entry granting code from the `/home/jdoImpl` directory permission to get metadata and manage `PersistenceCapable` instances is

```
grant codeBase "file:/home/jdoImpl/" {
  permission javax.JDOPermission "getMetadata";
  permission javax.JDOPermission "setStateManager";
};
```

23 JDO Query BNF

Grammar Notation

The grammar notation is taken from the Java Language Specification:
Terminal symbols are shown in **`bold`** in the productions of the lexical and syntactic grammars, and throughout this specification whenever the text is directly referring to such a terminal symbol. These are to appear in a program exactly as written.
Nonterminal symbols are shown in *`italic`* type. The definition of a nonterminal is introduced by the name of the nonterminal being defined followed by a colon. One or more alternative right-hand sides for the nonterminal then follow on succeeding lines.
The suffix “opt”, which may appear after a terminal or nonterminal, indicates an optional symbol. The alternative containing the optional symbol actually specifies two right-hand sides, one that omits the optional element and one that includes it.
When the words “one of” follow the colon in a grammar definition, they signify that each of the terminal symbols on the following line or lines is an alternative definition.

Parameter Declaration

This section describes the syntax of the `declareParameters` argument.

```
DeclareParameters:

 Parameters ,opt

Parameters:

 Parameter

 Parameters , Parameter

Parameter:

 Type Identifier
```

Variable Declaration

This section describes the syntax of the `declareVariables` argument.

```
DeclareVariables:

 Variables ;opt
```

Variables:

Variable

Variables ; Variable

Variable:

Type Identifier

Import Declaration

This section describes the syntax of the `declareImports` argument.

DeclareImports:

ImportDeclarations ;opt

ImportDeclarations:

ImportDeclaration

ImportDeclarations ; ImportDeclaration

ImportDeclaration:

import *Name*

Order Specification

This section describes the syntax of the `setOrdering` argument.

SetOrdering:

OrderSpecifications ,opt

OrderSpecifications:

OrderSpecification

OrderSpecifications , OrderSpecification

OrderSpecification:

Identifier **ascending**

Identifier **descending**

Filter Expression

This section describes the syntax of the `setFilter` argument.
Basically, the query filter expression is a Java boolean expression, where some of the Java expression are not permitted. The description follows the structure of the grammar for of Java expression in chapter 19.12 of the Java Language Specification. The description is bottom-up, i.e. the last rule Expression is the root of the filter expression syntax.
Please note, the grammar allows arbritary method calls (MethodInvocation), where JDO only permits calls to the methods `contains()`, `isEmpty()` , and a number of `String` methods. This restriction cannot be expressed in terms of the syntax and has to be ensured by a semantic check.

Primary:

Literal

this

(*Expression* **)**

FieldAccess

MethodInvocation

ArgumentList:

Expression

ArgumentList **,** *Expression*

FieldAccess:

Primary **.** *Identifier*

MethodInvocation:

Name **(** *ArgumentListopt* **)**

Primary **.** *Identifier* **(** *ArgumentListopt* **)**

PostfixExpression:

Primary

Name

UnaryExpression:

```
 + UnaryExpression
 - UnaryExpression
 UnaryExpressionNotPlusMinus
UnaryExpressionNotPlusMinus:
 PostfixExpression
 ~ UnaryExpression
 ! UnaryExpression
 CastExpression
CastExpression:
 ( Type ) UnaryExpression
MultiplicativeExpression:
 UnaryExpression
 MultiplicativeExpression * UnaryExpression
 MultiplicativeExpression / UnaryExpression
 MultiplicativeExpression % UnaryExpression
AdditiveExpression:
 MultiplicativeExpression
 AdditiveExpression + MultiplicativeExpression
 AdditiveExpression - MultiplicativeExpression
RelationalExpression:
 AdditiveExpression
 RelationalExpression < AdditiveExpression
 RelationalExpression > AdditiveExpression
```

```
 RelationalExpression <= AdditiveExpression
 RelationalExpression >= AdditiveExpression
EqualityExpression:
 RelationalExpression
 EqualityExpression == RelationalExpression
 EqualityExpression != RelationalExpression
AndExpression:
 EqualityExpression
 AndExpression & EqualityExpression
ExclusiveOrExpression:
 AndExpression
 ExclusiveOrExpression ^ AndExpression
InclusiveOrExpression:
 ExclusiveOrExpression
 InclusiveOrExpression | ExclusiveOrExpression
ConditionalAndExpression:
 InclusiveOrExpression
 ConditionalAndExpression && InclusiveOrExpression
ConditionalOrExpression:
 ConditionalAndExpression
 ConditionalOrExpression || ConditionalAndExpression
Expression:
 ConditionalOrExpression
```

Types

This section decribes a type specification, used in a parameter or variable declaration or in a cast expression.

Type

PrimitiveType

Name

PrimitiveType:

NumericType

boolean

NumericType:

IntegralType

FloatingPointType

IntegralType: one of

byte short int long char

FloatingPointType: one of

float double

Literals

A literal is the source code representation of a value of a primitive type, the `String` type, or the `null` type. Please refer to the Java Language Specification for the lexical structure of IntegerLiterals, FloatingPointLiterals, CharacterLiterals and StringLiterals.

IntegerLiteral: ...

FloatingPointLiteral: ...

BooleanLiteral: one of

true false

CharacterLiteral: ...

StringLiteral: ...

```
NullLiteral:
 null
Literal:
 IntegerLiteral
 FloatingPointLiteral
 BooleanLiteral
 CharacterLiteral
 StringLiteral
 NullLiteral
```

Names

```
Name:
 Identifier
 QualifiedName
QualifiedName:
 Name . Identifier
```

24 To Do List

This chapter will become “Items deferred to the next release” upon publication.

24.1 Nested Transactions

Define the semantics of nested transactions.

24.2 Savepoint, Undosavepoint

Related to nested transactions, savepoints allow for making changes to instances and then undoing those changes without making any data store changes. It is a single-child nested transaction.

24.3 Inter-PersistenceManager References

Explain how to establish and maintain relationships between persistent instances managed by different `PersistenceManager`s.

24.4 Enhancer Invocation API

A standard interface to call the enhancer will be defined.

24.5 Prefetch API

A standard interface to specify prefetching of instances by policy will be defined. The intended use it to allow the application to specify a policy whereby instances of persistence capable classes will be prefetched from the datastore when related instances are fetched. This should result in improved performance characteristics if the prefetch policy matches actual application access patterns.

24.6 BLOB/CLOB datatype support

JDO implementations can choose to implement mapping from java.sql.Blob datatype to byte arrays, and java.sql.Clob to String or other java type; but these mappings are not standard, and may not have the performance characteristics desired.

24.7 Managed (inverse) relationship support

In order for JDO implementations to be used for container managed persistence entity beans, relationships among persistent instances need to be explicitly managed. See the EJB Specification 2.0, sections 9.4.6 and 9.4.7 for requirements. The intent is to support these semantics when the relationships are identified in the metadata as inverse relationships.

24.8 Closing PersistenceManagerFactory at VM shutdown

The JDO specification does not provide for an orderly closing of PersistenceManagerFactories at VM shutdown. We could define a method to close them, but access to the method would need to be protected from unauthorized use.

24.9 Case-Insensitive Query

Use of String.toLowerCase() as a supported method in query filters would allow case-insensitive queries.

24.10 String conversion in Query

Supported String constructors String(<integer expression>) and String(<floating-point expression>) would make queries more flexible.

Appendix A: References

[1] Enterprise JavaBeans (EJB) specification - draft version 2.0:

`http://java.sun.com/products/ejb/docs.html`

[2] Java Transaction API (JTA) specification - version 1.0

`http://java.sun.com/products/jta/`

[3] Java 2 Platform Enterprise Edition (J2EE), Platform specification:

`http://java.sun.com/j2ee/docs.html`

[4] Java 2 Platform Enterprise Edition (J2EE), Connector Architecture:

`http://java.sun.com/j2ee/apidocs/`

`http://java.sun.com/j2ee/download.html#connectorspec`

Appendix B: Design Decisions

This appendix outlines some of the design decisions that were considered and not taken, along with the rationale.

B.1 Enhancer

The enhancer could generate code that would delegate to the associated StateManager every access (read or write) for every field. This design was rejected because of several factors.

- Code bloat: the enhanced code would add an extra method call to every access to a persistent field.
- Performance: the calls to the `StateManager` would add extra cycles to every access to a persistent field, even if the field were already fetched into the persistent instance.

The enhancer could require complete metadata descriptions for all persistence-capable classes and persistent and transactional fields, and further require that all classes be available during enhancement of any class.

This would allow the enhancer to generate the most efficient code, but imposes an extra burden on the user to keep the metadata and class definition absolutely in sync. If a field were declared in a class after the metadata was defined, the user would have to update the metadata to add the new field.

Requiring access to all classes during enhancement of any class was also seen as an extra burden on the user, who would have to execute the enhancement in an environment that did not necessarily reflect the runtime environment. There is also a performance penalty and additional complexity for the enhancer.

The decision that was taken was that the ehnancer must be able to determine the persistence-modifier (persistent or none) from the Java modifiers and type of a field. Further, the information needed to enhance a class is only the class file for the class being enhanced, plus the metadata for the class and classes directly reachable (via references or inheritance) from the class.

The java byte codes generated in a class for a field in another class do not contain much information about the modifiers (final or transient) of the field. They do have the field name and the field type, and whether the field is static. There is an implied access control that permits the generated access (package, protected, or public) but no distinction among the choices.

Therefore, a field that is not declared in the metadata must be enhanced to generate an accessor and mutator even though the field is not persistent. For example, for a final int field declared in a class, the field is not persistent, so it is not included in the list of persistent/transactional fields, but an accessor is generated for it. This accessor will only be used by other classes' accesses, and access will not be mediated (the StateManager will never be called). Accesses within the class are not enhanced.

B.2 PersistenceCapable

The PersistenceCapable interface could be eliminated entirely in favor of having all interrogatives operate via the PersistenceManager, not directly on the JDO instance. This would make the JDO instance entirely user-written. However, the impact would be that in order to find out which PersistenceManager, if any, was responsible for the JDO instance,

a new singleton would have to be provided. The singleton would have to register all PersistenceManager instances and ask each if it managed a specific JDO instance.

This was deemed too complex to manage, as well as too slow to find simple information that should be easily available.

B.3 Collection Factory

The collection factory could be defined as methods on PersistenceManager or as methods on a separate interface. Also, a single method that takes a type, or multiple methods, one for each type could be defined.

The decision was taken to define two methods on PersistenceManager based on the requirement to create an instance of a collection based on the type of an existing instance. This operation would be complex if individual methods were used, one per type.

A convenience interface can easily be created using the defined methods.

Appendix C: Revision History

This appendix outlines the significant changes during the evolution of the specification.

C.1 Changes since Draft 0.1

Added Appendix for revision history

Added Appendix for design decisions not taken

C.2 Changes since Draft 0.2

Changed the description for the persistent state (cached non-transactional values)

Added JDO instance state transition diagram and descriptions of state transitions.

Enhanced description of non-data store JDO identity.

Added persistent-new-dirty and persistent-new-clean states to the life cycle.

Removed the `checkpoint` method from the `Transaction` interface. This functionality is now done by the `TRANSACTION_RETAIN_VALUES` `Transaction` flag.

Added `jdoCopy` to the `PersistenceCapable` interface.

Added `Query` interface.

C.3 Changes since Draft 0.3

Changed `Query` signatures for `setVars` and `setParams`.

Changed all "`set`" `Query` signatures to return `void` instead of "`Query`".

Added description of key (JDO Identity) change semantics.

Added life cycle description for `deletePersistent`, a new interrogatory `jdoIsDeleted`, and two new states persistent-new-deleted and persistent-deleted.

Added Chapter 6 Persistent Object Model, which specifies the field types for persistent fields, including the required `Collection` types.

Added descriptions of enhancement to Chapter 13 JDO Enhancer, including serialization, cloning, and reflection.

Added multiple object versions of `makePersistent`, `makeTransactional`, `makeNontransactional`.

C.4 Changes since Draft 0.4

C.4.1 PersistenceManager

Removed flush and postCompletion from the API.

Changed refresh to indicate it is only effective in optimistic transactions.

Removed getFlags and setFlags, substituting getXXX and setXXX for all options.

Added getProperties which returns VendorName, VersionNumber, etc.

Added get/setUserObject which allows a user-specified object to be remembered by the PersistenceManager.

Required the implementation to support PersistenceManagerFactory, and specified the interface for it.

Associated the concept of Extent with makePersistent and deletePersistent. Only classes with a managed Extent can be parameters of these methods.

Added getObjectIdClass to allow the application to get the ObjectId class for a class.

C.4.2 Query

Added newQuery (Class cls, String filter).

Changed signature of compile to return void. This is not required to do anything but validate query elements.

Made the Query implementation class serializable. A serialized and restored query instance can be bound to a PersistenceManager by newQuery (Object).

Removed execute methods with four, five, and six parameters.

Allowed Date comparisons for equality and range queries.

Allowed String comparisons for equality and range queries.

Added “this” as a valid keyword in filters.

Added a query option to indicate faster queries that don’t execute the filter on cached instances.

Clarified that portable applications require all variables to be scoped by a contains clause.

Defined that unbound variables not scoped by a contains clause are scoped by the Extent of the class.

C.4.3 Object Model

Changed the name of “Tracked Second Class Object” to “Second Class Object”.

Required a transaction to be in effect to execute makePersistent and deletePersistent.

Allowed an implementation to treat all reference types as First Class Objects.

Sharing of SCOs is permitted but the semantics are not guaranteed to be portable.

C.4.4 Life Cycle

Removed state persistent-new-clean and changed the name of persistent-new-dirty to persistent-new.

Updated life cycle state diagram to simplify state transition descriptions.

Added section describing optimistic transaction state changes.

C.4.5 PersistenceCapable

Removed methods jdoIsReadReady and jdoIsWriteReady. None of the application’s business, these.

Changed the semantics of jdoIsTransactional to return false if an instance is read in an optimistic transaction. In an optimistic transaction, only new, deleted, modified instances and instances made transactional return true.

Added jdoGetPersistenceManager, jdoGetObjectId, and jdoMakeDirty.

C.5 Changes since Draft 0.5

Clarified NontransactionalRead, Optimistic, and RetainValues flag dependencies.

Added a table and diagrams of life cycle transitions.

Changed data store ObjectId to allow primitive wrapper classes to be used.

Added failed object array and methods to JDOException, JDOCanRetryException, JDODataStoreException, and JDOUserException.

Added a Chapter on Application Portability Guidelines.

Added a Chapter on XML Metadata.

Added two collection factories to PersistenceManager.

Added connection factory to PersistenceManagerFactory.

C.6 Changes since Draft 0.6 (Participant Review Draft)

Updated life cycle table to match transition descriptions for persistent-nontransactional instances. Clarified that all data accessed while a datastore transaction is in progress will be transactional.

Added a discussion on inheritance issues for persistence capable classes.

Added class JDOHelper with static methods to avoid calling JDO specific methods on PersistenceCapable classes.

Added a discussion on using the life cycle methods of PersistenceManager to clarify that the correct method must be called if an instance that implements a Collection interface is to be a parameter.

Query use of operator = was extended to include pre- and post-increment and -decrement operators.

Query variables need not be unique; if they need to be unique, then uniqueness can be specified with an additional query term.

Query examples were clarified as to their intent.

The terms persistent, non-persistent, transient were made consistent throughout the document. “Persistent field” and “non-persistent field” refer to fields as declared in the JDO metadata. “Transient field” refers to the field modifiers (orthogonal to persistent / non-persistent) and “transient instance” refers to an instance of a persistence capable class that is not persistent. “Persistent instance” refers to an instance of a persistence capable class that is persistent.

Derived fields were removed. These fields were supposed to be non-persistent fields whose values depended on values of persistent fields. For example, age depends on birthdate. The application will have to have a method age() instead of an instance variable age.

Transactional non-persistent fields were added. These fields have their values saved and restored during rollback transitions along with persistent fields.

More details were added on use of JDO in the EJB environment.

C.7 Changes since Draft 0.7

Binary compatibility table was added to 2.1.1.

Optional features were added to Portability Guidelines.

Section 5.5.2 was clarified to require that the JDO identity instance can be obtained immediately after the transition from transient to persistent-new.

The treatment of marking fields dirty for hidden fields was changed.

A table of arithmetic operators was added to the Query section.

C.8 Changes since Draft 0.8

Query filter defaults to “true” if not specified.

Added java.lang.BigInteger, java.lang.BigDecimal to object model.

Added cast operator (class) to query filter syntax.

Added bitwise invert operator to query filter syntax.

Added unary + to query filter syntax.

Added parentheses to query filter syntax.

Added String methods beginsWith and endsWith to query filter syntax.

Added chapter for StateManager interface.

Rewrote entire chapter on Reference Enhancer.

Updated PersistenceCapable interface to match Reference Enhancer.

Removed PersistenceManager.setObjectId.

Updated XML to conform to xml4j DOM and Apache/Xerces verifying parsers.

Added second-class XML attribute to field element.

Added null-value XML attribute to field element. This attribute specifies the behavior of the runtime system when a null-valued field mapped to a non-nullable data store element is stored. The user can choose to throw an exception or to convert the null value to a default data store value.

Changed the description of life cycle states and enhancer to indicate that primary key field access is always permitted, regardless of the life cycle state.

Added Extent chapter. The Extent interface was defined to be the result type of PersistenceManager.getExtent. The interface does not have the methods of Collection, so it can only be used for iteration or for specifying the candidate instances for Query.

Fields in an inherited class may not be managed by a persistence capable class. It is a future objective to allow a class to manage the state of inherited fields if it directly derives from a non-persistence capable class .

Clarified the behavior of null parameters in calls to PersistenceManager. Null values are permitted as parameters for PersistenceCapable instances, and permitted as elements of Collection and Object[] parameters, but are not permitted as parameters for Collection and Object[].

Added JDOPermission class to allow security management to enable jdo implementations without requiring ReflectPermission, which is too permissive.

C.9 Changes since Draft 0.9

Updated XML Metadata

- Added xml version number
- Changed definition of class element to allow multiple field, vendor elements
- Added jdo element which contains multiple package elements
- Added key-type to field element for Map types.
- Changed key-type in class element to identity-type
- Changed key-class in class element to objectid-class
- Added inverse to field element for managed relationships

- Added has-extent to class element

Fixed missing “static” in generated jdoInheritedFieldCount.

Fixed jdoGetXXX/jdoSetXXX in enhanced code for non-dfg fields. Transient instances would have thrown null pointer exception.

Fixed missing generated method in PersistenceCapable: PersistenceCapable jdoNewInstance(StateManager sm)

Fixed the reference to the Connector Architecture in Appendix A.

Updated ordering to include expressions and restrict the types ofordering expressions to primitives except boolean, wrappers except Boolean, BigDecimal, BigInteger, and Date.

Removed bitwise AND, OR, and XOR from query operators.

Changed signatures of PersistenceManager methods getObjectById and getTransactionalInstance to include a boolean flag indicating whether to validate that the instance exists in the datastore.

Clarified that getObjectId returns the identity as of the beginning of the transaction, in case the identity is being modified in the transaction.

C.10 Changes since draft 0.91

Changed xml has-extent to requires-extent

Corrected the signature of replacingIntField in StateManager.

Corrected the example code generated for PersistenceCapable jdoReplaceField.

Corrected the name of the verify parameter to validate in the signature of getObjectById.

Removed getTransactionalInstance in favor of overloading the meaning of getObjectById.

Changed the requirement to expose the hollow state to the application. A JDO implementation might perform a state transition of a hollow instance as if the application had read a field.

Changed inheritance rules to allow non-persistence-capable classes to have persistence-capable superclasses and subclasses.

Corrected the description of the field name in the markDirty method so an unqualified name refers to the field in the most-derived class.

Corrected the signature of the newInstance method in JDOHelper to return Object.

Updated the instance callback description to include the rationale and environment for callbacks.

Updated makePersistent and deletePersistent to remove the restriction that the class of the instances must have an Extent.

The behavior of failing instances in the life cycle methods was clarified to specify that all instances will be attempted, and all failing instances will be included in the exception.

The newCollectionInstance was modified to include an initialContents parameter.

A new method newMapInstance was created to allow construction of a second class map instance.

Optimistic transaction management was clarified to specify that instances accessed during an optimistic transaction are not enlisted in any datastore transaction until commit.

The ordering specification was modified to include String.

The isEmpty method was added to the allowed Collection methods in query.

The treatment of null-valued collection fields was specified to be identical to fields containing empty collections.

Specified the behavior of the iterator of an Extent if there are deleted or newly persistent instances in the Extent.

The chapter on EJB has been substantially redone.

Exceptions were updated as to the contents of the failed object array.

The meaning of JDOHelper.getObjectId versus PersistenceManager.getObjectId was clarified with regard to change of identity within a transaction.

Fixed (removed) all references to reference parameter in StateManager.

Changed interface in PersistenceCapable for creating new instances, registering the PersistenceCapable class with the runtime, and managing minimal "reflective" metadata for the runtime (managed field names and types).

Added chapters for JDOHelper and JDOImplHelper.

C.11 Changes since draft 0.92

PersistenceManager methods that take a collection or array of instances have been changed to include All in their names.

Text throughout the document has been clarified to refer to the specific exception thrown.

Corrected sample code generated by the enhancer.

Added PersistenceManagerFactory methods getPersistenceManager(String userid, String password).

Static fields for values of jdoFlags were added to the PersistenceCapable interface.

A new ELEMENT array was added to the XML metadata to specify for array types whether the elements are embedded or not.

Clarified the possible treatment of jdoFlags by the StateManager, and the handling of isLoaded.

Added methods PersistenceManager.getTransactionalObjectId, PersistenceCapable.jdoGetTransactionalObjectId, and JDOHelper.getTransactionalObjectId to cover the case of changing primary key in a transaction.

Changed the requirement for a compliant implementation to support all Collection types. The behavior of all Collection types is specified, but only Collection, Set, and HashSet are required.

Clarified the semantics of getObjectId with the validate flag set to true when the instance is in the cache, for the cases of transactional v. nontransactional instances.

Changed failedObjectArray to failedObject, and nestedException to nestedExceptionArray in JDOException.

C.12 Changes since draft 0.93

Removed the requirement for application identity key classes to implement equals for all object types that include the correct name and type fields.

Changed the state transition of persistent-deleted to be unchanged by refresh.

Added a generated constructor jdoNewObjectIdInstance to facilitate key class handling.

Added a generated constructor jdoNewInstance (StateManager sm, Object oid) to facilitate key class handling.

Added generated jdoCopyKeyFieldsToObjectId methods to facilitate key class handling.

Added nested interface ObjectIdFieldManager to facilitate key class handling.

Added PersistenceManagerFactory properties ConnectionFactory2 and ConnectionFactory2Name for application server optimistic transaction support.

Added loadFactor to the newCollectionInstance method.

Clarified handling of getObjectId, getObjectById, and validate.

Added methods close(Iterator) and closeAll() to Extent.

Added methods close (Object queryResult) and closeAll() to Query.

Updated EJB chapter to clarify life cycle changes.

Removed inverse from XML metadata.

Corrected some code examples in reference enhancer.

Added methods to support different query languages: PersistenceManager.newQuery (String language, Object query) and Set supportedQueryLanguages().

Added nested extensions, and package extensions to xml.

C.13 Changes since draft 0.94

Added PersistenceManager and PersistenceManagerFactory methods to support the Multithreaded property. This property indicates that the application is multithreaded (multiple threads will access instances managed by the PersistenceManager).

Removed the PersistenceCapable constructor that takes StateManager as an argument. The helper methods newInstance will use the default constructor instead, and will create protected default constructor if none exists.

Removed jdoVersionUID and replaced it with explicit byte[] jdoFieldFlags and Class jdoPersistenceCapableSuperclass.

Added static fields to define values for jdoFieldFlags elements.

Added a chapter on JDOPermission.

Added optional extension element to xml elements array, collection, and map.

Added Multithreaded property to PersistenceManager which indicates whether the PersistenceManager must synchronize accesses from multiple application threads.

Added allowNulls parameter to PersistenceManager newMapInstance.

Changed the name of the method getJDOImplHelper to getInstance.

Clarified the handling of abstract classes, which might be PersistenceCapable (for the benefit of concrete subclasses).

Removed the requirement for implementations to track modifications made to arrays.

Removed method getProperties from PersistenceManager. This method now is in PersistenceManagerFactory only.

Removed supportedQuery from PersistenceManager. This method has been replaced by supportedOptions, from which supported query languages should be available.

Added a method supportedOptions to PersistenceManagerFactory for the application to determine which optional features are supported by an implementation.

Added query BNF chapter.

Index

Index

Index

Forte Tools
901 San Antonio Road MS MPK16-304
Palo Alto, CA 94303

For U.S. Sales Office locations, call:
800 821-4643
In California:
800 821-4642

Australia: (02) 844 5000
Belgium: 32 2 716 7911
Canada: 416 477-6745
Finland: +358-0-525561
France: (1) 30 67 50 00
Germany: (0) 89-46 00 8-0
Hong Kong: 852 802 4188
Italy: 039 60551
Japan: (03) 5717-5000
Korea: 822-563-8700
Latin America: 415 688-9464
The Netherlands: 033 501234
New Zealand: (04) 499 2344
Nordic Countries: +46 (0) 8 623 90 00
PRC: 861-849 2828
Singapore: 224 3388
Spain: (91) 5551648
Switzerland: (1) 825 71 11
Taiwan: 2-514-0567
UK: 0276 20444

Elsewhere in the world,
call Corporate Headquarters:
415 960-1300
Intercontinental Sales: 415 688-9000